

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



HCMUTE
ĐỒ ÁN TỐT NGHIỆP

HỆ THỐNG ĐIỀU KHIỂN ĐÈN TÍN HIỆU GIAO THÔNG TỰ ĐỘNG DÙNG EDGE AI

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

Sinh viên: **NGUYỄN CHU BẢO**

MSSV: 22161220

NGUYỄN HOÀNG ANH TÂN

MSSV: 22161316

Hướng dẫn: **THS. Huỳnh Thị Thu Hiền**

TP. HỒ CHÍ MINH – 05/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



HCMUTE
ĐỒ ÁN TỐT NGHIỆP

HỆ THỐNG ĐIỀU KHIỂN ĐÈN TÍN HIỆU GIAO THÔNG TỰ ĐỘNG DÙNG EDGE AI

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

Sinh viên: **NGUYỄN CHU BẢO**

MSSV: 22161220

NGUYỄN HOÀNG ANH TÂN

MSSV: 22161316

Hướng dẫn: **THS. Huỳnh Thị Thu Hiền**

TP. HỒ CHÍ MINH – 05/2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Hệ thống điều khiển đèn tín hiệu giao thông tự động

đúng EDGE AD

Sinh viên: + Nguyễn Chu Bảo

MSSV: 22161220

+ Nguyễn Hoàng Anh Tân

MSSV: 22161316

Hướng dẫn: T.H.S. Huỳnh Thị Thu Hiền

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề rõ ràng, mục tiêu cụ thể, đề tài có tính mới, cấp thiết, có tính sáng tạo.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phương pháp hợp lý, phân tích đánh giá phù hợp.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/ kiểm định thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Kết quả có phân tích, đánh giá, kiểm định hợp lý.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Kết luận phù hợp.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Bố cục hợp lý, cấu trúc rõ ràng.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Có kỹ năng làm việc tốt.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Có trích dẫn tài liệu.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Đã kiểm tra trên mạng, không trùng lặp.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Có thái độ tinh thần làm việc nghiêm túc.

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo đạt, được phép bảo vệ.

Tp. HCM, ngày 26 tháng 06 năm 2022

Người nhận xét
(Ký và ghi rõ họ tên)

Huỳnh Thị Thu Hiền

* Giáo viên hướng dẫn có thể ghi tiếp vào mặt sau

PHIẾU GIAO NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên sinh viên: Nguyễn Chu Bảo	MSSV: 22161220
Ngành: Công nghệ Kỹ thuật Điện tử - Viễn thông.	Lớp: 22161VTVM2
Hệ đào tạo: Đại học chính quy	Khóa: 2022
Họ và tên sinh viên: Nguyễn Hoàng Anh Tân	MSSV: 22161316
Ngành: Công nghệ Kỹ thuật Điện tử - Viễn thông.	Lớp: 22161VTVM2
Hệ đào tạo: Đại học chính quy	Khóa: 2022

2. Thông tin đề tài

Tên đề tài: Hệ thống điều khiển đèn tín hiệu giao thông tự động dùng Edge AI

Ngày nhận đề tài: Ngày 02/03/2026 Ngày nộp đề tài: Ngày 26/06/2026

Thời gian bảo vệ: Ngày 03/07/2026

3. Lời cam đoan của sinh viên

Chúng em xin cam đoan đồ án tốt nghiệp với đề tài "Hệ thống điều khiển đèn tín hiệu giao thông tự động dùng Edge AI " là công trình nghiên cứu, thiết kế và thực nghiệm của riêng nhóm chúng em, được thực hiện dưới sự hướng dẫn khoa học của Ths. Huỳnh Thị Thu Hiền.

Các số liệu, kết quả đo đạc, thiết kế phần cứng, mã nguồn phần mềm và mô hình trí tuệ nhân tạo được trình bày trong đồ án là hoàn toàn trung thực, khách quan và chưa từng được công bố trong bất kỳ công trình nghiên cứu nào khác.

Mọi nguồn tài liệu tham khảo, các bài báo khoa học, datasheet linh kiện cũng như các thư viện mã nguồn mở được sử dụng để hỗ trợ cho đồ án đều đã được chúng em trích dẫn rõ ràng, minh bạch và ghi nhận nguồn gốc đầy đủ trong phần Tài liệu tham khảo.

TP. Hồ Chí Minh, ngày 28 tháng 05 năm 2026

Người thực hiện đề tài

Nguyễn Chu Bảo

Nguyễn Hoàng Anh Tân

LỜI CẢM ƠN

Đồ án tốt nghiệp là một cột mốc mang tính quyết định trong quá trình học tập, rèn luyện và nghiên cứu chuyên sâu của em tại khoa Điện – Điện tử, Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh. Đây không chỉ là cơ hội lớn để em tổng hợp và củng cố toàn bộ kiến thức chuyên ngành Điện tử – Viễn thông, mà còn là thử thách thực tế giúp em tiếp cận sâu sắc với các công nghệ hiện đại bao gồm: Trí tuệ nhân tạo, kỹ thuật thiết kế phần cứng vi mạch đa lớp trên nền tảng Altium Designer và tối ưu hóa hệ thống nhúng thời gian thực. Việc hoàn thành đề tài này đã giúp em hình thành tư duy hệ thống mạch lạc, nâng cao khả năng giải quyết các bài toán phần cứng và thuật toán phức tạp, từ đó tạo nền tảng vững chắc để em tự tin bước vào môi trường làm việc thực tế trong tương lai.

Chúng em xin chân thành cảm ơn quý thầy cô khoa Điện – Điện tử đã tạo điều kiện học tập tốt nhất cho chúng em. Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc nhất đến THS. Huỳnh Thị Thu Hiền, người đã trực tiếp dẫn dắt, dành nhiều thời gian quý báu để đưa ra những định hướng, đồng thời tận tình chỉ dẫn giúp em vượt qua các rào cản kỹ thuật phức tạp trong suốt quá trình triển khai thi công mạch và tối ưu hóa mô hình.

Với sự hỗ trợ tận tình của Thầy và quý thầy cô, chúng em đã hoàn thành đồ án với đề tài: “Hệ thống điều khiển đèn tín hiệu giao thông tự động dùng Edge AI”

Mặc dù đã dành nhiều tâm huyết và nỗ lực để hoàn thiện mô hình, nhưng do kinh nghiệm thực tế và thời gian nghiên cứu còn những giới hạn nhất định, đồ án chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp và chỉ bảo quý báu từ quý thầy cô để đồ án được hoàn thiện hơn, cũng như giúp chúng em mở rộng thêm kiến thức chuyên môn.

Một lần nữa, em xin trân trọng cảm ơn!

TP. Hồ Chí Minh, ngày 28 tháng 05 năm 2026

Người thực hiện đề tài

Nguyễn Chu Bảo

Nguyễn Hoàng Anh Tân

LỜI CAM ĐOAN

Chúng em xin cam đoan rằng toàn bộ nội dung trong đề án tốt nghiệp với đề tài: “Hệ thống điều khiển đèn tín hiệu giao thông tự động dùng Edge AI” là kết quả của quá trình học tập, nghiên cứu độc lập và thực nghiệm thực tế của chính bản thân em/chúng em, dưới sự hướng dẫn khoa học, tận tình của THS. Huỳnh Thị Thu Hiền.

Các số liệu thực nghiệm, sơ đồ mạch nguyên lý, bản vẽ thiết kế mạch in, mã nguồn chương trình xử lý nhận diện AI và Firmware điều khiển hệ thống được trình bày trong quyển báo cáo này đều do chúng em trực tiếp tính toán, thiết kế, thi công linh kiện và kiểm chứng trên mô hình thực tế. Các nguồn tài liệu tham khảo, các thư viện mã nguồn mở hoặc các kiến thức kế thừa đều được chúng em trích dẫn rõ ràng, trung thực và đầy đủ theo đúng quy định quy chuẩn học thuật của nhà trường.

Chúng em xin hoàn toàn chịu trách nhiệm về tính trung thực, tính nguyên bản của nội dung báo cáo này. Nếu có bất kỳ sự trùng lặp, sao chép trái phép hoặc gian lận học thuật nào, chúng em xin hoàn toàn chịu mọi hình thức kỷ luật từ phía nhà trường và Khoa Điện – Điện tử.

Em xin chân thành cảm ơn!

TP. Hồ Chí Minh, ngày 28 tháng 05 năm 2026

Người thực hiện đề tài

Nguyễn Chu Bảo

Nguyễn Hoàng Anh Tân

TÓM TẮT ĐỀ TÀI

Trong bối cảnh đô thị hóa, lưu lượng phương tiện tại các đô thị lớn ngày càng tăng cao. Tuy nhiên, hệ thống đèn giao thông hiện nay chủ yếu hoạt động theo chu kỳ cố định hoặc thủ công, khiến thời gian đèn không phản ánh đúng số lượng xe thực tế. Điều này dẫn đến nghịch lý đường vắng vẫn phải chờ đèn đỏ lâu, trong khi đường đông lại không được cấp đủ thời gian đèn xanh, dễ gây ùn tắc giao thông và lãng phí thời gian của người dân.

Xuất phát từ thực trạng đó, em thực hiện đồ án tốt nghiệp với đề tài “Hệ thống điều khiển đèn tín hiệu giao thông tự động dùng Edge AI” nhằm tự động tối ưu hóa chu kỳ đèn theo mật độ giao thông thực tế. Đề tài tập trung vào việc nghiên cứu và xây dựng một hệ thống điều tiết tín hiệu tự động, sử dụng máy tính nhúng Raspberry Pi 5 làm bộ xử lý kết nối với camera để thu thập dữ liệu và đếm số lượng phương tiện giao thông. Điểm nổi bật của đề tài nằm ở cơ chế xử lý và điều khiển đồng bộ đa tầng: từ việc phân tích mật độ xe bằng mạng nơ-ron tích chập tự thiết kế riêng và đếm số lượng xe bằng mô hình học sâu YOLO theo thời gian thực, truyền lệnh điều khiển xuống vi điều khiển ESP32 để vận hành cụm đèn giao thông và đồng bộ thời gian đếm ngược thời gian của module LED 7 đoạn, cho đến khả năng giám sát trực quan tài nguyên hệ thống qua màn hình oled.

Toàn bộ hệ thống được tích hợp gọn gàng trên bo mạch phần cứng tự thiết kế, đảm bảo khả năng vận hành liên tục và ổn định. Kết quả thực nghiệm cho thấy mô hình hoạt động chính xác, đồng bộ, đáp ứng tốt yêu cầu thực tiễn của hai làn đường giao nhau trong giao thông.

MỤC LỤC

LỜI CẢM ƠN	i
LỜI CAM ĐOAN	ii
TÓM TẮT ĐỀ TÀI	iii
MỤC LỤC	iv
DANH MỤC ẢNH.....	vii
DANH MỤC BẢNG.....	ix
DANH SÁCH TỪ VIẾT TẮT	x
CHƯƠNG 1 GIỚI THIỆU	1
1.1 Giới thiệu	1
1.2 Mục tiêu đề tài	2
1.3 Giới hạn đề tài.....	2
1.4 Phương pháp nghiên cứu	3
1.5 Đối tượng và phạm vi nghiên cứu	4
1.5.1 Đối tượng nghiên cứu.....	4
1.5.2 Phạm vi nghiên cứu.....	4
1.6 Bố cục quyền báo cáo	5
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT.....	7
2.1 Thị giác máy tính và mạng nơ-ron tích chập.....	7
2.2 Kiến trúc phát hiện phương tiện tối ưu cho thiết bị nhúng-Yolov26n	8
2.2.1 Cải tiến nổi bật từ Yolov26	9
2.2.2 Framework NCNN và triển khai trên Raspberry Pi 5	11
2.3 Cơ chế chú ý thị giác và giới thiệu khối GLKA (Global Large Kernel Attention)	12
2.3.1 GLKA – Global Large Kernel Attention.....	12
2.3.2 SENet – Squeeze-and-Excitation Network	16
2.3.3 Cơ chế chú ý không gian nhân lớn.....	17
2.4 Tái tham số hóa cấu trúc.....	18
2.4.1 Hợp nhất chuẩn hóa Batch Normalization vào tích chập.....	18
2.4.2 Chuyển nhân giãn về không gian thống nhất	18
2.5 Kiến trúc mạng nơ-ron tích chập siêu nhẹ – Simple_GLKA	19

2.5.1 Tích chập tách rời theo chiều sâu làm đơn vị cơ bản.....	20
2.5.2 Khối Inverted Residual với tỉ lệ mở rộng giảm thiểu.....	21
2.5.3 Phân bổ kênh đơn điệu và đầu phân loại tối giản.....	21
2.5.4 Ứng dụng vào Simple_GLKA.....	22
2.6 Tổng quan giao thức UART	24
2.7 Tổng quan về Ethernet.....	25
CHƯƠNG 3 THIẾT KẾ HỆ THỐNG	27
3.1 Mô tả hệ thống	27
3.2 Sơ đồ khối tổng quát.....	28
3.3 Thiết kế chi tiết từng khối.....	29
3.3.1 Khối nguồn	29
3.3.2 Khối nhận dữ liệu	30
3.3.3 Khối xử lý trung tâm	32
3.3.4 Khối điều khiển	33
3.3.5 Khối hiển thị.....	36
3.3.6 Khối thực thi.....	37
3.4 Sơ đồ nguyên lý	38
3.4.1 Sơ đồ nguyên lý trung tâm điều khiển Raspberry Pi 5.....	38
3.4.2 Sơ đồ nguyên lý mạch điều khiển ESP32	39
3.5 Thiết kế phần mềm	39
3.5.1 Thiết lập môi trường huấn luyện cho Yolov26n và Simple_GLKA....	39
3.5.2 Thuận toán huấn luyện	40
3.5.3 Quy trình xử lý dữ liệu	41
3.5.4 Triển khai hệ thống trên Raspberry Pi 5	42
3.6 Lưu đồ giải thuật hệ thống giao thông.....	43
3.6.1 Xây dựng lưu đồ giải thuật hệ thống.....	43
3.6.2 Lưu đồ giải thuật khối điều khiển	47
CHƯƠNG 4 KẾT QUẢ VÀ ĐÁNH GIÁ	52
4.1 Kết quả đánh giá độ chính xác dựa trên số liệu huấn luyện	52
4.1.1 Kết quả đánh giá với tiền xử lý ảnh	52
4.1.2 Phân loại mật độ giao thông (Simple_GLKA).....	53

4.1.3 Kết quả huấn luyện YOLOv26n.....	58
4.2 Hiệu năng xử lý và tiêu thụ năng lượng theo thời gian thực	60
4.3 Năng lượng tiêu thụ của hệ thống.....	63
4.4 Kết quả thi công phần cứng	64
4.4.1 Kết quả thi công mạch nguồn Raspberry Pi 5.....	64
4.4.2 Kết quả thi công mạch điều khiển ESP32	66
4.5 Kết quả hoạt động của hệ thống giao thông.	67
4.5.1 Kết quả vận hành ở chế độ tự động thích ứng (MODE m0 - m4).....	68
4.5.2 Kết quả vận hành ở chế độ ưu tiên	72
CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	74
5.1 Kết luận.....	74
5.1.1 Ưu điểm	75
5.1.2 Nhược điểm	75
5.2 Hướng phát triển	76
Tài Liệu Tham Khảo	77

DANH MỤC ẢNH

Hình 2.1: Cấu trúc Yolov26 dựa trên cấu trúc chính Utralytics	9
Hình 2.2: Bảng so sánh các định dạng chạy của Yolov26 từ nhà phát triển Utralytics [8]	12
Hình 2.3: Cấu trúc khối GLKA (Global Larger Kernel Attention)	13
Hình 2.4: Chú ý không gian đa nhánh với kỹ thuật tái tham số hóa.....	15
Hình 2.5: Cấu trúc khối SE [1].....	16
Hình 2.6: Cấu trúc SimpleEfficientCNN [19]	20
Hình 2.7: Cấu trúc khối Effcient Block gốc.....	21
Hình 2.8: Cấu trúc tổng quát của mạng tích chập Simple_GLKA	23
Hình 2.9: Cấu trúc khối GLKA_3 và GLKA_2+4.....	23
Hình 2.10: Tổng quan về UART.....	24
Hình 3.1: Sơ đồ khối của hệ thống.....	28
Hình 3.2: Sơ đồ nguyên lý mạch nguồn 3.3V	29
Hình 3.3: Sơ đồ nguyên lý mạch nguồn cho Raspberry Pi 5	30
Hình 3.4: Module camera Astra Pro	31
Hình 3.5: Kiến trúc SCI với Enhance Network (trái), Calibrate Network (giữa, chỉ dùng khi huấn luyện) và hàm mất mát tổng hợp. Khi triển khai, chỉ nhánh Enhance Network được giữ lại.....	32
Hình 3.6: Sơ đồ chân máy tính nhúng Raspberry Pi 5.....	33
Hình 3.7: Sơ đồ chân và chi tiết chức năng các chân	34
Hình 3.8: Sơ đồ kết nối chân chip ESP32	35
Hình 3.9: Sơ đồ nguyên lý khối chuyển đổi USB to UART.....	36
Hình 3.10: Sơ đồ nguyên lý kết nối module led 7 đoạn với ESP32	36
Hình 3.11: Sơ đồ kết nối chân màn hình OLED	37
Hình 3.12: Sơ đồ kết nối led giao thông với ESP32	37
Hình 3.13: Sơ đồ nguyên lý mạch nguồn 5V/5A.....	38
Hình 3.14: Sơ đồ nguyên lý tổng thể của mạch điều khiển ESP32	39
Hình 3.15: Quy trình xử lý dữ liệu.....	41
Hình 3.16: Lưu đồ giải thuật chương trình chính trên Raspberry Pi 5.	43
Hình 3.17: Lưu đồ giải thuật tổng quan Raspberry Pi 5 ở chế độ song song	44
Hình 3.18: Lưu đồ giải thuật tổng quan Raspberry Pi 5 ở chế độ độc lập.....	45
Hình 3.19: Lưu đồ giải thuật tiền xử lý ảnh.....	46
Hình 3.20: Lưu đồ giải thuật quyết định chế độ thời gian đèn	47
Hình 3.21: Lưu đồ giải thuật ESP32	48
Hình 3.22: Lưu đồ thuật toán xử lý đèn xanh (ưu tiên)	49
Hình 3.23: Lưu đồ thuật toán xử lý đèn đỏ (ép pha).....	50
Hình 3.24: Lưu đồ thuật toán tra bảng của ESP32.....	51

Hình 4.1: So sánh kết quả nhận diện phương tiện trong điều kiện ánh sáng yếu (độ sáng 0.2)	52
Hình 4.2: So sánh kết quả nhận diện phương tiện trong điều kiện ánh sáng yếu (độ sáng 0.3)	53
Hình 4.3: Bản đồ Grad-CAM của đặc trưng các mô hình biến thể.....	55
Hình 4.4: Tốc độ thực thi các biến thể trên GPU (RTX4050)	56
Hình 4.5: Tốc độ thực thi cấu trúc Simple_GLKA trên CPU.....	57
Hình 4.6: Kết quả huấn luyện Yolov26n	58
Hình 4.7: Ma trận nhầm lẫn của mô hình sau huấn luyện	59
Hình 4.8: Thời gian xử lý một chu kì của Raspberry Pi 5	60
Hình 4.9: Thời gian xử lý một chu kì.....	61
Hình 4.10: Điện năng tiêu thụ của Raspberry Pi5 trong 24h	63
Hình 4.11: Mô hình đèn giao thông ngã tư	64
Hình 4.12: Board mạch nguồn cấp riêng cho Raspberry	65
Hình 4.13: Board mạch điều khiển giao thông	66
Hình 4.14: Hệ thống ở trạng thái m0	68
Hình 4.15: Hệ thống ở trạng thái m1	69
Hình 4.16: Hệ thống ở trạng thái m2	69
Hình 4.17: Kết quả nhận diện phương tiện ở mode m2	70
Hình 4.18: Hệ thống ở trạng thái m3	70
Hình 4.19: Kết quả nhận diện phương tiện ở mode m3.....	71
Hình 4.20: Hệ thống ở trạng thái m4	71
Hình 4.21: Hệ thống ở chế độ ưu tiên	72

DANH MỤC BẢNG

Bảng 2.1 :Cấu hình phân cấp vùng cảm nhận (EFR) đa quy mô của các nhánh .	14
Bảng 3.1: Thông số kỹ thuật ESP32	34
Bảng 4.1: Bảng số liệu các biến thể và so sánh các phiên bản của Simple_GLKA.	54
Bảng 4.2: Thống kê chi tiết các chỉ số qua các mốc epoch tiêu biểu.....	59
Bảng 4.3: Thống kê độ ổn định của các thiết bị.....	62
Bảng 4.4: Kết quả kiểm tra và đánh giá mạch nguồn.	65
Bảng 4.5: Kết quả kiểm tra và đánh giá của mạch điều khiển giao thông.....	66
Bảng 4.6: Các chế độ vận hành của hệ thống giao thông.	67

DANH SÁCH TỪ VIẾT TẮT

ADC	Analog to Digital Converter	Bộ chuyển đổi tương tự sang số
AI	Artificial Intelligence	Trí tuệ nhân tạo
ARM	Advanced RISC Machine	Kiến trúc bộ vi xử lý nhúng
BN	Batch Normalization	Chuẩn hóa theo lô trong mạng nơ-ron
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
CPU	Central Processing Unit	Bộ xử lý trung tâm
CUDA	Compute Unified Device Architecture	Nền tảng tính toán song song của NVIDIA
DW	Depthwise Convolution	Tích chập theo chiều sâu
ERF	Effective Receptive Field	Vùng tiếp nhận hiệu dụng
ESP32	-	Vi điều khiển do Espressif Systems phát triển
FPS	Frames Per Second	Số khung hình xử lý mỗi giây
GAP	Global Average Pooling	Gộp trung bình toàn cục
GLKA	Global Large Kernel Attention	Module chú ý nhân lớn toàn cục
GPIO	General Purpose Input/Output	Chân vào/ra đa dụng
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
Grad-CAM	Gradient-weighted Class Activation Mapping	Bản đồ nhiệt trực quan hóa đặc trưng
I2C	Inter-Integrated Circuit	Giao thức truyền thông nối tiếp hai dây
IC	Integrated Circuit	Mạch tích hợp
IoU	Intersection over Union	Chỉ số đánh giá độ chồng lấp bounding box
JST	Japan Solderless Terminal	Chuẩn giắc cắm điện tử nhỏ
LED	Light Emitting Diode	Đi-ốt phát quang

LKA	Large Kernel Attention	Module chú ý nhân lớn
LPDDR4	Low Power Double Data Rate 4	Loại RAM tiết kiệm năng lượng
LSB	Least Significant Bit	Bit có trọng số thấp nhất
mAP	Mean Average Precision	Độ chính xác trung bình của mô hình
MSB	Most Significant Bit	Bit có trọng số cao nhất
NCNN	Neural network inference framework	Nền tảng suy luận mô hình trên thiết bị nhúng
NMS	Non-Maximum Suppression	Thuật toán loại bỏ bounding box trùng lặp
OLED	Organic Light Emitting Diode	Màn hình đi-ốt phát quang hữu cơ
ONNX	Open Neural Network Exchange	Định dạng mô hình học sâu mở
PCB	Printed Circuit Board	Bộ mạch in
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
ROI	Region of Interest	Vùng quan tâm trong ảnh
SCI	Self-Calibrated Illumination	Mô hình tăng cường ánh sáng tự hiệu chỉnh
SoC	System on Chip	Hệ thống tích hợp trên một chip
TCP	Transmission Control Protocol	Giao thức truyền thông mạng có kiểm soát
UART	Universal Asynchronous Receiver–Transmitter	Giao thức truyền thông nối tiếp
USB	Universal Serial Bus	Chuẩn giao tiếp nối tiếp
VRAM	Video Random Access Memory	Bộ nhớ truy cập ngẫu nhiên của GPU
YOLO	You Only Look Once	Thuật toán phát hiện đối tượng một lần quét

CHƯƠNG 1

GIỚI THIỆU

1.1 Giới thiệu

Trong bối cảnh đô thị hóa diễn ra nhanh chóng, lưu lượng phương tiện tại các thành phố lớn tăng cao khiến tình trạng ùn tắc giao thông trở nên phổ biến và ngày càng nghiêm trọng. Tuy nhiên, phần lớn hệ thống đèn giao thông hiện nay vẫn hoạt động theo chu kỳ cố định, nghĩa là thời gian đèn xanh và đèn đỏ là cố định và lặp lại liên tục mà không xét đến mức độ đông – vắng của từng tuyến đường. Cách vận hành này dẫn đến nhiều bất cập, chẳng hạn như các tuyến đường vắng vẫn phải chờ đèn đỏ trong một khoảng thời gian không cần thiết, trong khi những tuyến đường đông lại không được cấp đủ thời gian đèn xanh, gây nên ùn tắc kéo dài và làm giảm hiệu quả điều phối giao thông. Trong nhiều trường hợp ùn ứ phức tạp, việc điều khiển đèn giao thông phải chuyển sang hình thức thủ công khi lực lượng chức năng trực tiếp đứng tại giao lộ để điều tiết, vừa tốn nhân lực, vừa không bảo đảm tính ổn định và khó phản ứng kịp thời với tình hình thực tế.

Trong khi hệ thống điều khiển hiện tại còn nhiều hạn chế, những năm gần đây, công nghệ camera tích hợp trí tuệ nhân tạo đang phát triển mạnh mẽ và thu hút sự quan tâm lớn của các nhà nghiên cứu cũng như các đô thị thông minh. Camera AI có khả năng phân tích hình ảnh giao thông theo thời gian thực, nhận diện và đếm số lượng phương tiện với độ chính xác cao, mở ra hướng tiếp cận mới cho việc tối ưu hóa điều khiển đèn giao thông mà không cần phụ thuộc vào các cảm biến vật lý đắt tiền. Xu hướng áp dụng AI vào camera đang ngày càng trở nên phổ biến và được xem là một hướng phát triển tất yếu trong lĩnh vực giao thông.

Nhận thấy tiềm năng lớn của công nghệ này, đồng thời mong muốn tìm hiểu và đánh giá khả năng ứng dụng thực tế của AI trong bài toán điều khiển lưu lượng giao thông, nhóm quyết định lựa chọn đề tài “Hệ thống điều khiển đèn tín hiệu giao thông tự động dùng Edge AI”. Đề tài không chỉ mang ý nghĩa học thuật mà còn có tính ứng dụng cao, phù hợp để triển khai trên mô hình mô phỏng và có thể mở rộng thành giải pháp áp dụng cho hệ thống thật trong tương lai.

1.2 Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là nghiên cứu, thiết kế phần cứng và xây dựng hoàn thiện mô hình hệ thống điều khiển đèn giao thông có khả năng tự động điều chỉnh đèn giao thông dựa vào lưu lượng xe lưu thông trên đường thực tế. Để đạt được mục tiêu tổng quát trên, các mục tiêu cụ thể của đề tài bao gồm:

- Thiết kế và thi công: Xây dựng hệ thống kết hợp giữa máy tính nhúng Raspberry Pi 5 dùng để thu thập xử lý hình ảnh và vi điều khiển ESP32 đảm nhiệm vai trò điều khiển đèn giao thông.
- Phân loại mật độ và nhận diện phương tiện bằng AI: Nhúng mô hình AI siêu nhẹ Simple_GLKA và YOLOv26 trực tiếp lên Raspberry Pi 5 nhằm tự động phát hiện, phân loại mật độ và đếm số lượng xe.
- Tối ưu hóa chu kỳ đèn tín hiệu: Xây dựng thuật toán điều khiển dựa trên phương pháp tra bảng với các kịch bản định sẵn. Dựa vào kết quả đếm xe, hệ thống sẽ phân loại mật độ giao thông (đường vắng, bình thường, khá đông hoặc đông đúc) và tự động trích xuất thời gian đèn tương ứng, giúp giảm thiểu tối đa thời gian dừng chờ không cần thiết tại các ngã tư.

1.3 Giới hạn đề tài

Để phù hợp với nguồn lực và thời gian thực hiện đồ án, đề tài tự giới hạn trong các khuôn khổ kỹ thuật sau:

- Về thu thập và xử lý hình ảnh: Hệ thống chỉ nhận diện và đếm xe trên một làn đường duy nhất qua một camera cố định; không áp dụng cho giao lộ đa hướng hay hệ thống đa camera.
- Về giải thuật AI: Chỉ trích xuất và xử lý một khung hình duy nhất tại hai thời điểm cố định (khi đèn đỏ còn 5 giây và đèn xanh còn 2 giây) để tính chu kỳ đèn tiếp theo; không sử dụng thuật toán bám đuổi đối tượng liên tục (Tracking).
- Về kết nối và truyền thông: Giới hạn truyền thông liên trạm nội bộ giữa hai máy tính nhúng qua mạng Ethernet cục bộ và truyền chéo UART xuống vi điều khiển; không tích hợp bản đồ giao thông thực tế.

- Về tiêu chuẩn điều khiển: Dừng lại ở quy mô mô hình sơ bản thực nghiệm để chứng minh nguyên lý; không thiết kế mạch nguồn dự phòng hoặc áp dụng các giao thức công nghiệp phức tạp.

1.4 Phương pháp nghiên cứu

Phương pháp nghiên cứu tài liệu: Thu thập và phân tích các tài liệu kỹ thuật từ sách, tạp chí khoa học chuyên ngành Điện tử, Viễn thông và các mô hình AI. Tham khảo các nguồn tài liệu chính thống trên Internet về máy tính nhúng Raspberry Pi 5, vi điều khiển ESP32, mô hình học sâu YOLO v26 trong bài toán nhận diện đối tượng đối tượng, các chuẩn giao tiếp dữ liệu nối tiếp (UART, I2C) và các thuật toán tối ưu hóa tài nguyên khi chạy mô hình AI trên máy tính nhúng.

Phương pháp phân tích và thiết kế hệ thống: Dựa trên yêu cầu thực tiễn về giải quyết bài toán tuần tắc giao thông tại các nút giao, thực hiện phân tích và chia hệ thống thành các khối chức năng chuyên biệt bao gồm: khối xử lý trí tuệ nhân tạo thu thập dữ liệu hình ảnh, khối điều khiển chấp hành, khối hiển thị tín hiệu và khối bo mạch nguồn phân phối năng lượng nhằm tối ưu hóa khả năng tích hợp và vận hành ổn định.

Phương pháp lập trình và xây dựng hệ thống: Sử dụng ngôn ngữ lập trình Python kết hợp thư viện tối ưu hóa để triển khai mô hình nhận diện, phân loại và đếm phương tiện theo thời gian thực; kết hợp với việc lập trình nhúng xây dựng các thuật toán điều khiển và tra cứu bảng cấu hình thời gian chuyển pha đèn linh hoạt trên vi điều khiển.

Phương pháp thực nghiệm: Sau khi hoàn thiện thiết kế logic và mã nguồn, dưới sự hướng dẫn của giảng viên, hệ thống được tiến hành thiết kế nguyên lý, Layout PCB trên phần mềm Altium Designer và thi công lắp ráp mô hình phần cứng thực tế. Thực hiện kết nối giữa camera, bo mạch xử lý trung tâm Raspberry Pi 5, vi điều khiển ESP32 và hệ thống đèn LED tín hiệu, khối hiển thị để kiểm tra khả năng phối hợp, đồng bộ giữa phần cứng và phần mềm.

Phương pháp đánh giá và tối ưu hóa: Tiến hành thử nghiệm hệ thống trong các kịch bản giả lập lưu lượng phương tiện và điều kiện ánh sáng khác nhau để đánh giá độ chính xác (FPS, tỷ lệ nhận diện) của mô hình AI, đo lường độ trễ trong quá

trình truyền nhận dữ liệu và khả năng giảm ùn tắc giao thông thực tế của thuật toán tự động điều chỉnh đèn so với hệ thống đèn thông thường. Từ đó, thực hiện điều chỉnh các tham số cấu hình và tối ưu hóa chương trình để đảm bảo hệ thống vận hành với độ tin cậy và tính ổn định cao nhất.

1.5 Đối tượng và phạm vi nghiên cứu

1.5.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu trọng tâm của đề tài là hệ thống điều khiển giao thông thông minh ứng dụng nhận diện phương tiện bằng AI. Các thành phần chính được tập trung nghiên cứu bao gồm:

- Máy tính nhúng Raspberry Pi 5: Chạy mô hình YOLO v26 để nhận diện, phân loại và đếm số lượng phương tiện giao thông theo thời gian thực từ camera.
- Vi điều khiển ESP32: Đóng vai trò bộ điều khiển đèn giao thông bằng tín hiệu LED và đồng bộ thời gian đếm ngược lên khối hiển thị.
- Mô hình AI YOLO v26: Thuật toán học sâu được lựa chọn để huấn luyện, tối ưu hóa nhằm phát hiện và phân loại chính xác các phương tiện giao thông theo thời gian thực.
- Lưu lượng dòng xe: Xác định mật độ giao thông dựa trên số lượng xe đếm được trong một khoảng thời gian quy định để làm căn cứ tính toán thời gian chuyển pha đèn.
- Các phương tiện giao thông: Tập trung vào việc nhận diện, phân loại các nhóm xe phổ biến xuất hiện trong luồng video đầu vào bao gồm xe máy, ô tô, xe tải, xe van và xe buýt.
- Mô hình mô phỏng vật lý: Sử dụng mô phỏng nút giao thông thực tế, bao gồm làn đường chạy thử nghiệm, hệ thống đèn tín hiệu LED.

1.5.2 Phạm vi nghiên cứu

Trong giới hạn thời gian và nguồn lực của Đồ án tốt nghiệp, phạm vi nghiên cứu tập trung vào các nội dung chính:

- Về phạm vi xử lý hình ảnh, hệ thống chỉ tập trung nhận diện và đếm số lượng phương tiện trên một làn đường duy nhất, không mở rộng áp dụng

cho toàn bộ giao lộ nhiều hướng phức tạp. Dữ liệu hình ảnh đầu vào được thu thập thông qua duy nhất một camera đặt cố định tại một góc nhìn.

- Về cơ chế thuật toán, nhằm tối ưu hóa hiệu năng và giảm tải xử lý cho phần cứng nhúng, hệ thống chỉ trích xuất và xử lý một khung hình (frame) duy nhất trong mỗi chu kỳ khi còn mỗi 5 giây đèn đỏ và 2 giây đèn xanh để tính toán mật độ phương tiện cho pha đèn tiếp theo, hoàn toàn không sử dụng các thuật toán bám đuôi đối tượng liên tục.
- Về kiến trúc phần cứng, đề tài giới hạn ở việc sử dụng máy tính nhúng Raspberry Pi 5 làm bộ xử lý AI để phân tích hình ảnh, sau đó truyền dữ liệu điều khiển sang vi điều khiển ESP32 đóng vai trò điều khiển đèn tín hiệu.
- Bên cạnh đó, các nội dung nằm ngoài phạm vi nghiên cứu của đề tài bao gồm:
 - Không tối ưu hóa thuật toán chuyên sâu cho các bài toán mở rộng như hệ thống đa làn đường hoặc quản lý đa camera.
 - Không thiết kế hệ thống mạch điều khiển dự phòng và không áp dụng các giao thức truyền thông công nghiệp phức tạp.
 - Không tích hợp dữ liệu với hệ thống bản đồ giao thông thực tế.

1.6 Bố cục quyển báo cáo

Báo cáo được trình bày theo cấu trúc năm chương như sau:

- CHƯƠNG 1 - GIỚI THIỆU: Tổng quan về tính cấp thiết của đề tài, mục tiêu, phương pháp nghiên cứu, đối tượng và phạm vi thực hiện.
- CHƯƠNG 2 - CƠ SỞ LÝ THUYẾT: Trình bày kiến thức nền tảng bao gồm lý thuyết về thị giác máy tính, các kiến trúc mạng nơ-ron tích chập học sâu ứng dụng trong nhận diện phương tiện và tổng quan về các chuẩn giao thức truyền thông UART, Ethernet phục vụ cho giao tiếp và đồng bộ hệ thống.
- CHƯƠNG 3 - THIẾT KẾ HỆ THỐNG: Mô tả sơ đồ khối, phân tích chức năng các linh kiện, thiết kế sơ đồ nguyên lý, thiết kế phần mềm và lưu đồ giải thuật của hệ thống.

- CHƯƠNG 4 - KẾT QUẢ VÀ ĐÁNH GIÁ: Trình bày kết quả đạt được của mô hình thực tế, các kịch bản kiểm thử, đánh giá hiệu suất hoạt động và độ đồng bộ của hệ thống.
- CHƯƠNG 5 - KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN: Tổng kết những kết quả đã đạt được, phân tích ưu điểm, hạn chế và đề xuất các hướng cải tiến cho hệ thống trong tương lai.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 Thị giác máy tính và mạng nơ-ron tích chập

Thị giác máy tính (Computer Vision) là lĩnh vực nghiên cứu cho phép hệ thống tự động thu nhận, phân tích và diễn giải nội dung trực quan từ hình ảnh hoặc video. Trong bối cảnh giám sát giao thông đô thị, thị giác máy tính đóng vai trò nền tảng cho các tác vụ như phát hiện và theo dõi phương tiện, ước lượng mật độ dòng xe, nhận dạng biển số, hay điều phối pha đèn tín hiệu theo thời gian thực [1], [2].

Mạng nơ-ron tích chập (Convolutional Neural Network — CNN) là kiến trúc học sâu được thiết kế chuyên biệt để xử lý dữ liệu dạng lưới như ảnh số. Khác với các phương pháp xử lý ảnh truyền thống vốn phụ thuộc vào bộ lọc được thiết kế thủ công dựa trên tri thức chuyên gia, CNN có khả năng tự học và tinh chỉnh các đặc trưng biểu diễn trực tiếp từ dữ liệu thông qua quá trình huấn luyện có giám sát [3].

Khi một khung hình được đưa vào mô hình, ảnh đầu vào được biểu diễn dưới dạng ma trận điểm ảnh. Từ đó, các lớp tích chập sử dụng bộ lọc (Kernel) kích thước nhỏ trượt tuần tự trên toàn bộ ảnh, thực hiện phép nhân chập để sinh ra các bản đồ đặc trưng (feature map) và mỗi feature map phản ánh mức độ hiện diện của một đặc trưng cụ thể tại từng vị trí không gian trong ảnh [3]. Về mặt toán học, phép tích chập tại vị trí (i, j) được biểu diễn như sau :

$$F(i, j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m, n) \quad (2.1)$$

trong đó $F(i, j)$ là giá trị đầu ra tại vị trí (i, j) , I là ảnh đầu vào và K là kích thước của Kernel. Nhờ cơ chế này, mô hình có khả năng khuếch đại các vùng ảnh mang thông tin cấu trúc quan trọng trong khi giảm thiểu ảnh hưởng của nhiễu nền.

Điểm đáng chú ý hơn là CNN không chỉ đơn thuần trích xuất đặc trưng, mà học chúng theo cấu trúc phân cấp (hierarchical feature learning), từ đặc trưng cấp thấp đến biểu diễn ngữ nghĩa cấp cao theo chiều sâu của mạng. Ở các tầng nông, mô hình bắt đầu từ những tín hiệu nguyên thủy nhất: cạnh thẳng, góc cong, hay sự chuyển tiếp gradient độ sáng. Những mảnh rời rạc đó dần được tổ hợp lại ở các tầng trung gian thành các cấu trúc hình học có nghĩa hơn như đường viền bánh xe,

cụm đèn pha hay khung gầm xe. Đến các tầng sâu nhất, mô hình đã hình thành biểu diễn ngữ nghĩa đủ phong phú để phân biệt xe máy, ô tô con, xe tải hay xe buýt, ngay cả khi góc chụp hay tỉ lệ xuất hiện trong khung hình thay đổi đáng kể. [4]. Chính cơ chế phân cấp này là lý do CNN tổng quát hóa tốt trên dữ liệu thực tế, không bị ràng buộc vào một mẫu hình ảnh cố định.

2.2 Kiến trúc phát hiện phương tiện tối ưu cho thiết bị nhúng-Yolov26n

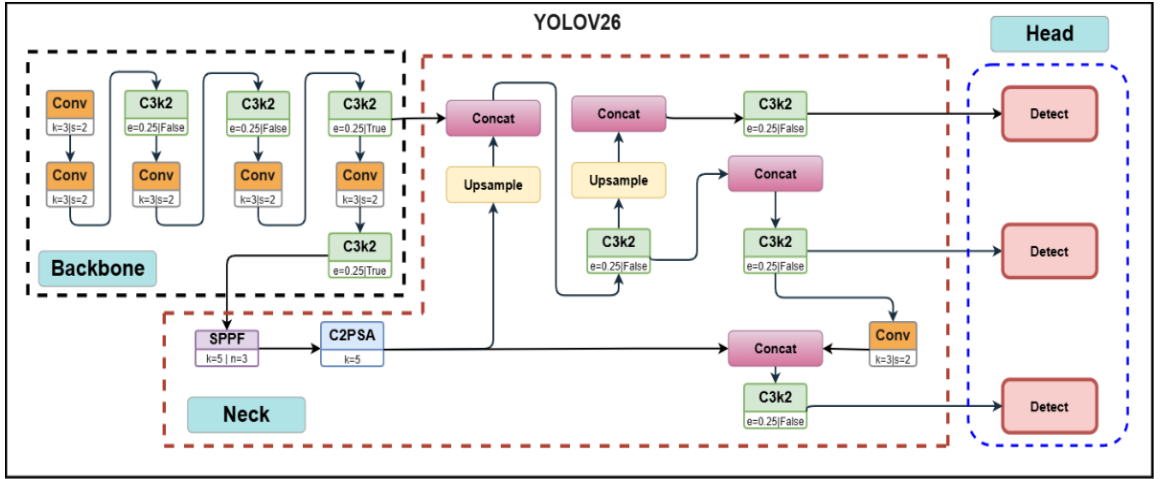
YOLO (You Only Look Once)[1] từ lâu đã được xem là một bước ngoặt lớn của ngành thị giác máy tính nhờ tư duy tiếp cận bài toán đầy thực dụng: biến toàn bộ quy trình định vị và phân lớp vật thể phức tạp thành một bài toán hồi quy duy nhất. Chính tư duy "nhìn một lần" này đã giúp hệ thống đạt được tốc độ xử lý thời gian thực mà khó có kiến trúc nào so được.

Tiếp nối thế mạnh đó, phiên bản thế hệ mới Ultralytics YOLOv26 ra đời nhưng mang theo một sứ mệnh thực tế hơn, đó là tối ưu hóa toàn diện cho các thiết bị nhúng và phần cứng công suất thấp theo triết lý "Edge-first". Bước đi đột phá nhất của YOLOv26 loại bỏ cơ chế Distribution Focal Loss (DFL) và thuật toán Non-Maximum Suppression (NMS) truyền thống, vốn là hai tác nhân chính gây ra hiện tượng sụt giảm hiệu năng khi chuyển mô hình từ môi trường huấn luyện sang triển khai thực tế. DFL làm tăng chi phí tính toán do phải ước lượng phân phối xác suất cho tọa độ hộp giới hạn, trong khi NMS yêu cầu duyệt tuần tự qua toàn bộ tập dự đoán để lọc trùng lặp, khiến quá trình suy luận khó song song hóa và tối ưu hóa trên phần cứng nhúng. Nhờ tái cấu trúc theo hướng suy luận đầu cuối (End-to-End) kết hợp chiến lược gán nhãn đơn trị, mô hình đạt tốc độ xử lý trên CPU tăng tới 43% với độ trễ ổn định và tuyến tính [5], [6], [7].

Đối với các hệ thống điều khiển giao thông thông minh sự chuyển dịch kiến trúc này mang ý nghĩa thực tiễn rõ rệt. Trong bối cảnh triển khai thực tế, hệ thống camera giám sát thường phải xử lý hai thách thức đặc thù: mật độ phương tiện biến động lớn theo thời điểm đòi hỏi tốc độ suy luận ổn định, và các phương tiện xuất hiện ở xa với kích thước nhỏ trong khung hình gây khó khăn cho quá trình phát hiện. Kiến trúc End-to-End của YOLOv26 góp phần giảm thiểu cả hai hạn chế này

thông qua việc loại bỏ các bước hậu xử lý tuần tự vốn trở thành nút thắt cổ chai khi lượng dự đoán tăng cao.

Bằng cách duy trì vận tốc phản hồi mượt mà bất kể khung hình đang trông trải hay kẹt cứng xe cộ, kết hợp với các thuật toán thích nghi thông minh như ProgLoss và STAL [6], [7], mô hình đảm bảo không bỏ sót bất kỳ phương tiện kích thước nhỏ nào ở rìa góc nhìn. Đây chính là cơ sở dữ liệu đáng tin cậy để các bộ điều khiển trung tâm phân luồng và tối ưu nhịp đèn tín hiệu theo thời gian thực. Quan trọng hơn, việc hoạt động nhẹ nhàng trên các dòng chip nhúng giá rẻ còn giúp các đô thị thông minh tối ưu hóa được một khoản chi phí khổng lồ về cả phần cứng lẫn điện năng vận hành 24/7.



Hình 2.1: Cấu trúc YOLOv26 dựa trên cấu trúc chính Ultralytics

2.2.1 Cải tiến nổi bật từ YOLOv26

Một trong những thay đổi kiến trúc đáng chú ý của YOLO26 là việc loại bỏ cơ chế Distribution Focal Loss (DFL). Kể từ YOLOv8, DFL được giữ nguyên qua các phiên bản kế tiếp đến YOLO11 với vai trò mô hình hóa tọa độ bounding box dưới dạng phân phối xác suất rời rạc [6]. Ở mỗi bước dự đoán, cơ chế này yêu cầu thực thi phép toán Softmax để tổng hợp tọa độ liên tục

$$\hat{y}_{DFL} = \sum_{i=0}^n i \cdot \text{Softmax}(w_i) = \sum_{i=0}^n i \cdot \frac{e^{w_i}}{\sum_{j=0}^n e^{w_j}} \quad (2.2)$$

Về mặt toán học, phép tính lũy thừa lặp (e^x) có chi phí tính toán cao và đòi hỏi nhiều chu kỳ xử lý hơn so với các phép toán đại số cơ bản. Khi mô hình được xuất sang các định dạng phiên bản khác nhau, sự xuất hiện liên tục của toán tử phi tuyến

Softmax này vô tình tạo ra một khoảng cách về mặt hiệu năng và làm phức tạp hóa sơ đồ tính toán của mô hình. Nhằm hóa giải điểm nghẽn này, YOLOv26 định hướng dịch chuyển hoàn toàn sang cơ chế hồi quy trực tiếp (Direct Regression) :

$$\hat{y}_{v26} = \mathcal{F}_{reg}(x) \in \mathbb{R} \quad (2.3)$$

Cơ chế lặp tuần tự này khiến thời gian xử lý phụ thuộc trực tiếp vào số lượng hộp dự đoán trong khung hình. Khi mật độ phương tiện tăng cao, khối lượng phép so sánh IoU tăng theo, dẫn đến độ trễ biến động không ổn định, đây là nút thắt cổ chai trong các ứng dụng điều khiển giao thông yêu cầu phản hồi thời gian thực. Để giải quyết vấn đề này, YOLO26 cấu trúc lại đầu dự đoán theo chiến lược gán nhãn một-một, trong đó mạng được huấn luyện để xuất ra đúng một hộp duy nhất cho mỗi đối tượng, biến quá trình suy luận thành một ánh xạ xác định và loại bỏ hoàn toàn sự phụ thuộc vào bước hậu xử lý [6].

Bên cạnh việc tinh giản cấu trúc dự đoán tọa độ, mô hình còn tái thiết kế lại toàn bộ cơ chế hậu xử lý nhằm đạt được tốc độ suy luận tối ưu. Thông thường, thuật toán NMS truyền thống loại bỏ các hộp dự đoán trùng lặp bằng cách duyệt tuần tự và so sánh chỉ số IoU [2], [6]:

$$s_i = \begin{cases} s_i & \text{if } \text{IoU}(M, b_i) < N_t \\ 0 & \text{if } \text{IoU}(M, b_i) \geq N_t \end{cases} \quad (2.4)$$

Trong đó M là hộp có độ tin cậy cao nhất và N_t là ngưỡng IoU cố định.

Tuy nhiên, quy trình lặp tuần tự này vô tình khiến thời gian xử lý (latency) bị phụ thuộc trực tiếp vào độ phức tạp của khung hình. Tại các bối cảnh có mật độ vật thể cao, khối lượng phép tính so sánh IoU tăng vọt sẽ gây ra hiện tượng trễ biến động thất thường. Để loại bỏ hoàn toàn "nút thắt" hậu xử lý này, YOLOv26 cấu trúc lại dự đoán đầu ra theo chiến lược gán nhãn đơn trị (one-to-one label assignment). Phương pháp này định hướng mạng bám học để xuất ra chính xác một hộp dứt khoát cho mỗi thực thể, chuyển đổi quá trình suy luận thành một ánh xạ xác định và giúp tối ưu hóa đáng kể tốc độ xử lý trên CPU [6].

Nhằm bù đắp sai số định vị từ việc loại DFL và ổn định hội tụ cho kiến trúc end-to-end không có điểm mỏ neo, hàm mất mát tịnh tiến ProgLoss tự động điều phối tỷ trọng giữa loss phân lớp L_{cls} và loss định vị L_{box} theo epoch t :

$$L_{total}(t) = \lambda_t \cdot L_{cls} + (1 - \lambda_t) \cdot L_{box} \quad (2.5)$$

Bằng cách cho hệ số λ_t giảm dần theo hàm cosine, mô hình sẽ ưu tiên khai thác các đặc trưng ngữ nghĩa ở giai đoạn đầu, sau đó dịch chuyển trọng tâm sang tinh chỉnh độ chính xác của tọa độ hộp ở giai đoạn cuối, giúp quá trình hội tụ diễn ra nhịp nhàng và ổn định.

Song song với đó, hiện tượng tiêu biến vật thể nhỏ dưới các góc nhìn camera từ trên cao được khắc phục thông qua cơ chế gán nhãn thích nghi STAL (Small-Target-Aware Label Assignment). Thay vì áp đặt một ngưỡng IoU cố định $\tau = 0.5$ vốn dễ làm mất gradient của các mục tiêu kích thước nhỏ, STAL thiết lập một ngưỡng động linh hoạt dựa trên tỷ lệ diện tích vật thể [15]:

$$\tau_{dynamic} = \tau_{base} \cdot \left(1 - \alpha \cdot \exp \left(-\frac{Area_{obj}}{Area_{img}} \right) \right) \quad (2.6)$$

Trong đó α đóng vai trò kiểm soát tốc độ suy giảm. Đối với các phương tiện ở xa hoặc có kích thước nhỏ, ngưỡng $\tau_{dynamic}$ sẽ tự động hạ thấp một cách thích nghi, cho phép các vùng mẫu có độ giao tách thấp vẫn giữ lại được gradient để cập nhật trọng số, giải quyết triệt để bài toán bỏ sót mục tiêu trong các kịch bản giám sát góc rộng [6].

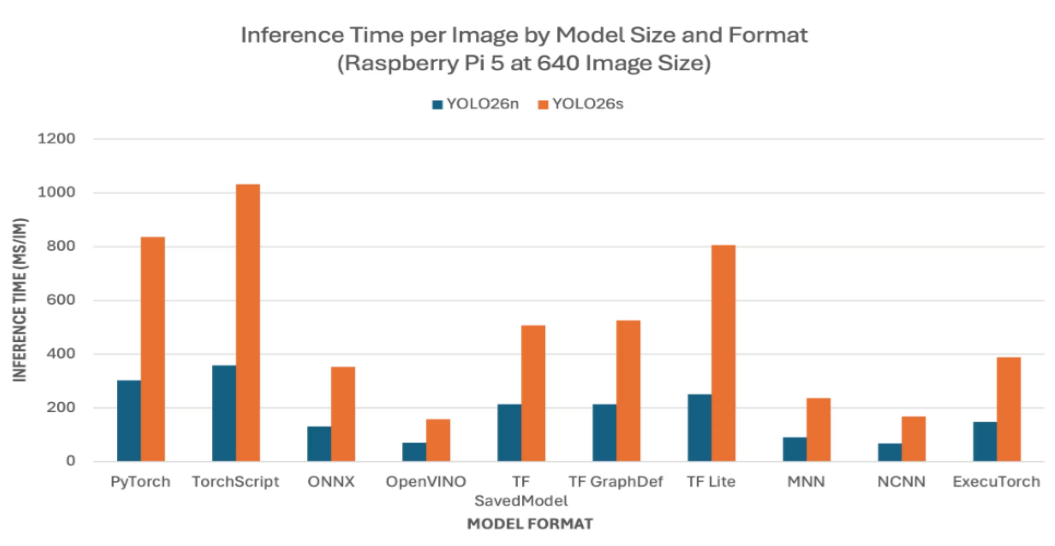
Cuối cùng, toàn bộ tiến trình học phức tạp này được tăng tốc hiệu quả nhờ trình tối ưu hóa thể hệ mới MuSGD. Bằng việc kết hợp SGD truyền thống với giải thuật Muon để tích hợp phép khử chỉnh giao Newton-Schulz trực tiếp vào quỹ đạo gradient, MuSGD mang lại tốc độ hội tụ nhanh và ổn định hơn hẳn so với AdamW hay SGD tiêu chuẩn trên các phiên bản tiền nhiệm. Cải tiến này không chỉ rút ngắn thời gian huấn luyện tổng thể nhờ giảm bớt giai đoạn làm nóng ban đầu, mà còn đảm bảo mô hình đạt được kết quả nhận diện nhất quán trên các tập dữ liệu có độ biến động cao [6], [7].

2.2.2 Framework NCNN và triển khai trên Raspberry Pi 5

Về mặt lý thuyết, NCNN là một framework suy luận đồ thị tính toán (inference framework) hiệu năng cao, được thiết kế từ gốc và tối ưu hóa chuyên biệt cho các nền tảng di động cũng như hệ thống nhúng kiến trúc ARM. Điểm cốt lõi giúp NCNN tối giản hóa tài nguyên nằm ở cơ chế gộp lớp toán tử (Layer Fusing). Thay

vì thực thi từng lớp một cách tuần tự và phải ghi/đọc dữ liệu trung gian liên tục vào bộ nhớ, NCNN sẽ kết hợp các cụm toán tử có tính liên đới trực tiếp để tính toán liền mạch ngay trên các thanh ghi phần cứng. Quy trình này không chỉ làm mỏng sơ đồ tính toán mà còn triệt tiêu các sai số làm tròn số thực dấu phẩy động trong các bước xử lý trung gian.

Chính nhờ thiết kế phần lõi không phụ thuộc vào các thư viện bên thứ ba và khả năng can thiệp sâu vào chuỗi chỉ thị của chip ARM, NCNN tạo ra một môi trường lý tưởng cho việc chuyển đổi định dạng dữ liệu (quantization). Khi mô hình loại bỏ được các nút thắt phi tuyến phức tạp như NMS hay DFL, sơ đồ phân phối trọng số sẽ trở nên đồng đều và tuyến tính hơn. Đây là tiền đề lý thuyết vững chắc giúp việc chuyển đổi mô hình sang cấu trúc số nguyên có độ rộng bit cố định đạt hiệu quả tối ưu, giảm thiểu tối đa hiện tượng phân rã độ chính xác thường gặp trên các framework thông thường.



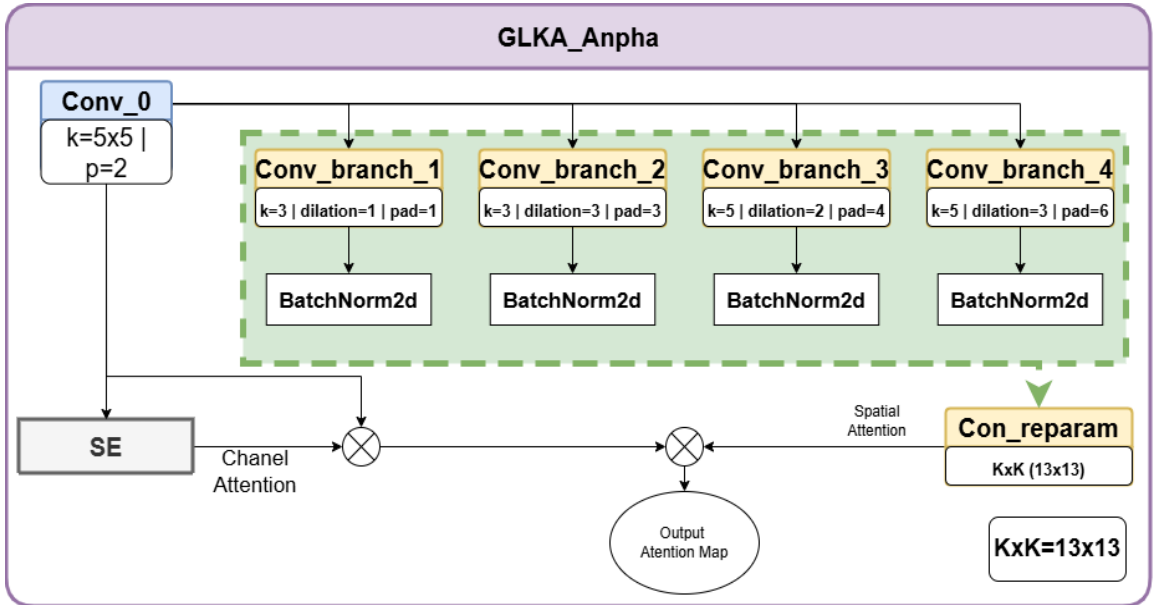
Hình 2.2: Bảng so sánh các định dạng chạy của YOLOv26 từ nhà phát triển Utralytics [8]

2.3 Cơ chế chú ý thị giác và giới thiệu khối GLKA (Global Large Kernel Attention)

2.3.1 GLKA – Global Large Kernel Attention

GLKA (Global Large Kernel Attention) là module chú ý thị giác được thiết kế nhằm giải quyết bài toán đặc trưng lớn của bài toán phân loại mật độ xe, đề xuất mở rộng cơ chế LKA theo ba hướng đồng thời. Thứ nhất, tích hợp thêm khối chú

ý kênh theo cơ chế SE (Squeeze-and-Excitation) [9] hoạt động song song với nhánh chú ý không gian, thay vì tuần tự như CBAM [10], thiết kế song song được xác nhận là vượt trội so với thiết kế tuần tự ở quy mô dữ liệu trung bình ($1k \leq N \leq 50k$) [11]. Thứ hai, thay thế nhánh tích chập giãn đơn lẻ của LKA gốc [8] bằng bốn nhánh tích chập sâu song song với các tỉ lệ giãn khác nhau, lấy cảm hứng từ triết lý đa nhánh trong UniRepLKNet [12], mô hình được cấu trúc nhiều nhánh tích chập với kích thước nhân và tỉ lệ giãn đa dạng được kết hợp để mở rộng vùng tiếp nhận hiệu dụng mà không tăng đáng kể chi phí tính toán. Thứ ba, áp dụng kỹ thuật tái tham số hóa cấu trúc từ RepVGG [13] để hợp nhất toàn bộ bốn nhánh huấn luyện thành một tích chập đơn khi triển khai, đảm bảo chi phí suy luận không tăng so với một tích chập sâu thông thường.



Hình 2.3: Cấu trúc khối GLKA (Global Larger Kernel Attention)

Cho đầu vào $x \in \mathbb{R}^{B \times C \times H \times W}$, GLKA trước tiên trích xuất đặc trưng toàn cục qua một lớp tích chập depthwise kích thước 5×5 :

$$g = DW_{5 \times 5}(x) \quad (2.7)$$

Từ đặc trưng toàn cục g , hai nhánh được triển khai song song. Nhánh kênh áp dụng khối SE để tạo bộ trọng số hiệu chỉnh theo từng kênh, sau đó nhân theo từng phần tử với g :

$$A_{ch} = SE(g) \otimes g \quad (2.8)$$

Nhánh không gian tổng hợp bốn luồng tích chập sâu giãn, mỗi nhánh đi qua một lớp Batch Normalization [14] độc lập trước khi cộng, tương tự cơ chế đa nhánh trong Diverse Branch Block [15]:

$$A_{sp} = \sum_{i=1}^4 \text{BN}_i \left(\text{DW}_{\text{dilated}}^{(i)}(g) \right) \quad (2.9)$$

Đầu ra cuối cùng là tích theo từng phần tử giữa hai nhánh:

$$y = A_{ch} \otimes A_{sp} \quad (2.10)$$

Điểm then chốt của thiết kế: cả A_{ch} và A_{sp} đều xuất phát từ cùng một đặc trưng g , không tồn tại phụ thuộc tuần tự giữa hai nhánh.

Bốn nhánh trong luồng không gian được thiết kế để bao phủ vùng tiếp nhận ở nhiều tầm xa khác nhau, với thông số cụ thể như Bảng 1:

Bảng 2.1 : Cấu hình phân cấp vùng cảm nhận (EFR) đa quy mô của các nhánh

Nhánh	Nhân tích chập (Kernel)	Độ giãn (Dilation)	Vùng cảm nhận
1	3x3	1	3x3
2	3x3	3	7x7
3	5x5	2	9x9
4	5x5	3	13x13

Việc lựa chọn tích chập depthwise 5×5 ở tầng trích xuất đặc trưng toàn cục xuất phát từ phân tích lý thuyết về vấn đề lưới trong tích chập giãn, được Wang et al. hệ thống hóa qua khung Hybrid Dilated Convolution (HDC) [16]. Khi xếp chồng nhiều lớp tích chập giãn có cùng tỉ lệ r , các điểm đóng góp vào phép tính chỉ xuất hiện theo mẫu bàn cờ — với $k=3$ và $r=2$, chỉ 9/25 điểm trong vùng $kD \times kD$ được sử dụng, đồng nghĩa mạng bỏ qua ít nhất 75% thông tin cục bộ.

Khung HDC đã định nghĩa khoảng cách tối đa M giữa hai điểm khác không trong vùng tiếp nhận theo công thức đệ quy:

$$M_i = \max [M_{i+1} - 2r_i, M_{i+1} - 2(M_{i+1} - r_i), r_i], \quad M_n = r_n \quad (2.11)$$

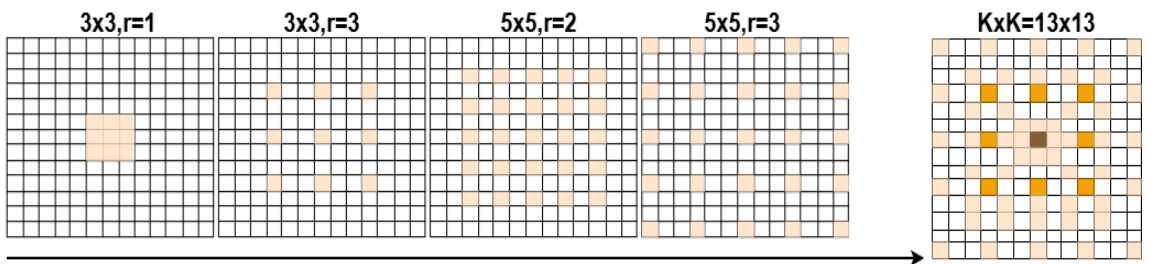
Điều kiện để vùng tiếp nhận phủ kín không gian mà không xuất hiện lỗ hổng là $M_2 \leq K$, với K là kích thước nhân tích chập. Xét bốn nhánh giãn trong GLKA với $r=\{1,3,2,3\}$: với $K=3$ (nhánh 1 và 2), tại $r=3$ điều kiện $M_2 = K=3$ vừa đủ thỏa mãn, không có biên an toàn; trong khi với $K=5$ (nhánh 3 và 4), điều kiện $M_2 = 3 \leq K = 5$ được thỏa mãn với biên dự phòng lớn hơn. Nhận xét này nhất quán với kết luận của RepLKNet [17]: nhân lớn hơn ở tầng đầu giúp mạng nắm bắt ngữ cảnh rộng mà không cần tăng chiều sâu.

Khối SE trong nhánh kênh được cài đặt bằng tích chập 1×1 thay vì lớp kết nối đầy đủ như SENet gốc và chức năng tương đương sau phép gộp trung bình toàn cục với tỉ lệ thu hẹp $r = 8$:

$$SE(g) = \sigma \left(\text{Conv}_{1 \times 1} \left(\text{ReLU6} \left(\text{Conv}_{1 \times 1} \left(\text{GAP}(g), C \rightarrow \frac{C}{8} \right) \right), \frac{C}{8} \rightarrow C \right) \right) \quad (2.12)$$

Chọn $r = 8$ thay vì $r = 16$ như SENet gốc [9] vì chiều kênh nhỏ tại các tầng đầu ($C = 64, 128$): với $r = 16$, chiều bottleneck chỉ còn 4 và 8 đơn vị, quá hẹp để học tương tác kênh hiệu quả.

Kỹ thuật tái tham số hóa cấu trúc được tiên phong bởi RepVGG [13] bài báo đầu tiên đề xuất ý tưởng tách biệt kiến trúc huấn luyện đa nhánh và kiến trúc triển khai một nhánh, sau đó hợp nhất thông qua hấp thụ chuẩn hóa theo lô và cộng nhân tích chập. Diverse Branch Block [15] mở rộng thêm với nhiều dạng nhánh đa dạng hơn. RepLKNet [17] áp dụng nguyên lý này cho nhân tích chập lớn, chứng minh rằng một tích chập theo chiều sâu giãn với nhân k và tỉ lệ giãn d tương đương tích chập tiêu chuẩn có nhân thừa kích thước vùng tiếp nhận hiệu dụng bằng $k + (k - 1)(d - 1)$. UniRepLKNet [12] tiếp tục khẳng định tính hiệu quả của tái tham số hóa trên nhiều dạng nhân lớn khác nhau. Ảnh dưới đây là minh họa cho kỹ thuật chú ý không gian đa nhánh :

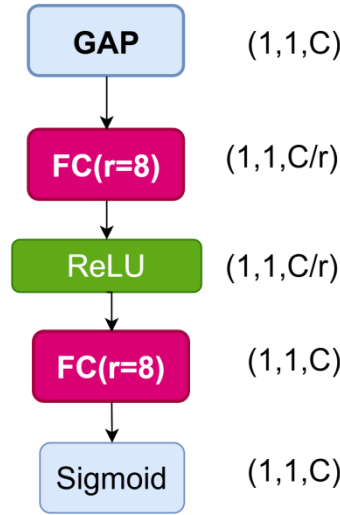


Hình 2.4: Chú ý không gian đa nhánh với kỹ thuật tái tham số hóa

GLKA kế thừa trực tiếp nguyên lý này để hợp nhất bốn nhánh tích chập theo chiều sâu giãn có vùng tiếp nhận hiệu dụng khác nhau (3×3 , 7×7 , 9×9 , 13×13) về một tích chập theo chiều sâu đơn 13×13 khi triển khai, đảm bảo chi phí suy luận không tăng so với một tích chập theo chiều sâu thông thường.

2.3.2 SENet – Squeeze-and-Excitation Network

SENet [9] giới thiệu khối SE (Squeeze-and-Excitation) — cơ chế chú ý kênh cho phép mạng học cách hiệu chỉnh tầm quan trọng của từng kênh đặc trưng một cách thích nghi, với chi phí tham số bổ sung chỉ $2C^2/r$.



Hình 2.5: Cấu trúc khối SE [1]

Khối SE hoạt động qua hai bước. Bước nén (Squeeze) thu gọn toàn bộ thông tin không gian của mỗi kênh về một giá trị vô hướng thông qua phép gộp trung bình toàn cục, tạo ra một vector mô tả thống kê toàn cục cho từng kênh. Bước kích hoạt (Excitation) tiếp nhận vector này và học bộ trọng số tầm quan trọng tương ứng thông qua hai lớp kết nối đầy đủ với hàm kích hoạt ReLU ở giữa và Sigmoid ở đầu ra:

$$SE(x) = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot \text{GAP}(x))) \otimes x \quad (2.13)$$

trong đó $W_1 \in \mathbb{R}^{C/r \times C}$ và $W_2 \in \mathbb{R}^{C \times C/r}$ là ma trận trọng số của hai lớp kết nối đầy đủ, σ là hàm Sigmoid, và $\otimes$ là phép nhân theo từng phần tử. Tỷ lệ thu hẹp r kiểm soát kích thước tầng ẩn bên trong khối với giá trị $r = 16$ được dùng mặc định trong SENet gốc [9].

Đầu ra là vector trọng số kênh $\text{se_weight} \in (0, 1)^C$ được nhân trực tiếp lên đặc trưng đầu vào, cho phép mạng nhấn mạnh các kênh mang thông tin phân biệt và triệt tiêu các kênh ít liên quan mà không thay đổi kích thước không gian của đặc trưng.

2.3.3 Cơ chế chú ý không gian nhân lớn

Bên cạnh hướng tiếp cận chú ý kênh và không gian, cơ chế chú ý nhân lớn (Large Kernel Attention — LKA) được giới thiệu trong kiến trúc VAN [18] đề xuất một giải pháp mở rộng vùng tiếp nhận hiệu dụng (Effective Receptive Field — ERF) mà không làm tăng chi phí tính toán theo bậc hai như các tích chập lớn truyền thống. Thay vì dùng trực tiếp một tầng tích chập kích thước $K \times K$, LKA phân rã thành chuỗi ba thành phần tuần tự với độ phức tạp tuyến tính:

$$\text{LKA}(x) = \text{Conv}_{1 \times 1} \left(\text{DW}_{\text{dilated}}^{(d)} (\text{DW}_{k \times k}(x)) \right) \otimes x \quad (2.14)$$

Trong đó $\text{DW}_{k \times k}$ là tích chập theo chiều sâu cục bộ kích thước $k \times k$ thu thập thông tin lân cận gần, $\text{DW}_{\text{dilated}}^{(d)}$ là tích chập theo chiều sâu giãn với tỉ lệ giãn d mở rộng tầm quan sát ra ngữ cảnh toàn cục, và $\text{Conv}_{1 \times 1}$ là tích chập điểm thiết lập tương quan giữa các kênh. Vùng tiếp nhận hiệu dụng tổng hợp của chuỗi ba thành phần này được tính theo:

$$\text{ERF}_{\text{LKA}} = k + (k - 1)(d - 1) + 2 \cdot \left\lfloor \frac{k}{2} \right\rfloor \quad (2.15)$$

Với cấu hình điển hình $k=5, d=3$, LKA đạt $\text{ERF} = 23 \times 23$ trong khi chi phí tính toán chỉ tăng tuyến tính theo kích thước bản đồ đặc trưng, thay vì bậc hai như tích chập tiêu chuẩn $K \times K$ [8].

UniRepLKNet [13] tiếp tục đẩy xa tiềm năng của tích chập nhân lớn theo một hướng khác: thay vì chỉ tối ưu chi phí tính toán, kiến trúc này tập trung vào khả năng biểu diễn đa miền thông qua bốn nguyên tắc thiết kế dành riêng cho mạng tích chập nhân lớn. Điểm cốt lõi là kết hợp các bộ lọc kích thước lớn với các nhánh tích chập nhỏ song song trong quá trình huấn luyện:

$$y = \text{DW}_{K \times K}(x) + \sum_{i=1}^N \text{DW}_{k_i \times k_i}(x), \quad K \gg k_i \quad (2.16)$$

2.4 Tái tham số hóa cấu trúc

Tái tham số hóa cấu trúc là kỹ thuật tách biệt kiến trúc huấn luyện và kiến trúc suy luận: sử dụng cấu trúc đa nhánh trong huấn luyện để tăng khả năng biểu diễn, sau đó hợp nhất toàn bộ về một nhánh đơn tương đương khi triển khai. Kỹ thuật này được tiên phong bởi RepVGG [13], trong đó mỗi khối huấn luyện gồm ba nhánh song song mỗi nhánh theo sau bởi một lớp Batch Normalization riêng, rồi được hợp nhất thành một tích chập duy nhất sau huấn luyện mà không mất độ chính xác. RepLKNet [17] mở rộng nguyên lý này cho tích chập nhân lớn, còn UniRepLKNet [12] tiếp tục đề xuất khối Dilated Reparam với nhiều nhánh tích chập giãn song song, hấp thụ toàn bộ về một tích chập đơn không giãn khi suy luận và ý tưởng này đã được kế thừa trực tiếp trong thiết kế module GLKA

2.4.1 Hợp nhất chuẩn hóa Batch Normalization vào tích chập

Lớp Chuẩn hóa Batch (Batch Normalization - BN) được đề xuất [14] nhằm ổn định phân phối đầu vào của mỗi lớp trong quá trình huấn luyện, qua đó cho phép sử dụng tốc độ học cao hơn và giảm sự phụ thuộc vào khởi tạo tham số. Tại thời điểm suy luận (inference), khi các tham số thống kê đã cố định, BN thực chất là một phép biến đổi tuyến tính và có thể được tích hợp trực tiếp vào lớp tích chập đứng trước nó. Cụ thể, mỗi cặp Conv–BN có thể thu gọn thành một lớp tích chập đơn duy nhất với trọng số và bias tương đương [13]:

$$W'_i = \frac{\gamma_i}{\sigma_i} W_i, \quad b'_i = \beta_i - \frac{\mu_i \gamma_i}{\sigma_i}, \quad \sigma_i = \sqrt{\text{Var}_i + \varepsilon} \quad (2.17)$$

Trong đó, γ_i và β_i lần lượt là các tham số tỉ lệ và dịch chuyển học được; μ_i và Var_i là giá trị trung bình và phương sai tích lũy thu được trong quá trình huấn luyện; ε là hằng số để đảm bảo ổn định số học [14]. Sau phép hợp nhất này, mỗi nhánh Conv–BN trở thành một tích chập đơn thuần, tạo điều kiện cho phép cộng kernel giữa các nhánh với nhau [13].

2.4.2 Chuyển nhân giãn về không gian thông nhất

Một tích chập giãn với kernel kích thước $k \times k$ và tỉ lệ giãn r tương đương với một tích chập thông thường có kernel thưa với kích thước:

$$(k - 1) \cdot r + 1 \quad (2.18)$$

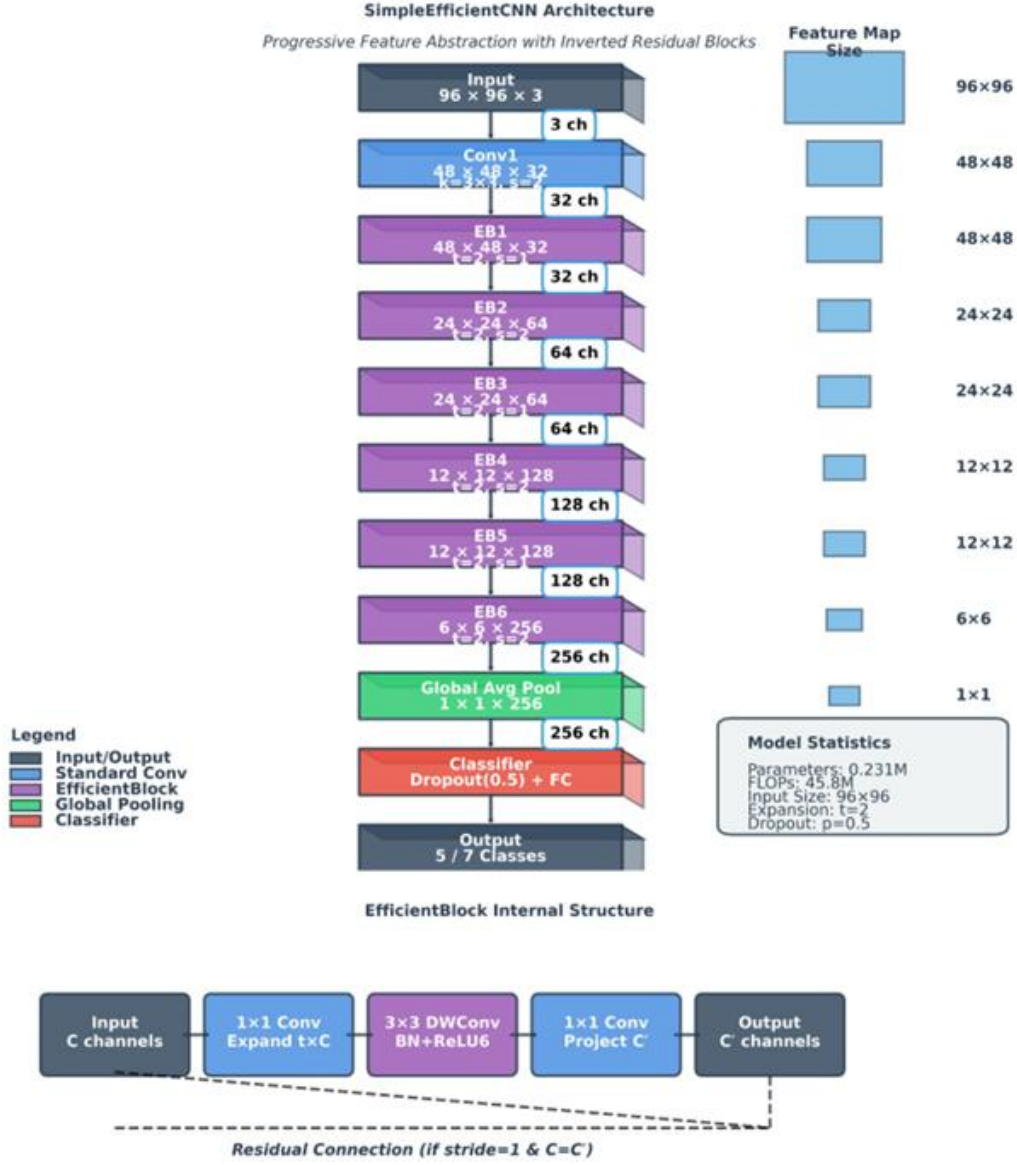
Bốn nhánh trong GLKA có vùng tiếp nhận hiệu dụng lần lượt là 3, 7, 9, 13 và được zero-padding đồng nhất về 13×13 trước khi cộng [12]:

$$W_{\text{equiv}} = \sum_{i=1}^4 \text{ZeroPad}\left(W_{\text{fused}}^{(i)}, p_i\right), \quad b_{\text{equiv}} = \sum_{i=1}^4 b_{\text{fused}}^{(i)} \quad (2.19)$$

Sau tái tham số hóa, toàn bộ module chú ý không gian đa nhánh (bốn tích chập và bốn lớp chuẩn hóa Batch Normalization) được thay thế bằng một tích chập theo chiều sâu đơn 13×13 . Đầu ra hoàn toàn đồng nhất giữa giai đoạn huấn luyện và triển khai với sai số nhỏ hơn 10^{-5} , đồng thời loại bỏ toàn bộ bộ nhớ và chi phí tính toán của ba nhánh dư thừa.

2.5 Kiến trúc mạng nơ-ron tích chập siêu nhẹ – Simple_GLKA

Simple_GLKA được xây dựng trực tiếp trên khung kiến trúc của mô hình siêu nhẹ SimpleEfficientCNN [19], đây là một mạng tích chập được thiết kế cho bài toán phân loại hình ảnh trên thiết bị nhúng với ngân sách tính toán hạn chế. Thay vì thiết kế từ đầu, chúng tôi kế thừa bốn nguyên lý cốt lõi của SimpleEfficientCNN và thực hiện một thay đổi có chủ đích duy nhất: thay thế tích chập theo chiều sâu tại Block 2–4 bằng module GLKA.



Hình 2.6: Cấu trúc SimpleEfficientCNN [19]

2.5.1 Tích chập tách rời theo chiều sâu làm đơn vị cơ bản

Thành phần đóng góp lớn nhất vào hiệu quả của SimpleEfficientCNN là việc thay thế tích chập tiêu chuẩn bằng tích chập tách rời theo chiều sâu là một kỹ thuật được giới thiệu trong MobileNetV2 [20]. Thay vì thực hiện phép tích chập đồng thời trên cả không gian lẫn kênh với chi phí $\mathcal{O}(H \cdot W \cdot C_{in} \cdot C_{out} \cdot k^2)$, phép tính được phân tách thành hai bước độc lập:

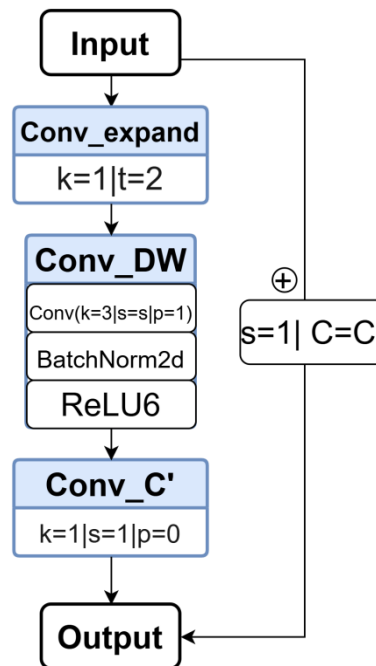
$$\text{DWSCConv}(x) = \text{PW}_{1 \times 1}(\text{DW}_{k \times k}(x)) \quad (2.20)$$

trong đó $\text{DW}_{k \times k}$ là tích chập theo chiều sâu, mỗi kênh được xử lý độc lập bởi một bộ lọc riêng và $\text{PW}_{1 \times 1}$ là tích chập điểm có nhiệm vụ hòa trộn thông tin giữa các

kênh. Chi phí tính toán tổng hợp giảm xuống còn $\mathcal{O}(H \cdot W \cdot C \cdot (k^2 + C_{out}))$, tức nhỏ hơn k^2 lần so với tích chập tiêu chuẩn cùng cấu hình và đã được tác giả xác nhận đây là thành phần có ảnh hưởng lớn nhất: thay thế tích chập tách rời bằng tích chập tiêu chuẩn làm tăng 153% số tham số trong khi độ chính xác giảm 1%.

2.5.2 Khối Inverted Residual với tỉ lệ mở rộng giảm thiểu

Mỗi EfficientBlock trong áp dụng cấu trúc inverted residual của MobileNetV2 [20]: mở rộng kênh qua tích chập điểm 1×1 đến tích chập theo chiều sâu 3×3 sau đó thu hẹp kênh qua tích chập điểm 1×1 , với kết nối tắt khi $\text{stride} = 1$ và số kênh đầu vào bằng đầu ra. Ảnh dưới đây minh họa cho khối EfficientBlock :



Hình 2.7: Cấu trúc khối Efficient Block gốc

Điểm khác biệt then chốt so với MobileNetV2 là tỉ lệ mở rộng được đặt $t=2$ thay vì $t=6$. Lựa chọn này xuất phát từ quan sát thực nghiệm : với bài toán phân loại đặc trưng hình thái mịn, tỉ lệ mở rộng lớn gây dư thừa tham số mà không cải thiện độ chính xác. Kết quả là mô hình chỉ cần 0,231M tham số — ít hơn 6,6 lần so với MobileNetV3-Small trong cùng điều kiện thử nghiệm [19].

2.5.3 Phân bổ kênh đơn điệu và đầu phân loại tối giản

SimpleEfficientCNN áp dụng chiến lược tăng kênh đơn điệu $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ xuyên suốt các tầng mà không dùng bất kỳ module chú ý

nào. Kiến trúc của SimpleEfficientCNN đã cho thấy việc thêm các thành phần phức tạp như kết nối tắt hay dropout không mang lại cải thiện đáng kể — kiến trúc tối giản với tích chập tách rời làm đơn vị chính đã đủ năng lực biểu diễn cho bài toán phân loại hình ảnh độ phân giải thấp.

Thay vì dùng Flatten kết hợp nhiều lớp kết nối đầy đủ, đầu phân loại được đơn giản hóa tối đa thành phép gộp trung bình toàn cục về 1×1 , tiếp theo là Dropout và một lớp tuyến tính duy nhất:

$$\hat{y} = \text{Linear}(\text{Dropout}(\text{GAP}(f))) \quad (2.21)$$

Kết quả thực nghiệm cho thấy phép gộp trung bình (GAP) phù hợp hơn Max Pooling khi tổng hợp đặc trưng toàn cục từ bản đồ đặc trưng kích thước nhỏ, với mức chênh lệch 0,15 điểm phần trăm độ chính xác khi thay thế. Thiết kế này đồng thời giúp giảm bộ nhớ GPU đỉnh xuống còn 20,5 MB, thấp hơn 9,4 lần so với ResNet34 [19].

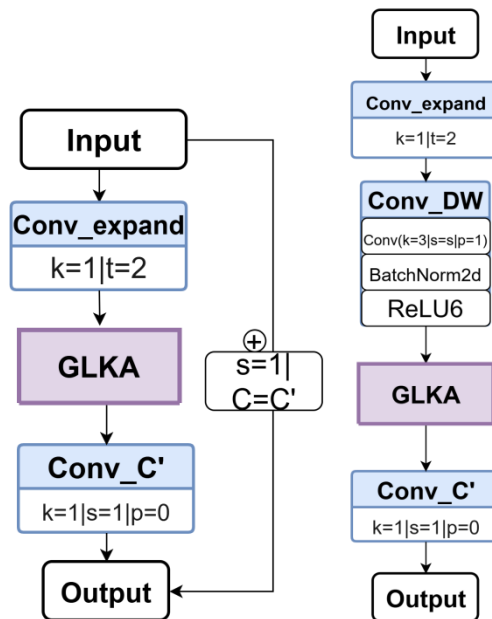
2.5.4 Ứng dụng vào Simple_GLKA

Simple_GLKA kế thừa toàn bộ khung trên và thực hiện một thay đổi có chủ đích duy nhất tại Block 2–4: thay thế tích chập theo chiều sâu 3×3 bằng khối GLKA. Lý do nằm ở sự khác biệt về yêu cầu vùng tiếp nhận giữa hai bài toán: phân loại hạt gạo chỉ cần nắm bắt đặc trưng hình thái cục bộ với tích chập 3×3 là đủ, trong khi phân loại mật độ giao thông đòi hỏi vùng tiếp nhận hiệu dụng lớn hơn (13×13) để quan sát được sự phân bố không gian của phương tiện trên toàn khung hình.

Input	Layer	t	c	s	Output
224x224x3	Conv1	-	32	2	112x112x32
112x112x32	EB1	2	32	1	112x112x32
112x112x32	GLKA_2	2	64	2	56x56x64
56x56x64	GLKA_3	2	64	1	56x56x64
56x56x64	GLKA_4	2	128	2	28x28x128
28x28x128	EB5	2	128	1	28x28x128
28x28x128	EB6	2	256	2	28x28x256
28x28x256	GAP	-	256	-	1x1x256
1x1x256	Classifier	-	-	-	% class

Hình 2.8: Cấu trúc tổng quát của mạng tích chập Simple_GLKA

GLKA được áp dụng tại các tầng có bản đồ đặc trưng kích thước trung bình (56×56 và 28×28) nơi vùng tiếp nhận 13×13 đủ bao phủ ngữ cảnh không gian cần thiết. Áp dụng tại khối 1 (112×112) sẽ gây lãng phí tính toán; tại khối 5–6 thì không cần thiết vì mạng đã học đủ đặc trưng trừu tượng ở độ phân giải thấp.



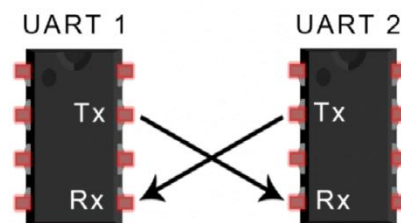
Hình 2.9: Cấu trúc khối GLKA_3 và GLKA_2+4

Cách tích hợp GLKA vào từng khối có sự khác biệt tùy theo vai trò của khối trong kiến trúc (Hình 2.8). Tại khối 3 ($\text{stride} = 1, C_{in} = C_{out}$), GLKA thay thế trực tiếp tầng tích chập theo chiều sâu và kết nối tắt được giữ nguyên cấu trúc residual đảo chiều được bảo toàn hoàn toàn. Tại khối 2 và 4 ($\text{stride} = 2$, thay đổi số kênh), do kết nối tắt không áp dụng được, một tầng tích chập theo chiều sâu 3×3 tiêu chuẩn được giữ lại trước GLKA để đảm bảo quá trình giảm độ phân giải diễn ra ổn định trước khi module chú ý xử lý đặc trưng.

Tất cả các nguyên lý còn lại như tỉ lệ mở rộng $t=2$, phân bố kênh đơn điệu $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$, đầu GAP, và Dropout đều được giữ nguyên để đảm bảo tính nhẹ và khả năng triển khai thực tế trên Raspberry Pi 5.

2.6 Tổng quan giao thức UART

Giao thức UART là một chuẩn truyền thông nối tiếp bất đối xứng, cho phép hai thiết bị trao đổi dữ liệu trực tiếp với nhau mà không cần sử dụng tín hiệu xung nhịp đồng bộ (Clock). Thay vì dùng dây xung nhịp chung như I2C hay SPI, UART sử dụng cơ chế thiết lập tốc độ truyền dữ liệu giống nhau tại cả hai đầu thiết bị, gọi là tốc độ Baud (Baud rate). Cấu trúc kết nối cơ bản của UART cực kỳ đơn giản, chỉ bao gồm hai dây tín hiệu chính là TX (Transmit - Truyền dữ liệu) và RX (Receive - Nhận dữ liệu). Để quá trình trao đổi diễn ra, chân TX của thiết bị này phải được nối với chân RX của thiết bị kia và ngược lại, tạo thành một vòng lặp truyền nhận khép kín.



Hình 2.10: Tổng quan về UART

Về cơ chế truyền tin, dữ liệu trong UART được tổ chức thành từng khung (Frame) bao gồm các thành phần cố định: bit bắt đầu (Start bit), dữ liệu chính (Data bits - thường là 8 bit), bit kiểm tra lỗi tùy chọn (Parity bit) và một hoặc hai bit kết thúc (Stop bit). Do không có đường xung nhịp đồng bộ, việc truyền nhận dữ liệu dựa hoàn toàn vào việc lấy mẫu tín hiệu tại các khoảng thời gian được định sẵn

theo tốc độ Baud (Baud rate). Trong các hệ thống nhúng xử lý thời gian thực, tốc độ cao 115200 bps thường được ưu tiên lựa chọn để đảm bảo các gói tin lệnh và dữ liệu phản hồi được truyền tải với độ trễ tối thiểu.

Trong đồ án hệ thống điều khiển giao thông thông minh, giao thức UART đóng vai trò là cầu nối truyền thông cốt lõi giữa bộ xử lý AI biên Raspberry Pi 5 và vi điều khiển chấp hành ESP32. Thông qua đường truyền nối tiếp này, Raspberry Pi 5 có thể gửi xuống các mã hiệu chế độ đèn thích ứng (từ m0 đến m4) hoặc các mã lệnh ưu tiên khẩn cấp (mã "A", "a", "B", "b"). Ngược lại, vi điều khiển ESP32 cũng sử dụng kênh UART để chủ động phát ngược các chuỗi ký tự phản hồi (run hoặc run1) lên bộ xử lý trung tâm nhằm kích hoạt luồng chụp ảnh và quét mật độ giao thông. Với tính năng hỗ trợ nhiều bộ UART phần cứng độc lập (Hardware Serial), hệ thống cho phép vừa duy trì đường truyền liên kết nội bộ ổn định giữa Pi 5 và ESP32, vừa có thể xuất dữ liệu debug lên máy tính đồng thời, tạo điều kiện thuận lợi cho việc giám sát và tối ưu hóa giải thuật trong quá trình thực nghiệm.

2.7 Tổng quan về Ethernet

Ethernet là một công nghệ truyền dẫn dữ liệu tiêu chuẩn được sử dụng phổ biến để kết nối các mạng cục bộ (LAN), cho phép các thiết bị phần cứng có thể giao tiếp với nhau thông qua một giao thức mạng và bộ quy tắc chung. Ethernet quy định cách thức các thiết bị cấu trúc định dạng và truyền tải gói tin, đảm bảo các nút mạng trong cùng phân đoạn có thể phát hiện, tiếp nhận và xử lý dữ liệu một cách chính xác. Trong cấu trúc phần cứng, cáp Ethernet đóng vai trò là môi trường truyền dẫn hữu tuyến vật lý để luân chuyển ổn định các luồng tín hiệu dữ liệu này.

Ethernet mang lại cho người dùng rất nhiều lợi ích vượt trội. Đây cũng chính là lý do Ethernet là gì ngày càng được nhiều người quan tâm và sử dụng.

- Độ trễ thấp, ổn định cao: Ethernet truyền dẫn qua cáp vật lý có vỏ bọc chống nhiễu, đảm bảo luồng dữ liệu liên tục với độ trễ tối thiểu, khác với kết nối không dây dễ bị gián đoạn do vật cản hoặc nhiễu sóng.

- Bảo mật và kiểm soát trực quan: Đường cáp vật lý cố định cho phép người kỹ sư dễ dàng quản lý thiết bị truy cập mạng nội bộ, ngăn chặn can thiệp hoặc điều hướng băng thông trái phép từ bên ngoài.
- Tương thích và mở rộng linh hoạt: Ethernet hỗ trợ tương thích ngược và mở rộng qua thiết bị chuyển mạch (Switch/Router), cho phép nâng cấp từ mô hình hai trạm lên mạng lưới nhiều nút mà không cần thay đổi phần mềm cốt lõi.
- Chi phí hợp lý: Chi phí triển khai và thiết bị Ethernet ở mức thấp, phù hợp với nhiều quy mô ứng dụng khác nhau.

CHƯƠNG 3

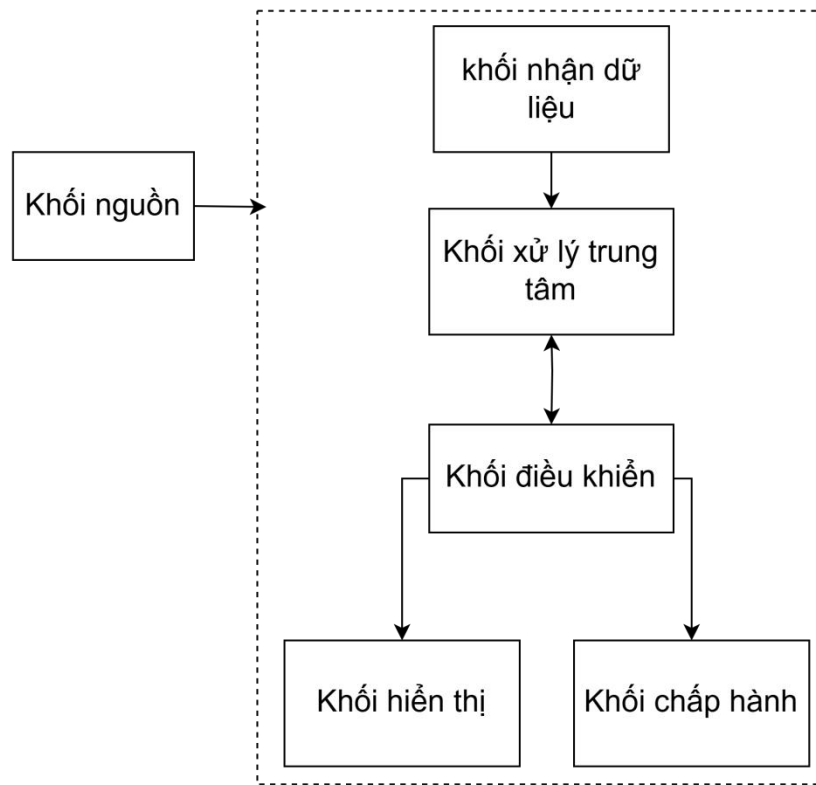
THIẾT KẾ HỆ THỐNG

3.1 Mô tả hệ thống

Hệ thống điều khiển đèn tín hiệu giao thông dựa trên mật độ phương tiện là một giải pháp tích hợp giữa công nghệ thị giác máy tính và điều khiển tự động. Hệ thống sử dụng camera để thu thập hình ảnh tại giao lộ, truyền về khối xử lý trung tâm để phân tích lưu lượng xe, từ đó xuất lệnh điều khiển thời gian sáng của cụm đèn tín hiệu. Để mô hình vận hành hiệu quả và bám sát thực tế hệ thống phải thực hiện tốt việc tiền xử lý nhằm tối ưu hóa chất lượng ảnh đầu vào, phục vụ cho quá trình nhận diện, phân loại và tính toán mật độ xe theo thời gian thực. Dựa trên dữ liệu này, hệ thống sẽ tự động tinh chỉnh chu kỳ đèn (Xanh - Vàng - Đỏ) một cách linh hoạt theo lưu lượng hiện tại. Toàn bộ quá trình kết nối giữa khối xử lý trung tâm và khối điều khiển phần cứng phải diễn ra liên tục, ổn định, đồng thời hiển thị trực quan thời gian đếm ngược tại các trụ đèn.

Hệ thống phải đáp ứng thời gian thực với độ trễ từ lúc nhận diện hình ảnh đến khi xuất lệnh điều khiển ở mức tối thiểu là 1 giây. Thuật toán phân tích phương tiện cần đạt độ chính xác cao tối thiểu là 85% trong điều kiện môi trường ánh sáng tiêu chuẩn.

3.2 Sơ đồ khối tổng quát



Hình 3.1: Sơ đồ khối của hệ thống

Hệ thống điều khiển bao gồm các khối chức năng chính được mô tả như sau:

Khối thu thập dữ liệu đóng vai trò là tầng đầu vào, chịu trách nhiệm quan sát và ghi nhận tình trạng giao thông thực tế. Dữ liệu hình ảnh được truyền về máy tính nhúng Raspberry Pi 5 để phân tích.

Khối nguồn đóng vai trò huyết mạch, đảm bảo cách ly nhiễu và cung cấp điện áp 5V dòng cao cho máy tính nhúng Raspberry Pi 5 và 3.3V cho vi điều khiển một cách ổn định và liên tục cho toàn bộ hệ thống hoạt động.

Khối xử lý trung tâm đóng vai trò là "bộ não" điều khiển toàn bộ hoạt động của hệ thống. Khối này tiếp nhận hình ảnh, chạy thuật toán YOLO để tính toán mật độ phương tiện. Dựa trên kết quả, nó sẽ đưa ra kịch bản điều khiển (từ m1 đến m4) và truyền mệnh lệnh xuống tầng thực thi thông qua giao tiếp UART.

Khối điều khiển sử dụng vi điều khiển ESP32, đóng vai trò là tầng quản lý điều khiển và thực thi thời gian thực. Nhận lệnh từ khối xử lý trung tâm để điều khiển đóng ngắt các chân I/O điều khiển các cụm LED. Đồng thời phản hồi trạng thái ngược về cho khối xử lý trung tâm.

Khối hiển thị: Sử dụng module LED 7 đoạn để trực quan hóa thời gian đếm ngược của các pha đèn tín hiệu. Bên cạnh đó, màn hình OLED được tích hợp để giám sát trực tiếp các thông số hệ thống (như tài nguyên RAM, CPU của Raspberry Pi 5).

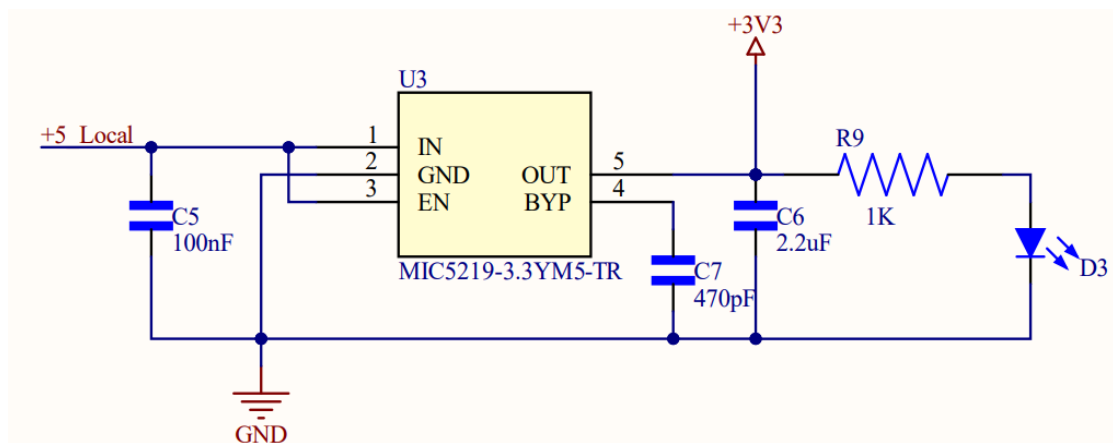
Khối chấp hành: Bao gồm các cụm đèn tín hiệu giao thông (Đỏ, Vàng, Xanh). Khối này có nhiệm vụ tiếp nhận trực tiếp các mức logic từ vi điều khiển ESP32 để đóng ngắt ánh sáng, thực thi việc phân luồng phương tiện tại giao lộ.

3.3 Thiết kế chi tiết từng khối

3.3.1 Khối nguồn

Khối nguồn đảm nhiệm việc cung cấp năng lượng ổn định cho toàn bộ hệ thống. Trong thiết kế của đồ án, khối nguồn được thiết kế để có thể lấy nguồn từ USB cho phần mạch chính.

Hình 3.2 mô tả sơ đồ chân kết nối các khối cấp nguồn cho hệ thống. Đây là khối cấp nguồn 3.3V được chuyển đổi từ 5V để cấp nguồn cho vi điều khiển ESP32.



Hình 3.2: Sơ đồ nguyên lý mạch nguồn 3.3V

Khối nguồn hệ thống sử dụng IC ổn áp tuyến tính MIC5219-3.3 [25] để đảm bảo vi điều khiển hoạt động ổn định, nguồn điện sẽ đi qua mạch điều áp để hạ xuống mức 3.3V. Nguồn điện này cung cấp cho khối điều khiển và các thành phần ngoại vi, đồng thời được làm sạch nhiễu nhờ các tụ lọc và tích hợp mạch bảo vệ an toàn. Ngoài ra, trạng thái của nguồn đầu ra cũng được hiển thị rõ ràng thông qua một đèn LED báo hiệu.

Khối nguồn hệ thống sử dụng IC hạ áp xung TPS5450 [26] với khả năng cung cấp dòng ngõ ra liên tục lên đến 5A, đảm bảo đáp ứng đầy đủ và dư công suất cho máy tính nhúng Raspberry Pi 5 xử lý các thuật toán AI hiệu năng cao mà không lo bị sụt áp. Nguồn điện đầu vào 20V sẽ đi qua mạch điều áp xung để hạ xuống mức 5V, đồng thời được lọc sạch nhiễu sóng nhờ hệ thống tụ lọc cùng cuộn cảm công suất lớn. Ngoài ra, mức điện áp đầu ra luôn được duy trì chính xác và ổn định nhờ mạch cầu phân áp hồi tiếp liên tục, giúp bảo vệ an toàn cho toàn bộ hệ thống trong suốt quá trình vận hành.

Khối nhận dữ liệu đóng vai trò là đôi mắt của toàn bộ hệ thống. Chịu trách nhiệm ghi nhận tình trạng giao thông theo thời gian thực. Thiết bị được lựa chọn cho hệ thống này là module camera Astra Pro giao tiếp qua chuẩn USB. Việc lựa chọn và sử dụng module camera theo chuẩn UVC (USB Video Class) giúp thiết bị dễ dàng tương ứng với hệ điều hành máy tính nhúng Raspberry Pi 5 giúp đảm bảo tốc độ truyền tải khung hình ổn định để cung cấp dữ liệu đầu vào.



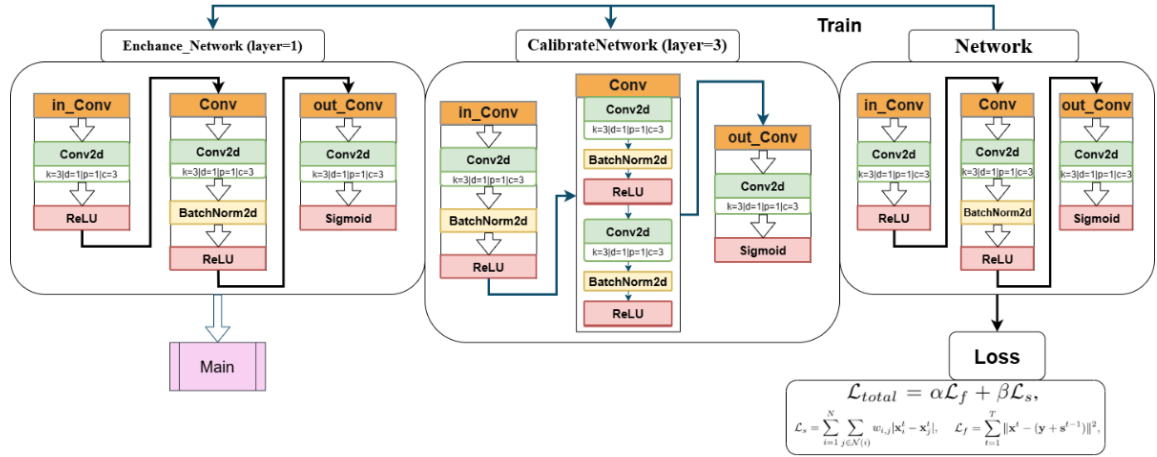
Hình 3.4: Module camera Astra Pro

Trong điều kiện thực địa, camera giám sát giao thông thường xuyên đối mặt với thiếu sáng ban đêm khiến đặc trưng phương tiện bị suy giảm nghiêm trọng và ảnh hưởng trực tiếp đến độ chính xác các tầng nhận diện phía sau. Để xử lý hiệu quả trên Raspberry Pi 5, hệ thống tích hợp SCI (Self-Calibrated Illumination) [21], một khung học máy nhẹ dựa trên mô hình Retinex, phân rã ảnh quan sát thành tích của ảnh phản xạ và bản đồ độ rọi:

$$\mathbf{y} = \mathbf{z} \otimes \mathbf{x}, \quad \mathbf{z} = \mathbf{y} \oslash \mathbf{x} \quad (3.1)$$

Trong đó $\mathbf{z}$ chứa thông tin nội tại của cảnh vật, $\mathbf{x}$ phản ánh điều kiện chiếu sáng môi trường. Mục tiêu tăng cường là ước tính $\mathbf{x}$ rồi khôi phục $\mathbf{z}$ qua phép chia.

Như minh họa ở Hình 3.5, kiến trúc SCI [21] gồm hai nhánh hoạt động tuần tự trong quá trình huấn luyện: Enhance Network ước tính bản đồ độ rọi qua cơ chế học lặp với trọng số chia sẻ giữa các giai đoạn, giúp mô hình cực nhẹ (vài chục KB) nhưng vẫn đủ năng lực biểu diễn; Calibrate Network tạo tín hiệu phản hồi từ kết quả trung gian để ổn định quá trình huấn luyện, ngăn các giai đoạn lặp bị lệch hướng tích lũy.



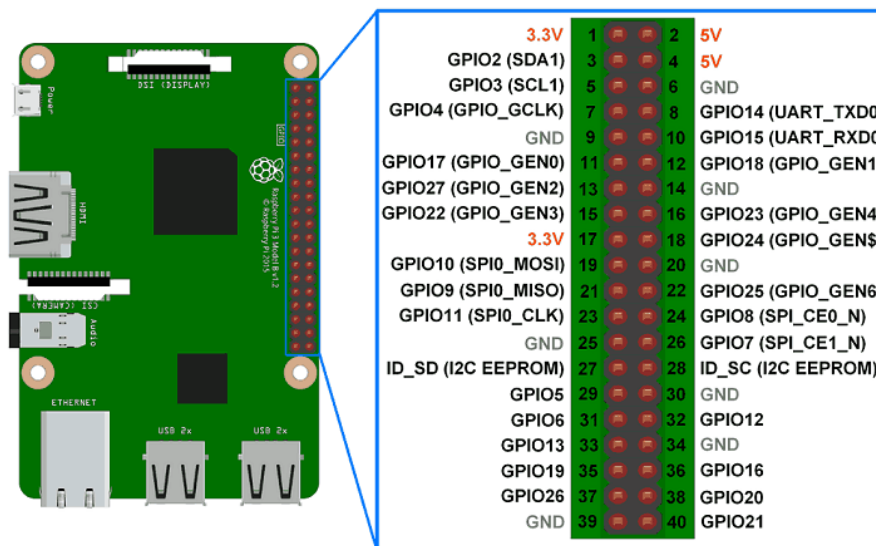
Hình 3.5: Kiến trúc SCI với Enhance Network (trái), Calibrate Network (giữa, chỉ dùng khi huấn luyện) và hàm mất mát tổng hợp. Khi triển khai, chỉ nhánh Enhance Network được giữ lại.

Tuy nhiên, áp dụng SCI vô điều kiện trên mọi khung hình sẽ gây lãng phí tài nguyên và có thể làm méo ảnh vốn đã đủ sáng. Vì vậy, hệ thống áp dụng cơ chế kích hoạt SCI có chọn lọc: độ sáng khung hình được tính toán và chỉ kích hoạt khi giá trị này thấp hơn ngưỡng 0,4. Tiếp theo, vùng quan tâm (Region of Interest-ROI) được trích xuất theo đa giác tùy chỉnh bao phủ khu vực mặt đường, loại bỏ nền không liên quan và tập trung tính toán vào vùng có phương tiện.

3.3.3 Khối xử lý trung tâm

Khối xử lý trung tâm, hệ thống sử dụng máy tính nhúng Raspberry Pi 5 phiên bản 2GB RAM đảm nhiệm vai trò tính toán và phân tích dữ liệu. Thiết bị tiếp nhận luồng ảnh đầu vào và phân tách thành hai nhánh xử lý song song: ảnh được thu nhỏ về kích thước 224×224 pixel phục vụ mô hình phân loại mật độ giao thông Simple_GLKA, đồng thời được chuẩn hóa về 480×480 pixel theo phương pháp letterboxing (thêm viền đen) để đưa vào mô hình nhận diện phương tiện YOLOv26n. Sau khi hoàn tất hai tác vụ suy luận, kết quả mật độ và số lượng xe được tổng hợp rồi truyền xuống vi điều khiển ESP32 thông qua giao tiếp UART, từ đó điều phối trực tiếp chu kỳ hoạt động của hệ thống đèn tín hiệu giao thông.

Hình 3.6 là sơ đồ chân của Raspberry Pi 5 phiên bản 2GB RAM trong hệ thống.



Hình 3.6: Sơ đồ chân máy tính nhúng Raspberry Pi 5

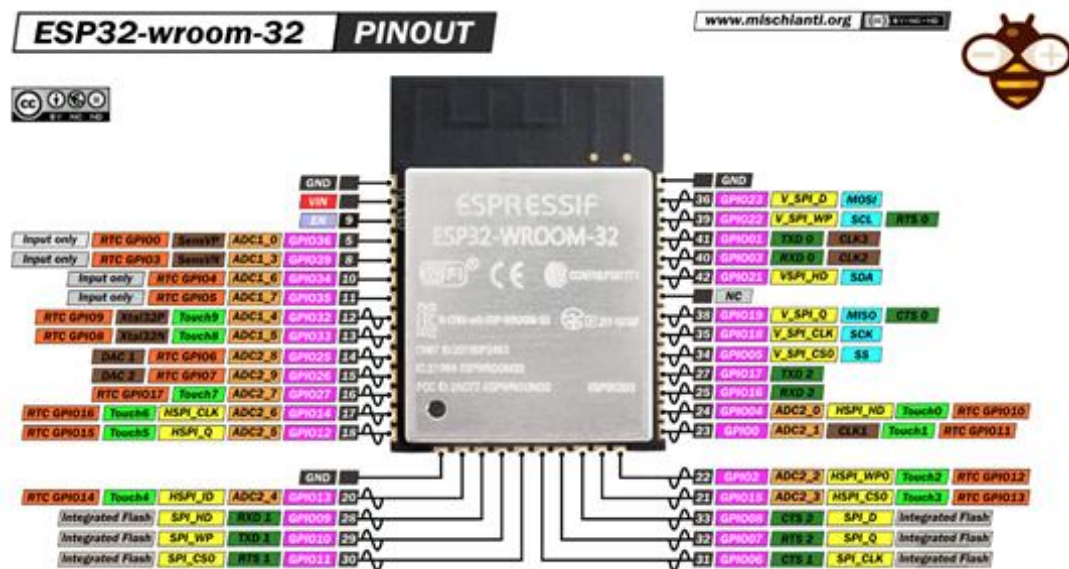
Về nền tảng phần cứng, hệ thống sử dụng máy tính nhúng Raspberry Pi 5 vì bo mạch này được trang bị chip xử lý 4 nhân ARM Cortex-A76 (tốc độ 2.4GHz) [24]. Nhờ sức mạnh này, hệ thống có thể tự chạy mô hình AI nhận diện xe YOLOv26 trực tiếp trên bo mạch một cách nhanh chóng mà không cần kết nối internet hay gửi dữ liệu lên đám mây.

Về dung lượng bộ nhớ, đề tài quyết định chọn phiên bản RAM 2GB để tối ưu chi phí. Lý do là thuật toán của hệ thống chỉ cần chụp và xử lý ảnh tại những thời điểm chuẩn bị chuyển pha đèn để đếm số lượng xe, không theo dõi (tracking) xe di chuyển liên tục. Với cách giải quyết này, mức RAM 2GB là hoàn toàn đủ để hệ thống nhận diện mượt mà, giữ được tốc độ khung hình (FPS) ổn định mà không gây lãng phí tài nguyên.

3.3.4 Khối điều khiển

Trong kiến trúc của đề tài, module ESP32-WROOM-32E-N4 đóng vai trò là khối điều khiển. SoC này sở hữu bộ xử lý lõi kép 32-bit Xtensa LX6 với xung nhịp tối đa 240 MHz, cùng 520 KB SRAM và 4MB bộ nhớ Flash [23][27]. Kiến trúc kép (Dual-core) này cực kỳ quan trọng, cho phép một lõi chuyên trách việc duy trì giao tiếp UART liên tục với máy tính nhúng Raspberry Pi 5, trong khi lõi còn lại thực thi các luồng đếm thời gian pha đèn tín hiệu (Xanh, Vàng, Đỏ) một cách chính xác để không xảy ra hiện tượng trễ hay xung đột hệ thống.

Hình 3.7 mô tả hình ảnh tổng quát của ESP32, gồm sơ đồ chân và chi tiết chức năng các chân của module ESP32.



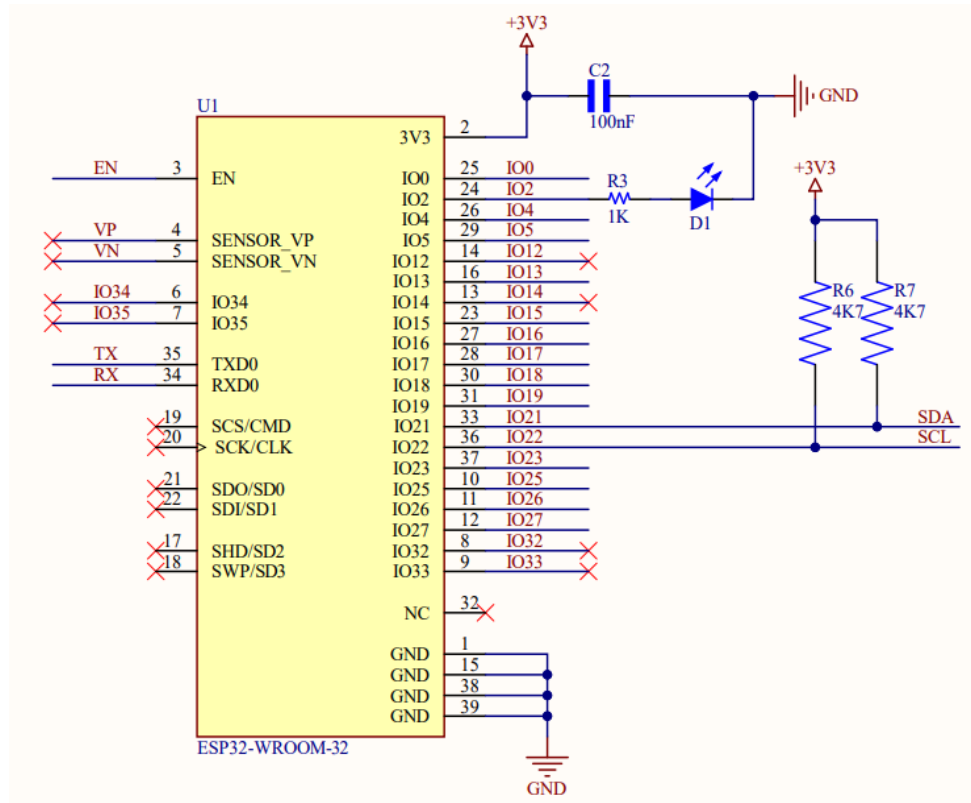
Hình 3.7: Sơ đồ chân và chi tiết chức năng các chân

Bảng 3.1 Mô tả thông số kỹ thuật ESP32, phù hợp cho hệ thống điều khiển đèn giao thông.

Bảng 3.1: Thông số kỹ thuật ESP32

Thông số	Giá trị
Vi điều khiển	ESP32 Xtensa LX6 Dual-Core
Tần số hoạt động	80 – 240 MHz
Điện áp hoạt động	3.0 – 3.6V
Điện áp cấp khuyến nghị	5V qua USB hoặc VIN
SRAM	520 KB
Flash	4 MB (tùy phiên bản)
Wi-Fi	802.11 b/g/n
Bluetooth	Bluetooth v4.2 và BLE
Số chân GPIO	Khoảng 30 GPIO
ADC	12-bit
DAC	2 kênh 8-bit
PWM	Hỗ trợ nhiều kênh PWM
UART	3 UART
I2C	2 I2C
Dòng cấp khuyến nghị	500 mA
Dòng tiêu thụ	~80–260 mA khi Wi-Fi hoạt động
Nhiệt độ hoạt động	-40°C đến 125°C

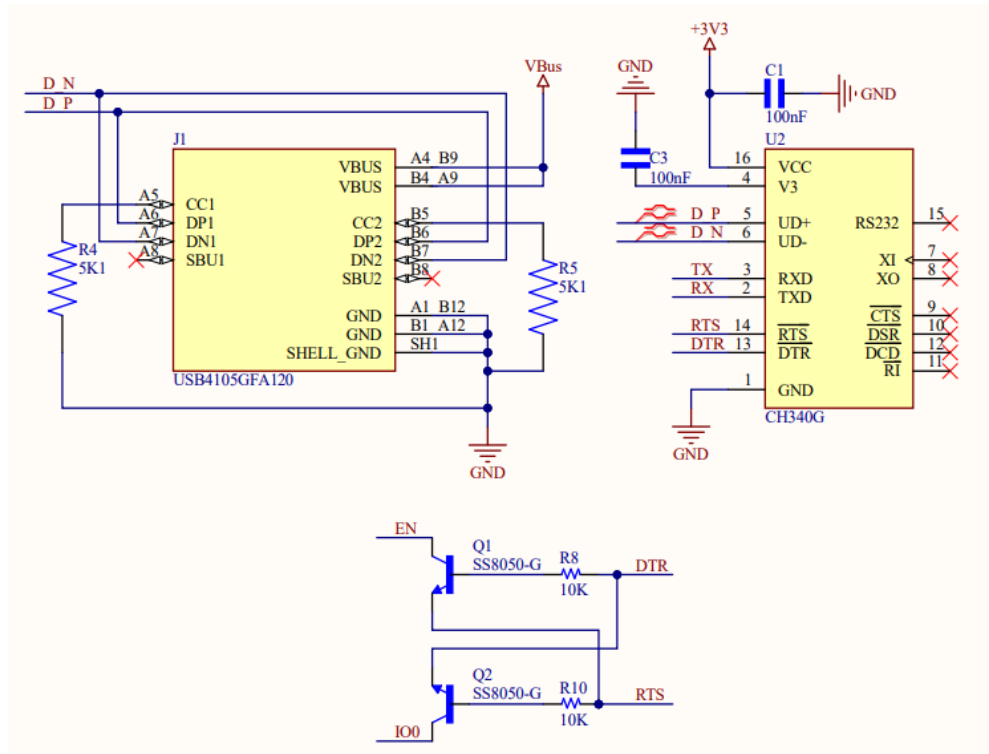
Hình 3.8 mô tả sơ đồ chân kết nối vi xử lý trung tâm ESP32 với mô tả các chân giao tiếp ngoại vi.



Hình 3.8: Sơ đồ kết nối chân chip ESP32

Khối chuyển đổi USB-to-UART có vai trò làm giao tiếp trung gian giữa máy tính và vi điều khiển ESP32. Chức năng chính của khối này là để nạp chương trình cho ESP32 thông qua chuyển đổi tín hiệu truyền thông từ chuẩn USB Type-C của máy tính sang tín hiệu UART. Ngoài nhiệm vụ truyền dữ liệu, khối này còn có khả năng cấp nguồn 5V cho ESP32 hoạt động thông qua cổng USB.

Hình 3.9 mô tả sơ đồ chân kết nối khối chuyển đổi USB sang UART bao gồm 1 cổng USB Type-C, phía dưới là IC chuyển đổi tín hiệu USB sang UART và mạch hỗ trợ tự động bootloader cho MCU.

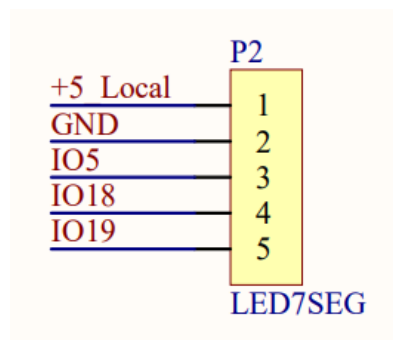


Hình 3.9: Sơ đồ nguyên lý khối chuyển đổi USB to UART

3.3.5 Khối hiển thị

Trong hệ thống điều khiển giao thông, đối với chức năng hiển thị tại chỗ, hệ thống sử dụng module LED 7 đoạn 74HC595. Nhiệm vụ của khối hiển thị thời gian đếm ngược của các pha đèn tín hiệu giao thông. Giúp người tham gia giao thông có thể dễ dàng quan sát nhanh chóng thời gian chuyển pha và tình trạng hoạt động của hệ thống ngay tại vị trí giao lộ.

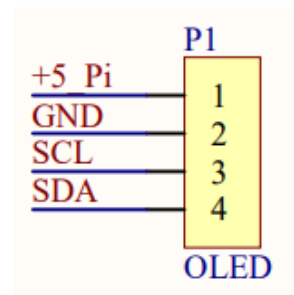
Hình 3.10 mô tả sơ đồ kết nối chân giữa 74HC595 với ESP32 trong thiết kế sơ đồ nguyên lý.



Hình 3.10: Sơ đồ nguyên lý kết nối module led 7 đoạn với ESP32

Để hỗ trợ việc kiểm tra và giám sát kỹ thuật, hệ thống sử dụng một màn hình OLED 0.96 inch, kết nối trực tiếp với Raspberry Pi 5 qua chuẩn giao tiếp I2C. Có nhiệm vụ hiển thị các thông số: dung lượng RAM đang sử dụng, mức độ tải của CPU và nhiệt độ bo mạch khi hệ thống đang chạy mô hình nhận diện AI.

Hình 3.11 là sơ đồ kết nối chân của màn hình oled với hệ thống máy tính nhúng Raspberry Pi 5 thông qua chuẩn I2C.

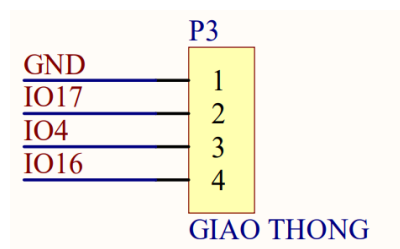


Hình 3.11: Sơ đồ kết nối chân màn hình OLED

3.3.6 Khởi thực thi

Khởi thực thi trực tiếp điều tiết luồng lưu thông tại giao lộ, chuyển đổi các quyết định điều khiển từ vi điều khiển thành tín hiệu thị giác trực quan cho người tham gia giao thông. Khối này được cấu thành từ cụm mô phỏng đèn giao thông bao gồm các bóng LED tiêu chuẩn với ba màu trạng thái quy ước: Đỏ, Vàng và Xanh. Trong hệ thống, module này chịu trách nhiệm giả lập các pha đèn tín hiệu tại giao lộ, giúp người xem nhận biết ngay lập tức làn đường nào đang được phép lưu thông, chuẩn bị dừng hay phải dừng lại theo kịch bản điều khiển.

Hình 3.12 là sơ đồ kết nối chân của mạch hiển thị đèn giao thông kết nối với vi điều khiển ESP32

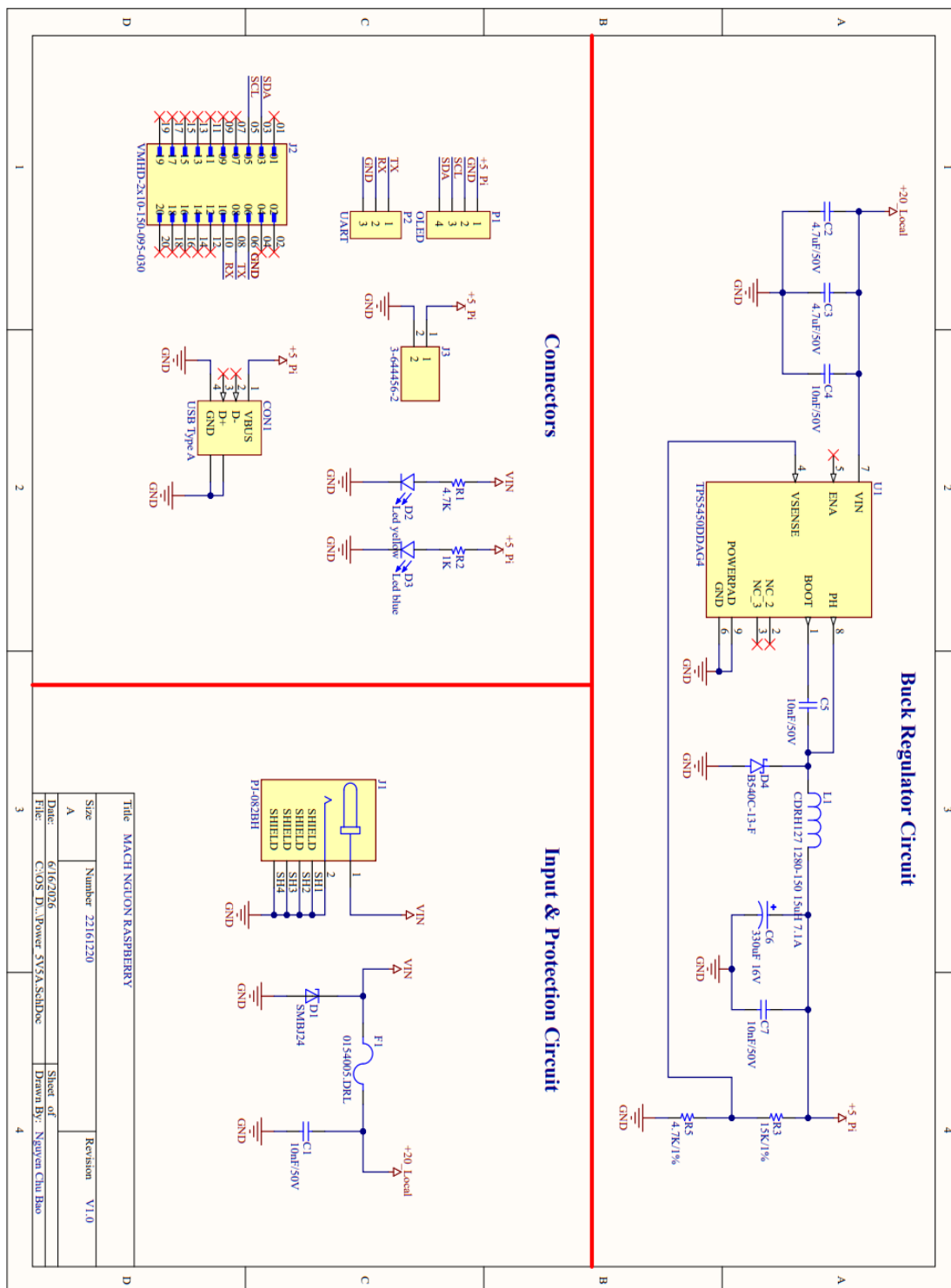


Hình 3.12: Sơ đồ kết nối led giao thông với ESP32

3.4 Sơ đồ nguyên lý

3.4.1 Sơ đồ nguyên lý trung tâm điều khiển Raspberry Pi 5

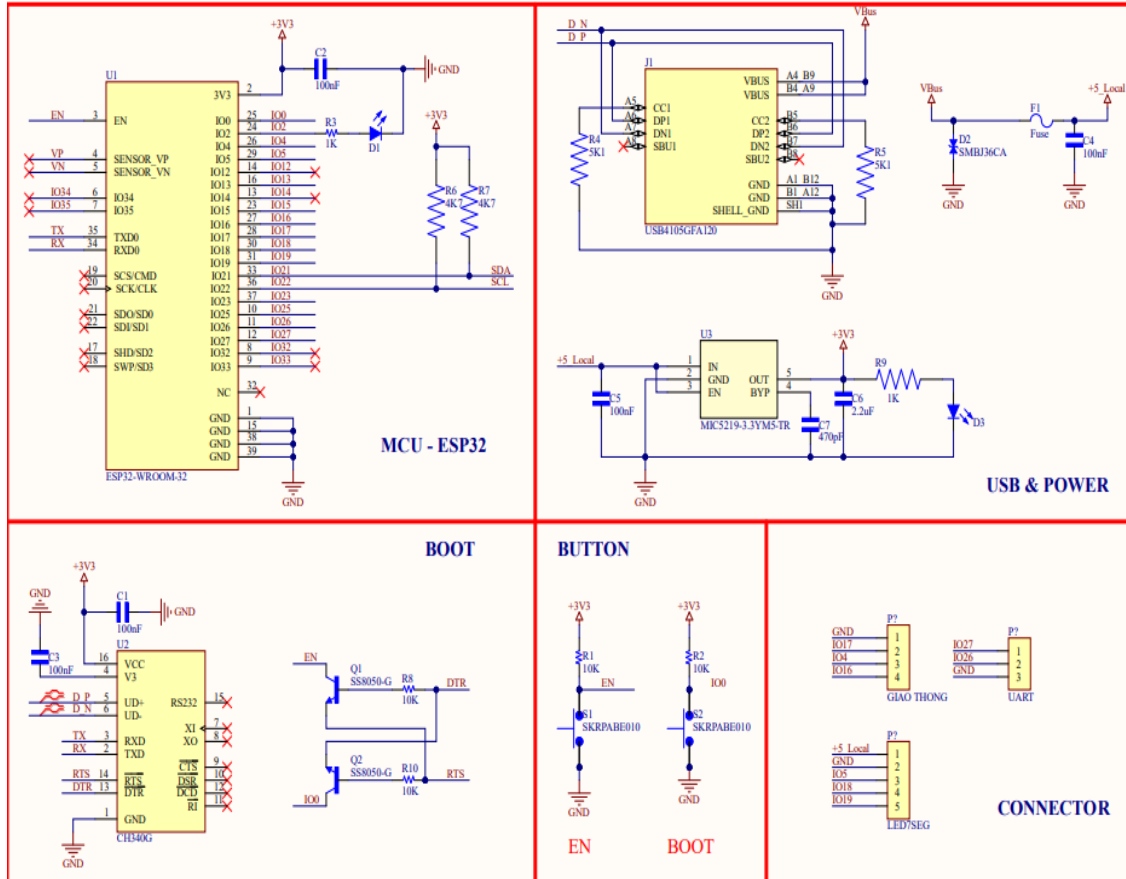
Dựa trên thiết kế chi tiết khối nguồn dòng cao cung ứng cho máy tính nhưng Raspberry Pi 5 trình bày ở các mục trên, sơ đồ nguyên lý tổng thể của mạch nguồn cho Raspberry Pi 5 được tổng hợp lại ở Hình 3.13.



Hình 3.13: Sơ đồ nguyên lý mạch nguồn 5V/5A

3.4.2 Sơ đồ nguyên lý mạch điều khiển ESP32

Dựa trên thiết kế chi tiết và giao tiếp phần cứng của từng khối đã được trình bày ở các mục trên, sơ đồ nguyên lý tổng thể của mạch điều khiển ESP32 được tổng hợp lại ở Hình 3.14.



Hình 3.14: Sơ đồ nguyên lý tổng thể của mạch điều khiển ESP32

3.5 Thiết kế phần mềm

3.5.1 Thiết lập môi trường huấn luyện cho Yolov26n và Simple_GLKA

Quá trình huấn luyện được thực hiện trên GPU NVIDIA RTX 4050 (6GB VRAM, CUDA 12.x), đảm bảo tốc độ hội tụ nhanh và ổn định. Môi trường triển khai thực tế sử dụng Raspberry Pi 5 (ARM Cortex-A76, 2GB RAM) làm nền tảng chính, với Raspberry Pi 4B (ARM Cortex-A72, 4GB RAM) đóng vai trò so sánh hiệu năng nhúng.

Thực nghiệm sử dụng hai tập dữ liệu riêng biệt tương ứng với hai bài toán. Tập dữ liệu phát hiện phương tiện được tự thu thập tại các nút giao thông đô thị TP.HCM, được gán nhãn thủ công thông qua sự giám sát nghiêm ngặt để đảm bảo

chất lượng dữ liệu đầu vào. Tập huấn luyện bao gồm 8799 ảnh chia theo tỉ lệ huấn luyện/kiểm định là 7.903/896, với 5 lớp đối tượng: ô tô, xe van, xe máy, xe buýt và xe tải. Phân bố lớp có sự chênh lệch đáng kể, trong đó ô tô và xe tải chiếm đa số với lần lượt 16.636 và 16.321 mẫu, xe máy ở mức trung bình với 12.652 mẫu, trong khi xe buýt chỉ có 2.246 mẫu và xe van là ít nhất với 1.338 mẫu. Tập dữ liệu phân loại mật độ gồm 3.391 ảnh huấn luyện và 993 ảnh kiểm định, được gán nhãn theo hai lớp kẹt xe và không kẹt xe với phân bố tương đối cân bằng, thuận lợi cho bài toán phân loại.

3.5.2 Thuận toán huấn luyện

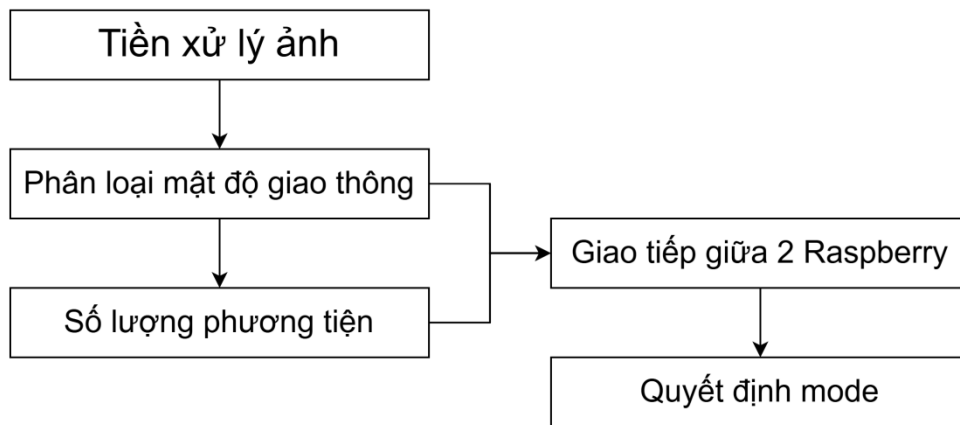
Mô hình YOLOv26n được huấn luyện trong 120 epoch, sử dụng bộ tối ưu MuSGD kết hợp chiến lược ProgLoss theo đề xuất gốc của tác giả. Chiến lược này điều chỉnh trọng số hàm mất mát theo tiến trình huấn luyện, giúp ổn định quá trình hội tụ trong điều kiện phân bố lớp mất cân bằng. Mô hình tốt nhất được lưu dựa trên chỉ số mAP@50-95 trên tập kiểm định.

Mô hình Simple_GLKA được huấn luyện tối đa 25 epoch với cơ chế lưu hai điểm kiểm tra riêng biệt theo độ chính xác cao nhất và giá trị mất mát thấp nhất. Do quy mô tập huấn luyện nhỏ hơn 5.000 ảnh, quy trình tăng cường dữ liệu được cấu hình theo chế độ thích nghi dành cho camera cố định, bao gồm cắt ngẫu nhiên giữ nguyên ngữ cảnh (tỉ lệ 0,8–1,0), biến đổi affine nhẹ (xoay $\pm 5^\circ$, dịch chuyển 5%), lật ngang xác suất 0,5, nhiễu màu sắc giới hạn (hue 0,05) và che khuất ngẫu nhiên xác suất 0,3, nhằm bảo toàn đặc trưng không gian phù hợp với điều kiện triển khai thực tế. Cấu hình tối ưu hóa sử dụng SGD với tốc độ học 0,0015, động lượng 0,9, suy giảm trọng số 7×10^{-4} , kích thước batch 32, ảnh đầu vào $224 \times 224 \times 3$ và tỉ lệ dropout 0,3 trước tầng phân loại. Lịch trình học CosineAnnealingLR được áp dụng để giảm dần tốc độ học, hướng đến hội tụ ổn định trong suốt quá trình huấn luyện. Sau khi hoàn tất, mô hình được tái tham số hóa, hợp nhất các nhánh tích chập song song trong khối GLKA thành một tích chập đơn, tạo ra mô hình triển khai biên tối ưu về tốc độ suy luận.

3.5.3 Quy trình xử lý dữ liệu

Hệ thống không xử lý liên tục từng khung hình mà áp dụng cơ chế kích hoạt có chọn lọc tại hai thời điểm cố định trong chu kỳ đèn: khi đèn đỏ còn 5 giây và đèn xanh còn 2 giây. Tín hiệu kích hoạt được ESP32 chủ động gửi lên Raspberry Pi 5 qua UART dưới dạng chuỗi run hoặc run1, trong đó run kích hoạt luồng quét AI thông thường và run1 kích hoạt xử lý thoát kẹt khẩn cấp.

Khi nhận tín hiệu kích hoạt, quy trình xử lý dữ liệu diễn ra theo bốn giai đoạn tuần tự như minh họa ở Hình 3.15.



Hình 3.15: Quy trình xử lý dữ liệu

Giai đoạn đầu tiên là tiền xử lý ảnh. Khung hình từ camera được trích xuất vùng quan tâm theo đa giác tùy chỉnh bao phủ khu vực mặt đường. Độ sáng được đánh giá qua phân vị 95 của kênh V trong không gian màu HSV; nếu thấp hơn ngưỡng 0,4, mô hình SCI được kích hoạt để tăng cường ánh sáng với hệ số 1,5 lần trên miền số thực trước khi cắt giới hạn, đảm bảo vùng tối được kéo sáng mà không làm vỡ nét vùng đã đủ sáng. Ảnh sau tăng cường được tạo ra hai đầu ra song song: thu nhỏ về 224×224 cho Simple_GLKA và letterbox về 480×480 cho YOLOv26n.

Giai đoạn thứ hai là phân loại mật độ giao thông. Simple_GLKA nhận ảnh 224×224 và phân loại trạng thái tổng thể thành kẹt xe hoặc thoáng. Nếu phát hiện kẹt xe, hệ thống chuyển sang vòng lặp giám sát liên tục, cập nhật trạng thái mỗi giây cho đến khi tình trạng được giải tỏa.

Giai đoạn thứ ba là đếm số lượng phương tiện. Khi trạng thái thoáng được xác nhận, YOLOv26n nhận ảnh letterbox 480×480 và đếm số lượng phương tiện theo từng lớp đối tượng. Kết quả được ánh xạ sang chế độ thời gian đèn từ m1 đến m4

theo ngưỡng cố định: dưới 2 xe là m1, từ 2 đến 4 xe là m2, từ 5 đến 7 xe là m3, và từ 8 xe trở lên là m4.

Giai đoạn thứ tư là giao tiếp giữa hai Raspberry Pi 5 và ra quyết định chế độ. Sau mỗi bước phân loại và đếm xe, hai trạm đồng bộ kết quả với nhau qua Ethernet bằng cơ chế bắt tay có gắn nhãn giai đoạn. Khối logic quyết định tổng hợp trạng thái cục bộ và trạng thái từ trạm đối diện để xác định lệnh điều khiển cuối cùng: ưu tiên hướng kẹt nếu có, hoặc cấp thời gian xanh theo nhánh mật độ cao hơn nếu cả hai đều thoáng. Lệnh được gửi xuống ESP32 qua UART ở tốc độ 115200 bps để vận hành cụm đèn tín hiệu.

3.5.4 Triển khai hệ thống trên Raspberry Pi 5

Sau khi huấn luyện, hai mô hình được chuyển đổi sang định dạng phù hợp với đặc thù phần cứng nhúng. Mô hình YOLOv26n được chuyển sang định dạng NCNN với lượng tử hóa 16-bit, giảm đáng kể dung lượng bộ nhớ và tăng tốc độ suy luận trên kiến trúc ARM mà không phụ thuộc vào runtime ngoài. Mô hình Simple_GLKA sau bước tái tham số hóa được xuất sang định dạng ONNX, cho phép suy luận thông qua ONNX Runtime trên Raspberry Pi 5 với hiệu năng ổn định và khả năng tương thích đa nền tảng.

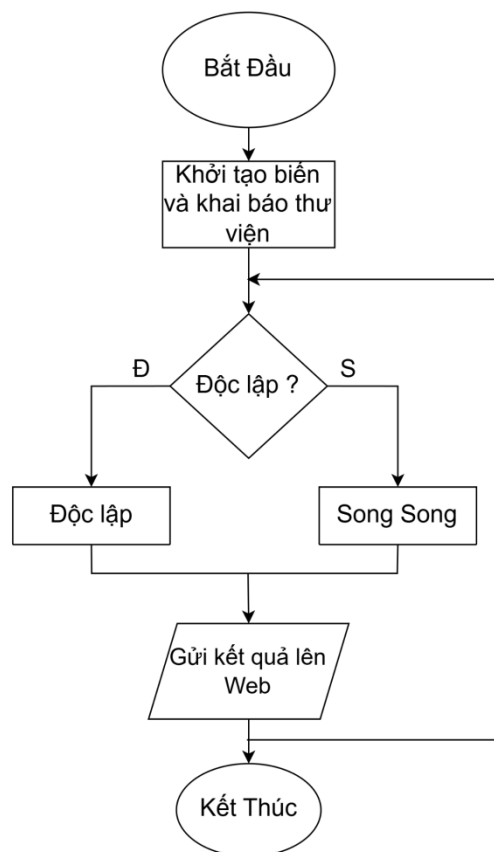
Về triển khai vận hành, hệ thống được cấu hình chạy nền thông qua hai dịch vụ systemd độc lập: traffic_ai.service đảm nhận toàn bộ luồng xử lý AI và điều khiển đèn, oled.service phụ trách hiển thị thông số hệ thống lên màn hình OLED. Cả hai dịch vụ được thiết lập tự khởi động khi Raspberry Pi 5 được cấp nguồn, đảm bảo hệ thống hoạt động liên tục mà không cần can thiệp thủ công.

Một giao diện web giám sát nội bộ được tích hợp cho phép quan sát trạng thái hệ thống theo thời gian thực trong quá trình thực nghiệm, bao gồm kết quả phân loại mật độ từ Simple_GLKA, ảnh nhận diện phương tiện từ YOLOv26n, độ sáng khung hình, trạng thái kết nối Ethernet giữa hai trạm và lệnh điều khiển đang được gửi xuống ESP32.

3.6 Lưu đồ giải thuật hệ thống giao thông

3.6.1 Xây dựng lưu đồ giải thuật hệ thống

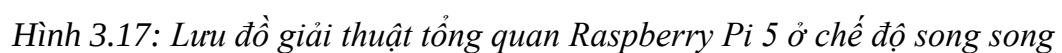
Để định hướng kịch bản xử lý cho toàn bộ hệ thống giao thông thông minh, chương trình chính trên máy tính nhúng Raspberry Pi 5 được cấu trúc theo một luồng logic tổng quan, làm căn cứ điều hướng cho các phân hệ chức năng chuyên biệt. Hình 3.16 thể hiện lưu đồ giải thuật và quy trình xử lý logic chính trên Raspberry Pi 5.



Hình 3.16: Lưu đồ giải thuật chương trình chính trên Raspberry Pi 5.

Đầu tiên, chương trình chính thực hiện khối Khởi tạo biến và khai báo thư viện để thiết lập môi trường phần mềm nền tảng. Tiếp theo, thuật toán tiến hành kiểm tra điều kiện tại khối “Độc lập?” để xác định kịch bản vận hành thực tế của mô hình đèn: nếu đúng, hệ thống sẽ rẽ nhánh sang thực thi phân hệ Độc lập, nếu sai chương trình sẽ kích hoạt phân hệ Song song liên kết mạng liên trạm. Cuối cùng, sau khi một trong hai phân hệ hoàn thành chu kỳ tính toán và điều khiển đèn, luồng dữ liệu tổng hợp sẽ được chuyển tiếp qua khối Gửi kết quả lên web để cập nhật

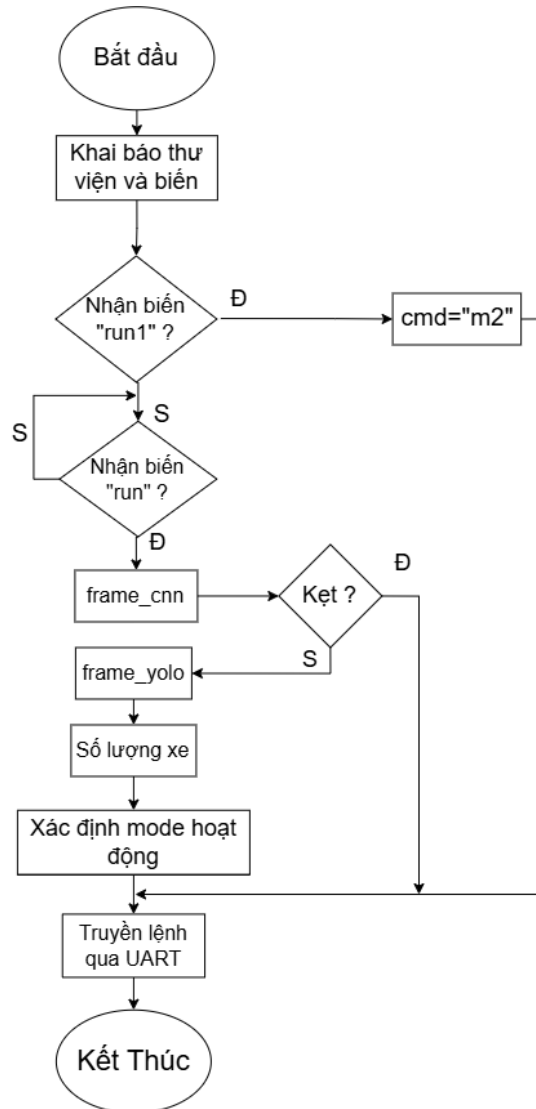
Hình 3.17 thể hiện lưu đồ giải thuật tổng quan Raspberry Pi 5 khi vận hành ở chế độ song song.



44

trích xuất ảnh đưa vào Simple_GLKA kiểm tra trạng thái kẹt, nếu kẹt hai trạm Pi5 gửi "A/B" qua Ethernet và chạy vòng lặp CNN liên tục đến khi nhận hạ pha "a"/"b" rồi xuất qua UART kết thúc, nếu không kẹt, gửi "a/b" đồng bộ trạm, chuyển sang YOLO đếm xe, tính $Xe = \max(xe_a, xe_b)$, xác định mode m1–m4 và gửi mã điều khiển xuống ESP32 qua UART kết thúc chu kỳ.

Hình 3.18 thể hiện lưu đồ giải thuật và quy trình xử lý logic chi tiết của máy tính nhúng Raspberry Pi 5 khi vận hành ở chế độ đơn nhánh.

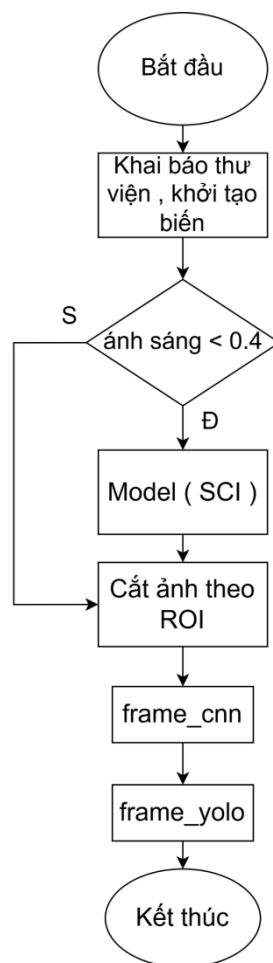


Hình 3.18: Lưu đồ giải thuật tổng quan Raspberry Pi 5 ở chế độ độc lập

Đầu tiên, hệ thống khởi tạo cấu hình và tiến hành kiểm tra biến khẩn cấp run1. Nếu đúng (nhánh Đ), lệnh cấu hình an toàn cmd="m2" lập tức gửi qua khối truyền lệnh UART xuống ESP32 và kết thúc chu kỳ, nếu sai (nhánh S), hệ thống sẽ đợi nhận tín hiệu run từ ESP32 để tiến hành trích xuất dữ liệu ảnh qua khối frame_cnn.

Tiếp theo, mạng Simple_GLKA kiểm tra trạng thái Kẹt: nếu đúng, luồng thuật toán sẽ rẽ nhánh để xuất ngay lệnh điều khiển qua khối UART và kết thúc chu kỳ xử lý khẩn cấp. Nếu sai, chương trình tiếp tục chuyển xuống phân hệ đếm xe chi tiết thông qua khối YOLO (frame_yolo). Cuối cùng, khối xác định mode hoạt động sẽ quyết định một mã chế độ đèn tối ưu từ m1 đến m4 dựa trên giá trị tổng hợp Xe, truyền trực tiếp mã này xuống vi điều khiển chấp hành qua khối Truyền lệnh UART và chính thức kết thúc chu kỳ.

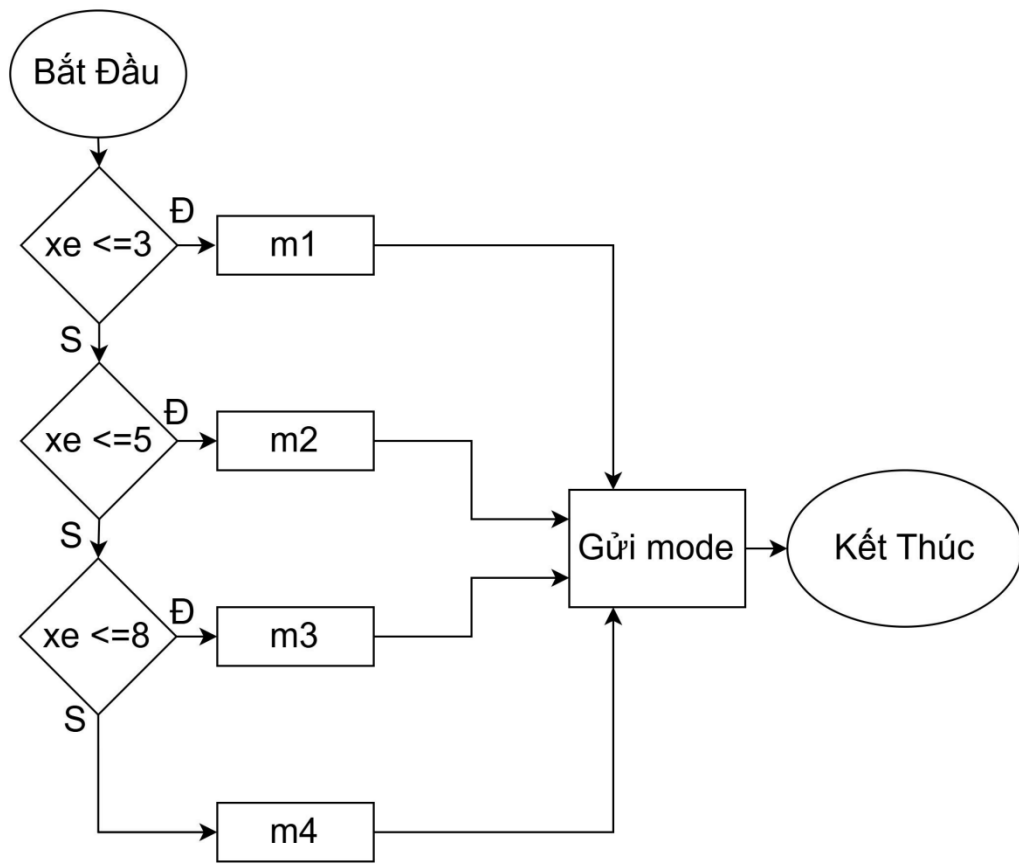
Hình 3.19 thể hiện lưu đồ giải thuật và quy trình xử lý logic của khối tiền xử lý ảnh.



Hình 3.19: Lưu đồ giải thuật tiền xử lý ảnh

Tại đây, hệ thống kiểm tra độ sáng môi trường của những bức ảnh chụp được; nếu ánh sáng yếu (< 0.4), ảnh được đưa qua mô hình SCI để tối ưu tương phản, ngược lại sẽ tiến hành cắt vùng quan tâm (ROI) trực tiếp để xuất ra hai luồng dữ liệu frame_cnn và frame_yolo.

Hình 3.20 thể hiện lưu đồ giải thuật và quy trình xử lý logic của thời gian đèn.

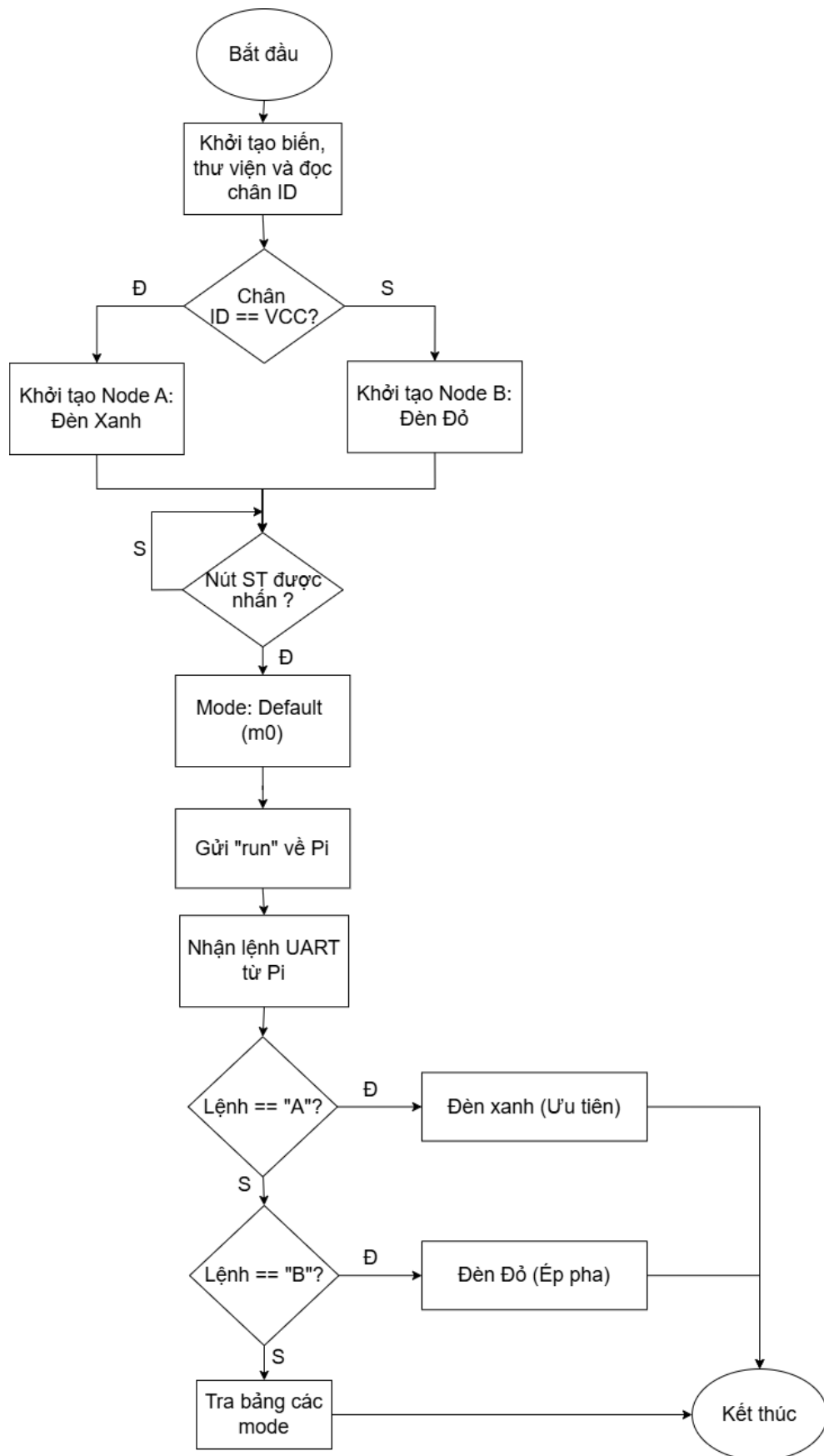


Hình 3.20: Lưu đồ giải thuật quyết định chế độ thời gian đèn

Khởi xác định mode hoạt động đóng vai trò xử lý logic chiến lược, tiếp nhận tham số số lượng xe đầu vào từ mô hình YOLO. Thuật toán tiến hành so sánh tuyến tính với các ngưỡng mật độ được cài đặt sẵn để rẽ nhánh điều kiện: nếu số xe < 3 kích hoạt chế độ thấp tải m1, số xe < 5 kích hoạt chế độ trung bình m2, xe < 8 kích hoạt chế độ cao tải m3, và các trường hợp còn lại chuyển sang chế độ giới hạn m4. Kết thúc so sánh, mã Mode đèn được chốt để gửi xuống tầng điều khiển.

3.6.2 Lưu đồ giải thuật khối điều khiển

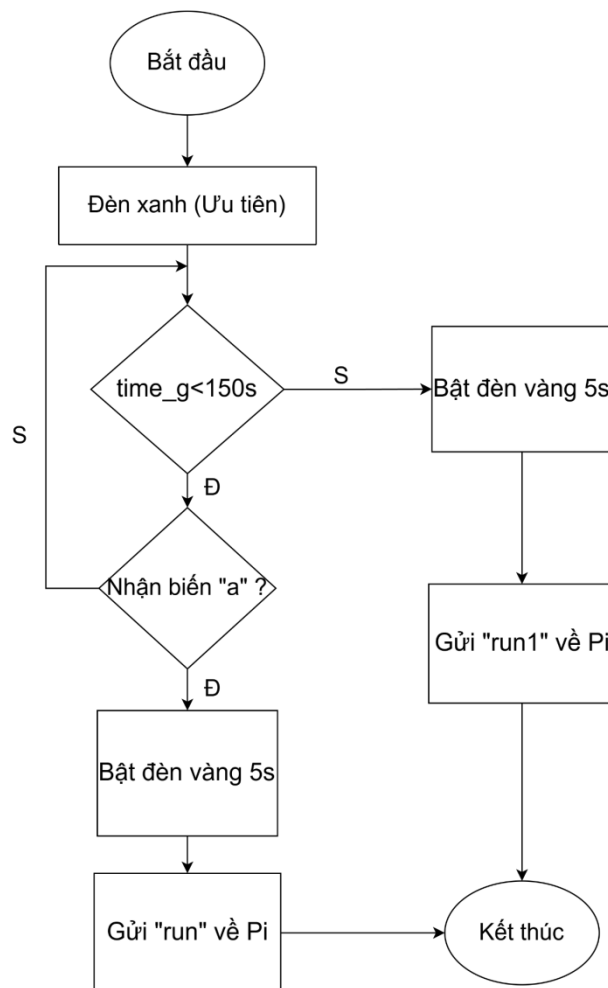
Hình 3.21 thể hiện lưu đồ giải thuật và quy trình xử lý logic của vi điều khiển ESP32 khi tiếp nhận dữ liệu điều khiển từ máy tính nhúng Raspberry Pi 5.



Hình 3.21: Lưu đồ giải thuật ESP32

Thuật toán khởi chạy bằng việc kiểm tra trạng thái Chân ID phần cứng để tự động phân biệt và thiết lập pha đèn ban đầu cho Node A (Xanh) hoặc Node B (Đỏ). Sau khi nhận tín hiệu kích hoạt từ nút nhấn ST, hệ thống vào chế độ mặc định (m0), sau khi chạy xong chế độ (m0) thì gửi lệnh "run" qua UART để báo cáo về khối xử lý trung tâm. Tại vòng lặp chính, vi điều khiển liên tục đọc dữ liệu UART và rẽ nhánh thực thi các chương trình con tương ứng: Đèn xanh ưu tiên (nếu lệnh là "A"), Đèn đỏ ép pha (nếu lệnh là "B"), nếu không thỏa mãn hai điều kiện trên A và B tiến hành Tra bảng các mode (cho các kịch bản còn lại). Sau khi thực thi xong, hệ thống kết thúc một chu kỳ quét lệnh và tự động quay lại trạng thái chờ lệnh mới.

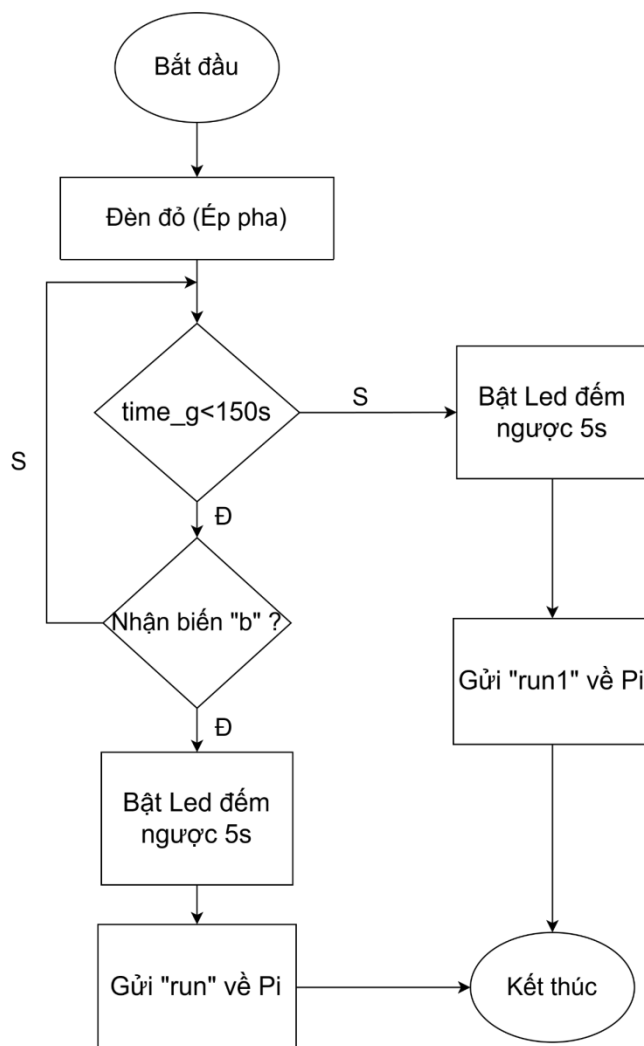
Hình 3.22 mô tả thuật toán xử lý trạng thái Đèn xanh (Ưu tiên) của vi điều khiển ESP32 khi có sự điều phối từ Raspberry Pi 5.



Hình 3.22: Lưu đồ thuật toán xử lý đèn xanh (ưu tiên)

Chương trình bắt đầu bằng việc bật đèn Xanh để ưu tiên khi nhận lệnh “A” và đi vào vòng lặp giám sát liên tục với thời gian tối đa là 150 giây. Trong khoảng thời gian này, nếu vi điều khiển nhận được tín hiệu "a" (báo hiệu ngã tư đã hết unction), hệ thống sẽ chủ động ngắt pha Xanh sớm, chuyển sang 5 giây Đèn Vàng đậm, gửi lệnh "run" báo cáo về Pi và kết thúc hàm. Ngược lại, nếu thời gian sáng vượt quá ngưỡng 150s mà vẫn chưa hết unction, hệ thống sẽ tự động ép thoát vòng lặp, bật 5 giây Vàng đậm, xuất lệnh "run1" và kết thúc kịch bản.

Hình 3.23 mô tả thuật toán xử lý trạng thái Đèn đỏ (Ép pha) của vi điều khiển ESP32 khi có sự điều phối từ Raspberry Pi 5.

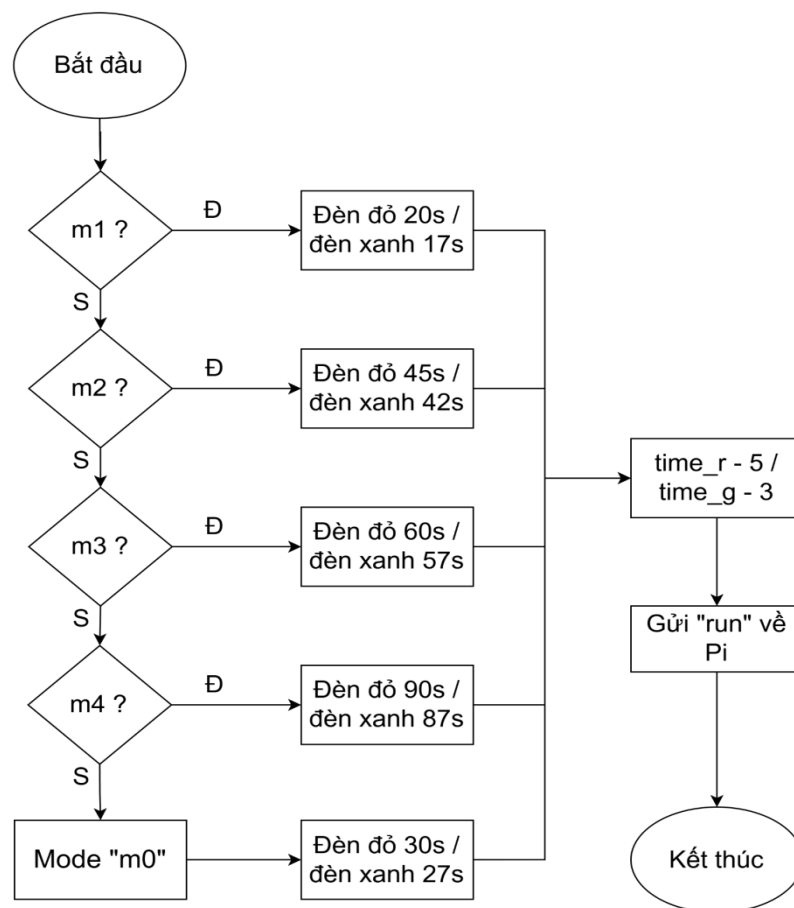


Hình 3.23: Lưu đồ thuật toán xử lý đèn đỏ (ép pha)

Khi kích hoạt, hệ thống lập tức bật đèn Đỏ và chủ động tắt toàn bộ LED đếm ngược để báo hiệu trạng thái chờ đặc biệt. Vi điều khiển đi vào vòng lặp giám sát

tối đa 150 giây. Nếu nhận được tín hiệu "b" trong khoảng thời gian này, hệ thống sẽ thoát vòng lặp, bật lại LED và đếm lùi 5 giây cuối, sau đó gửi lệnh "run" về máy tính trung tâm và kết thúc. Trong trường hợp quá 150 giây mà chưa có lệnh giải tỏa, hệ thống vẫn tự động ngắt vòng lặp, hiển thị đếm lùi 5 giây, xuất báo cáo "run1" và kết thúc.

Hình 3.24 mô tả luồng tra bảng mode của vi điều khiển ESP32 khi có sự điều phối từ Raspberry Pi 5.



Hình 3.24: Lưu đồ thuật toán tra bảng của ESP32

Hệ thống sử dụng cấu trúc rẽ nhánh tuần tự để kiểm tra biến Mode. Nếu giá trị khớp với các kịch bản từ m1 đến m4, ESP32 sẽ nạp cấu hình thời gian đếm ngược tương ứng cho cụm đèn. Nhằm tối ưu thời gian xử lý của AI, khi đèn Đỏ còn 5 giây (hoặc đèn Xanh còn 3 giây), hệ thống sẽ gửi lệnh "run" về máy tính nhúng Pi và kết thúc. Đặc biệt, nếu không khớp với bất kỳ mode nào, vi điều khiển sẽ tự động thực thi kịch bản dự phòng m0 đảm bảo duy trì an toàn giao thông một cách độc lập.

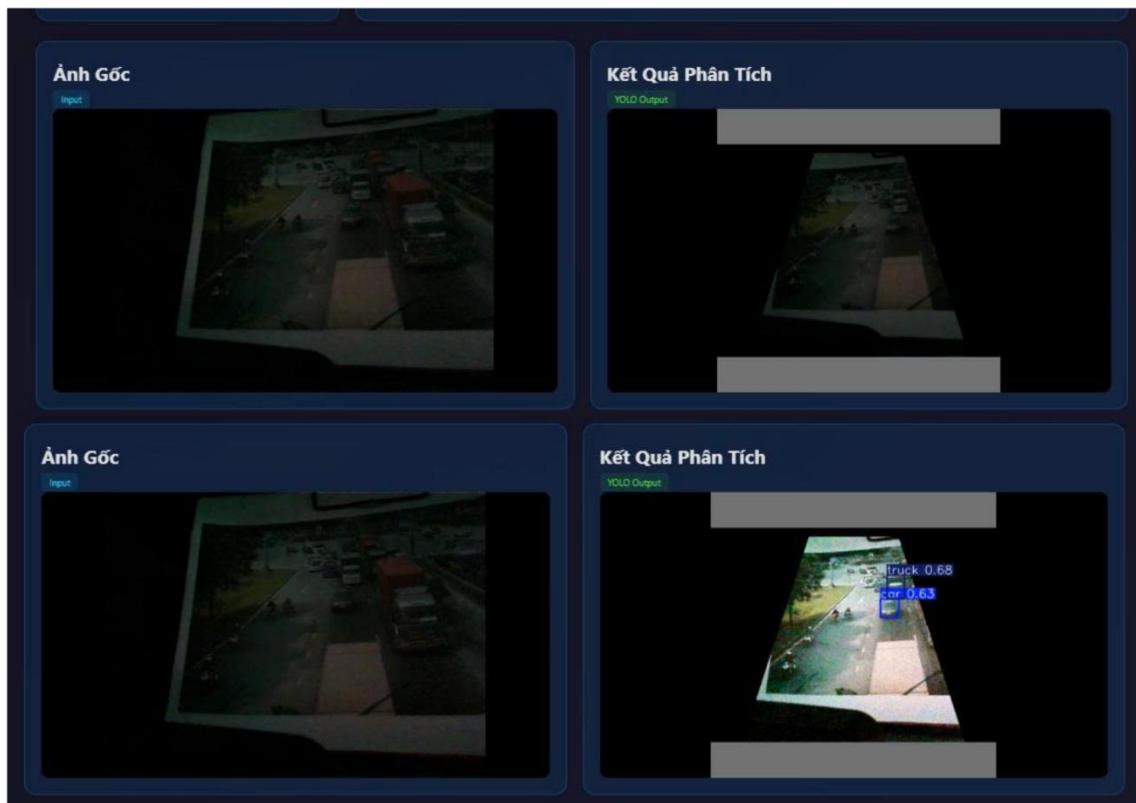
CHƯƠNG 4

KẾT QUẢ VÀ ĐÁNH GIÁ

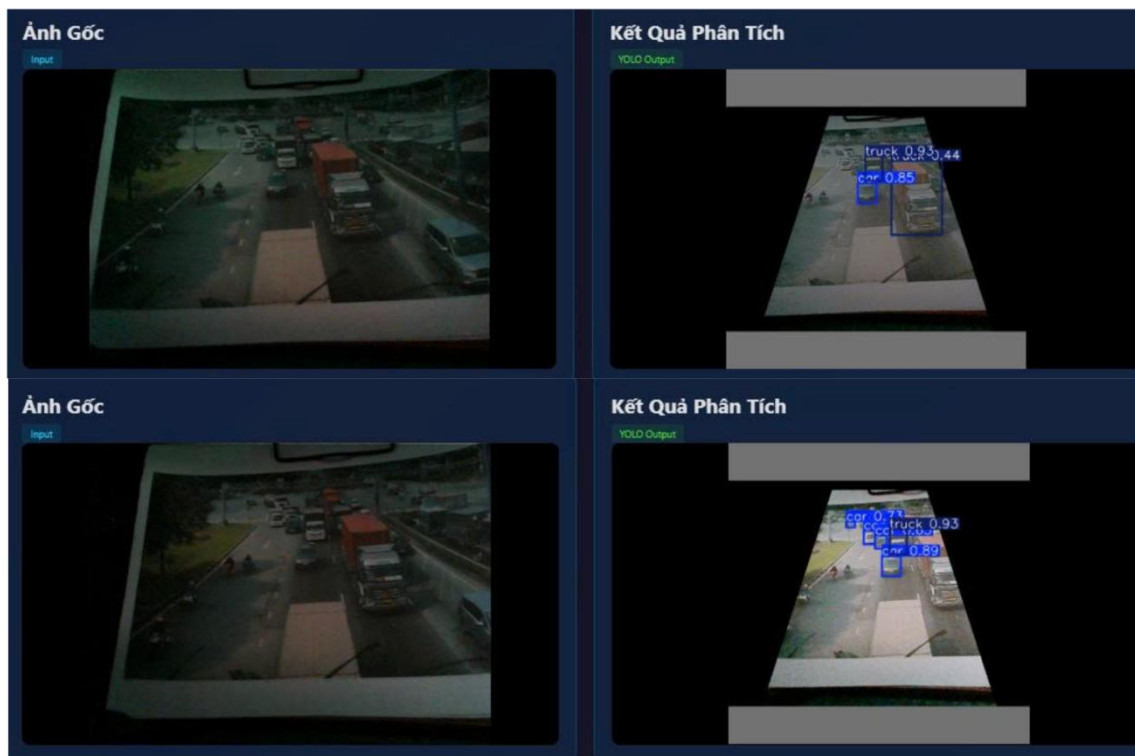
4.1 Kết quả đánh giá độ chính xác dựa trên số liệu huấn luyện

4.1.1 Kết quả đánh giá với tiền xử lý ảnh

Hình ảnh 4.1 và 4.2 dưới đây so sánh kết quả nhận diện phương tiện trong điều kiện ánh sáng yếu cho thấy sự khác biệt rõ rệt giữa ảnh chưa qua tiền xử lý và ảnh đã được tăng cường ánh sáng. Ở trường hợp đầu, ảnh đầu vào ảnh rất tối khiến mô hình YOLOv26n không phát hiện được bất kỳ phương tiện nào. Sau khi áp dụng bước tiền xử lý ánh sáng, vùng ảnh được khôi phục rõ nét hơn, các đường viền và hình dạng phương tiện trở nên phân biệt được, từ đó mô hình nhận diện thành công nhiều phương tiện với độ tin cậy chấp nhận được. Kết quả này khẳng định vai trò thiết yếu của bước tiền xử lý ánh sáng trong việc nâng cao hiệu năng nhận diện phương tiện ở điều kiện môi trường bất lợi.



Hình 4.1: So sánh kết quả nhận diện phương tiện trong điều kiện ánh sáng yếu (độ sáng 0.2)



Hình 4.2: So sánh kết quả nhận diện phương tiện trong điều kiện ánh sáng yếu (độ sáng 0.3)

Tuy nhiên, bước tăng cường ánh sáng chỉ phát huy hiệu quả trong điều kiện thiếu sáng nghiêm trọng. Khi độ sáng ảnh đầu vào đạt mức 0.4 trở lên, việc áp dụng tiền xử lý ánh sáng không còn mang lại lợi ích mà thậm chí làm giảm hiệu năng so với việc đưa ảnh gốc trực tiếp vào mô hình. Nguyên nhân là do YOLOv26n được huấn luyện trên tập dữ liệu vốn đã bao gồm các mẫu ảnh trong điều kiện thiếu sáng, nên bản thân mô hình đã có khả năng thích ứng nhất định với môi trường tối. Việc can thiệp thêm vào ảnh ở mức sáng đủ cao có thể làm biến đổi phân phối đặc trưng đầu vào theo hướng bất lợi, dẫn đến kết quả nhận diện kém hơn.

4.1.2 Phân loại mật độ giao thông (Simple_GLKA)

Để xác định cấu hình tối ưu, sáu biến thể được huấn luyện và đánh giá trên cùng tập dữ liệu với cùng siêu tham số. Kết quả được tổng hợp tại Bảng 4.1.

Bảng 4.1: Bảng số liệu các biến thể và so sánh các phiên bản của Simple_GLKA.

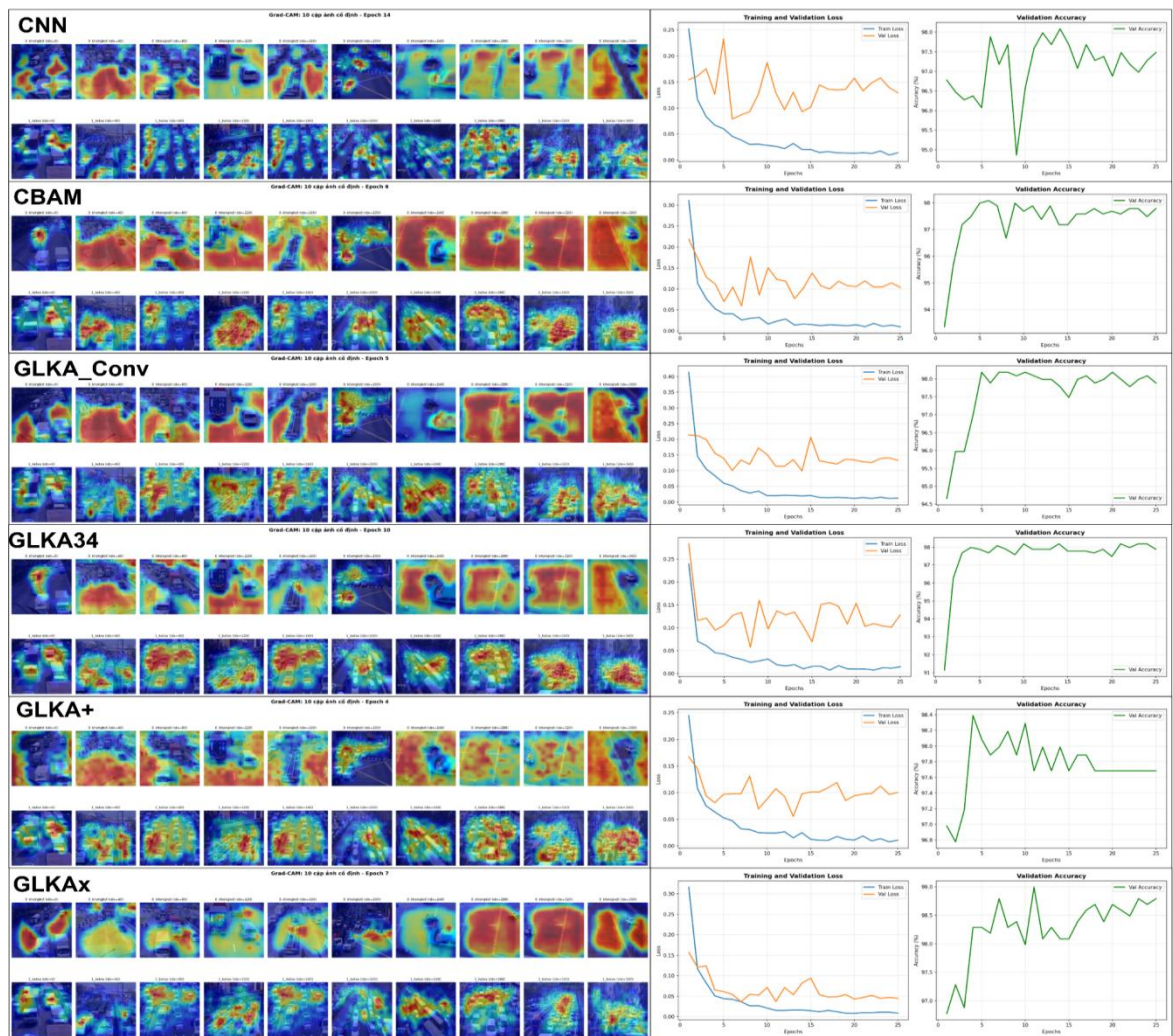
Model	Precision	Recall	Accuracy (%)	Val_loss	Parameters(M)
CNN	0.9812	0.9809	98,09	0.093	0.234
CBAM	0.9814	0.9809	98.09	0.1043	0.278
GLKA_conv	0.9824	0.9819	98.19	0.1401	0.276
GLAK+	0.9844	0.9839	98,39	0.0811	0.278
GLKA34	0.9825	0.9819	98.19	0.0973	0.27
GLKAx	0.99	0.9899	98.99	0.037	0.278

Quan sát tổng thể cho thấy biến thể GLKAx đạt hiệu năng vượt trội so với tất cả các biến thể còn lại trên mọi chỉ số đánh giá. So với mô hình mạng nơ-ron tích chập cơ sở, GLKAx cải thiện độ chính xác thêm 0,90% (từ 98,09% lên 98,99%), trong khi giá trị mất mát kiểm định giảm từ 0,0930 xuống còn 0,0370 — tương đương mức giảm khoảng 60%. Sự sụt giảm đáng kể này phản ánh khả năng hội tụ tốt hơn và mức độ khớp quá mức thấp hơn rõ rệt so với đường cơ sở.

Khi so sánh hai biến thể sử dụng phép toán kết hợp khác nhau GLKAx (nhân theo từng phần tử) và GLAK+ (cộng trực tiếp) có thể thấy rằng phép nhân theo từng phần tử mang lại hiệu quả cao hơn đáng kể. Cụ thể, GLKAx đạt độ chính xác 98,99% với giá trị mất mát kiểm định chỉ 0,037, trong khi GLAK+ chỉ đạt 98,39% với giá trị mất mát kiểm định 0,0811. Điều này xuất phát từ bản chất của hai phép toán: phép nhân theo từng phần tử tạo ra tương tác phi tuyến giữa cơ chế chú ý theo kênh và cơ chế chú ý theo không gian, cho phép mô hình học được mối quan hệ phức tạp hơn giữa "đặc trưng nào quan trọng" và "vị trí nào cần chú ý". Trong khi đó, phép cộng chỉ đơn thuần tổng hợp hai tín hiệu mà không tạo ra cơ chế điều chỉnh lẫn nhau giữa hai nhánh, dẫn đến giá trị mất mát kiểm định cao hơn và độ ổn định huấn luyện kém hơn.

Biến thể GLKA34 là phiên bản chỉ áp dụng cơ chế chú ý tại khối thứ 3 và khối thứ 4, bỏ qua khối thứ 2 so với GLKAx nên chỉ đạt độ chính xác 98,19%, gần sát mức của mạng nơ-ron tích chập cơ sở (98,09%). Kết quả này cho thấy sự tham gia của khối thứ 2 đóng vai trò không thể thiếu trong toàn bộ kiến trúc: các đặc trưng được trích xuất ở giai đoạn sớm này mang thông tin cục bộ quan trọng mà nếu bỏ qua, mô hình sẽ mất đi một phần khả năng tổng hợp ngữ cảnh không gian ở nhiều tầng trừu tượng khác nhau. Điều đó dẫn đến vùng tiếp nhận hiệu dụng bị thu hẹp và làm suy giảm khả năng phân biệt giữa các lớp.

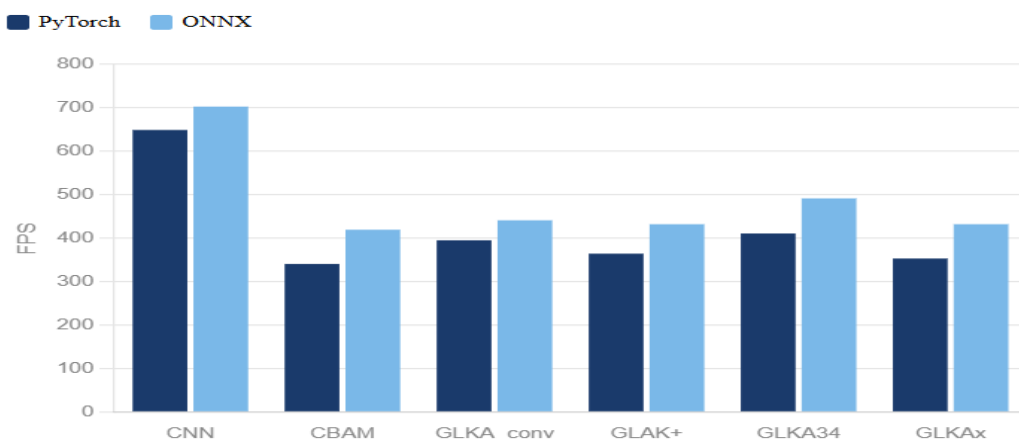
Bản đồ nhiệt Grad-CAM trong Hình 4.2 cung cấp góc nhìn trực quan về khả năng trích xuất đặc trưng của từng biến thể. Trực quan hơn về cách mô hình mạng tích chập đã được thiết kế để phù hợp với bài toán phân loại mật độ giao thông yêu cầu tốc độ cao.



Hình 4.3: Bản đồ Grad-CAM của đặc trưng các mô hình biến thể

Mạng nơ-ron tích chập cơ sở SimpleEfficientCNN [13] cho thấy vùng kích hoạt phân tán và thiếu nhất quán, chưa phản ánh đúng phân bố mật độ trong ảnh. Mất mát kiểm định dao động mạnh và độ chính xác trồi sụt liên tục, cho thấy mô hình chưa học được đặc trưng có tính phân biệt rõ ràng. Biến thể sắp xếp đa chú ý như CBAM [2] cải thiện một phần, vùng kích hoạt bắt đầu bám vào các cụm phương tiện hơn ở phân loại kẹt xe, nhưng vẫn bị ảnh hưởng bởi nhiễu nền và đường độ chính xác còn xuất hiện các điểm sụt đột ngột. GLKA_Conv có vùng kích hoạt rõ hơn ở một số mẫu nhưng chưa ổn định, mất mát kiểm định dao động khá mạnh ở giai đoạn đầu huấn luyện. GLKA34 là biến thể kém ổn định nhất, nhiễu ảnh bản đồ nhiệt học nhiễu làm đặc trưng, cho thấy mô hình không xác định được vùng đặc trưng có ý nghĩa. Độ chính xác dao động đều gần như không thay đổi độ chính xác, phản ánh hệ quả của việc bỏ qua thông tin từ khối thứ hai khiến mô hình thiếu đi một tầng trừu tượng quan trọng để đọc mật độ giao thông. GLKA+ có bản đồ nhiệt tương đối hơn nhưng vẫn còn bị nhiễu nền nhiều, bỏ qua các đặc trưng quan trọng đường độ chính xác dao động thất thường, cho thấy phép cộng chưa tạo được sự phối hợp hiệu quả giữa hai luồng chú ý.

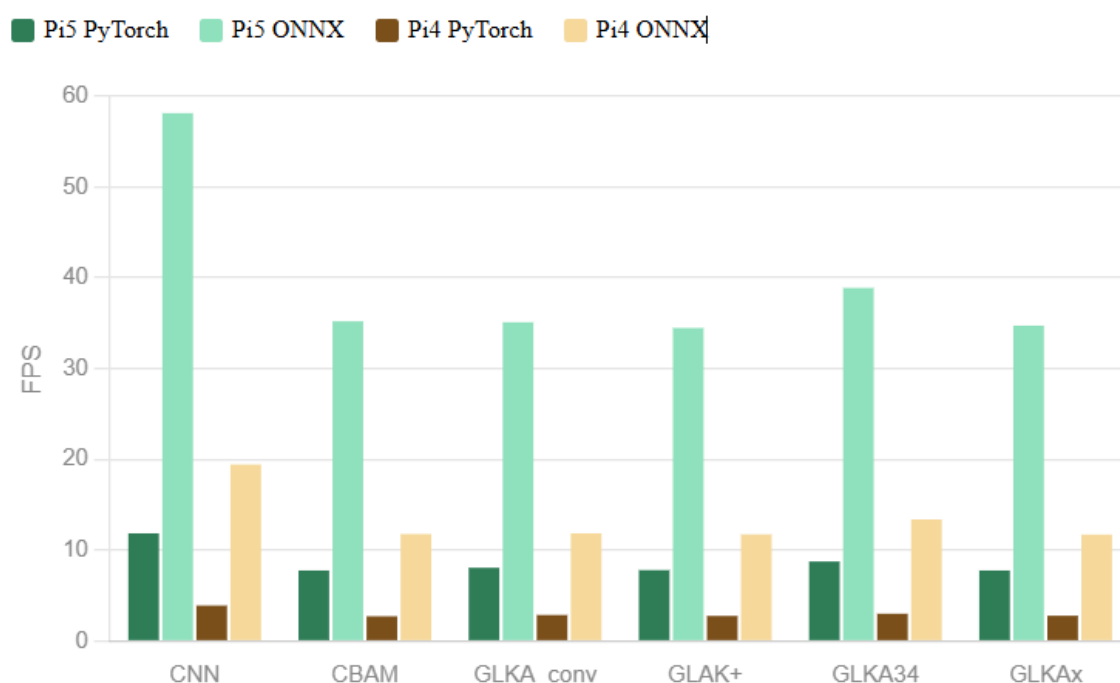
GLKAx cho thấy vùng kích hoạt nhất quán và có ngữ nghĩa rõ ràng nhất: với ảnh kẹt xe, vùng đỏ bao trùm toàn bộ cụm phương tiện dày đặc; với ảnh thông thoáng, mô hình nhận diện được phần mặt đường trống như đặc trưng phân biệt. Mất mát kiểm định thấp và ổn định, đường độ chính xác hội tụ tốt, xác nhận cơ chế kết hợp theo kênh và theo không gian bằng phép nhân từng phần tử giúp mô hình học được đúng tín hiệu đặc trưng của mật độ giao thông.



Hình 4.4: Tốc độ thực thi các biến thể trên GPU (RTX4050)

Kết quả đo kiểm tốc độ suy luận cho thấy sự khác biệt rõ rệt giữa các nền tảng phần cứng và định dạng triển khai. Trên GPU, tất cả các biến thể đều đạt tốc độ rất cao, trong đó CNN cơ sở có tốc độ cao nhất với định dạng ONNX, trong khi các biến thể phức tạp hơn như GLKA+, GLKAx và CBAM có tốc độ thấp hơn do cấu trúc sâu hơn. Định dạng ONNX nhìn chung mang lại cải thiện nhỏ so với PyTorch trên GPU do chi phí tính toán đã được tối ưu hóa bởi phần cứng tăng tốc.

Pi5 & Pi4 — PyTorch vs ONNX



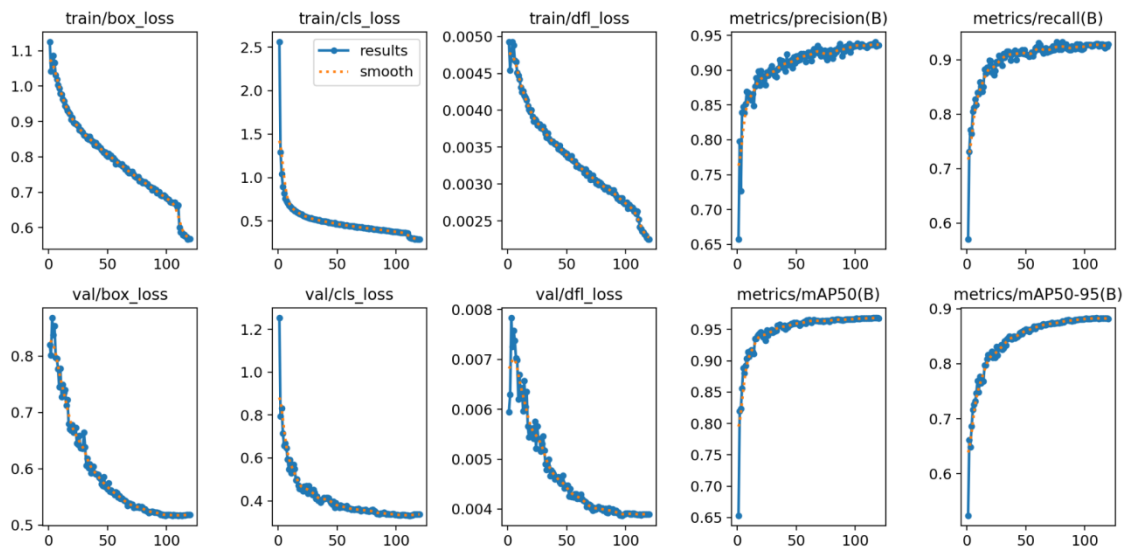
Hình 4.5: Tốc độ thực thi cấu trúc Simple_GLKA trên CPU

Trên thiết bị nhúng, sự chênh lệch giữa hai định dạng thể hiện rõ hơn đáng kể. Tại Raspberry Pi 5, việc chuyển đổi sang định dạng ONNX Runtime mang lại tốc độ tăng xấp xỉ 4 đến 5 lần so với PyTorch trên tất cả các biến thể, phản ánh hiệu quả của tối ưu hóa đồ thị tính toán và loại bỏ chi phí thông dịch của Python runtime. Cụ thể, mô hình CNN cơ sở đạt mức tăng tốc lớn nhất khi chuyển sang ONNX, trong khi GLKA34 cho thấy sự cân bằng tốt giữa độ phức tạp và tốc độ, đạt hiệu suất cao hơn các biến thể cùng số lượng tham số như GLKA+ và GLKAx trên cả hai nền tảng nhúng. Tại Raspberry Pi 4, xu hướng tương tự được quan sát với mức tăng tốc tương đương, tuy nhiên tốc độ tuyệt đối thấp hơn đáng kể so với Pi 5 do kiến trúc vi xử lý kém mạnh hơn. Kết quả này cho thấy định dạng ONNX Runtime

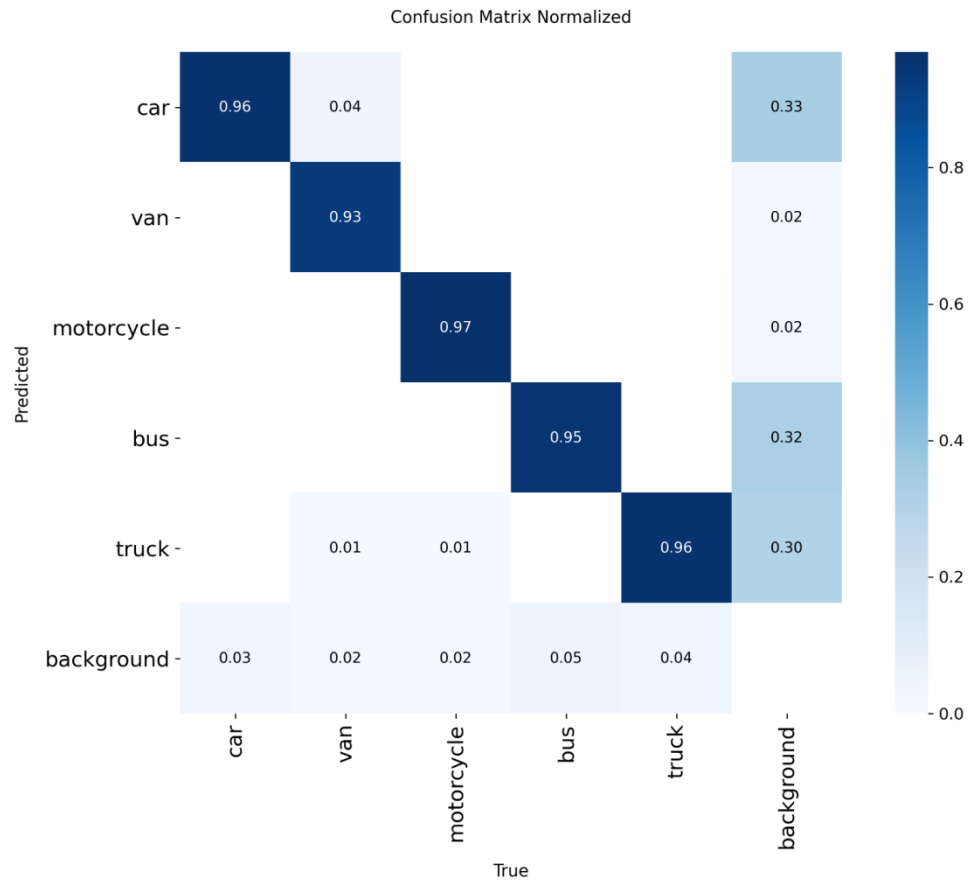
là lựa chọn phù hợp cho triển khai thực tế trên các thiết bị biên có tài nguyên hạn chế.

4.1.3 Kết quả huấn luyện YOLOv26n

Quá trình huấn luyện diễn ra ổn định trong 120 epoch. Các đường mất mát huấn luyện và kiểm định ở cả ba thành phần hộp giới hạn, phân loại và phân phối đều giảm đều và hội tụ tốt, không có dấu hiệu khớp quá mức rõ rệt. Đường Precision và Recall tăng nhanh trong giai đoạn đầu, sau đó chậm dần và bão hòa ở giai đoạn cuối. Chỉ số mAP@50 và mAP@50-95 cũng thể hiện xu hướng cải thiện liên tục từ đầu đến cuối quá trình huấn luyện.



Hình 4.6: Kết quả huấn luyện Yolov26n



Hình 4.7: Ma trận nhầm lẫn của mô hình sau huấn luyện

Bảng 4.2: Thống kê chi tiết các chỉ số qua các mốc epoch tiêu biểu.

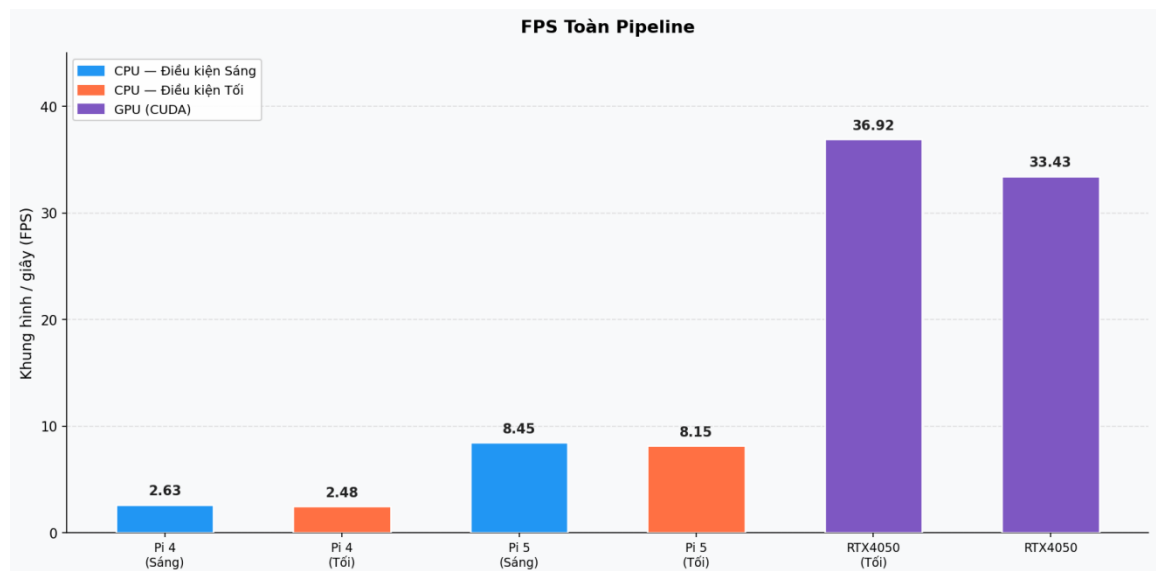
Epoch	Precision	Recall	F1-score	mAP@50	mAP@50-95
10	0,8627	0,8377	0,8500	0,9063	0,7480
30	0,9003	0,8898	0,8950	0,9449	0,8261
50	0,9200	0,9104	0,9152	0,9610	0,8625
70	0,9304	0,9155	0,9229	0,9631	0,8730
100	0,9349	0,9262	0,9305	0,9668	0,8818
120	0,9357	0,9283	0,9319	0,9678	0,8822

Ở epoch 10, mô hình còn ở giai đoạn học ban đầu với Precision đạt 0,8627 và mAP@50 đạt 0,9063. Từ epoch 10 đến epoch 50, các chỉ số tăng trưởng nhanh và

rõ rệt, $mAP@50$ tăng từ 0,9063 lên 0,9610 và $mAP@50-95$ tăng từ 0,7480 lên 0,8625, phản ánh giai đoạn mô hình học đặc trưng hiệu quả nhất. Từ epoch 50 trở đi, tốc độ cải thiện chậm dần, các chỉ số tiếp tục tăng nhưng với biên độ nhỏ hơn đáng kể. Đến epoch 120, mô hình đạt hiệu suất cao nhất toàn bộ quá trình huấn luyện với $mAP@50$ đạt 0,9678, F1-score đạt 0,9319 và $mAP@50-95$ đạt 0,8822, cho thấy mô hình đã hội tụ ổn định và không còn cải thiện đáng kể.

4.2 Hiệu năng xử lý và tiêu thụ năng lượng theo thời gian thực

Để đánh giá hiệu năng thực tế của hệ thống nhận diện biển báo giao thông, nhóm đã tiến hành đo điểm chuẩn chu kỳ xử lý trên ba nền tảng phần cứng khác nhau gồm Raspberry Pi 4, Raspberry Pi 5 và GPU RTX 4050, trong cả hai điều kiện ánh sáng là đủ sáng và tối. Kết quả được tổng hợp qua các biểu đồ dưới đây, phản ánh toàn diện thời gian xử lý, tốc độ khung hình và mức độ phân bổ tài nguyên theo từng giai đoạn của chu kỳ xử lý. Hình 4.8 dưới đây là bảng so sánh tốc độ xử lý trong một giây của 3 thiết bị Raspberry Pi4, Pi5 và RTX4050 với hai điều kiện ánh sáng khác nhau.

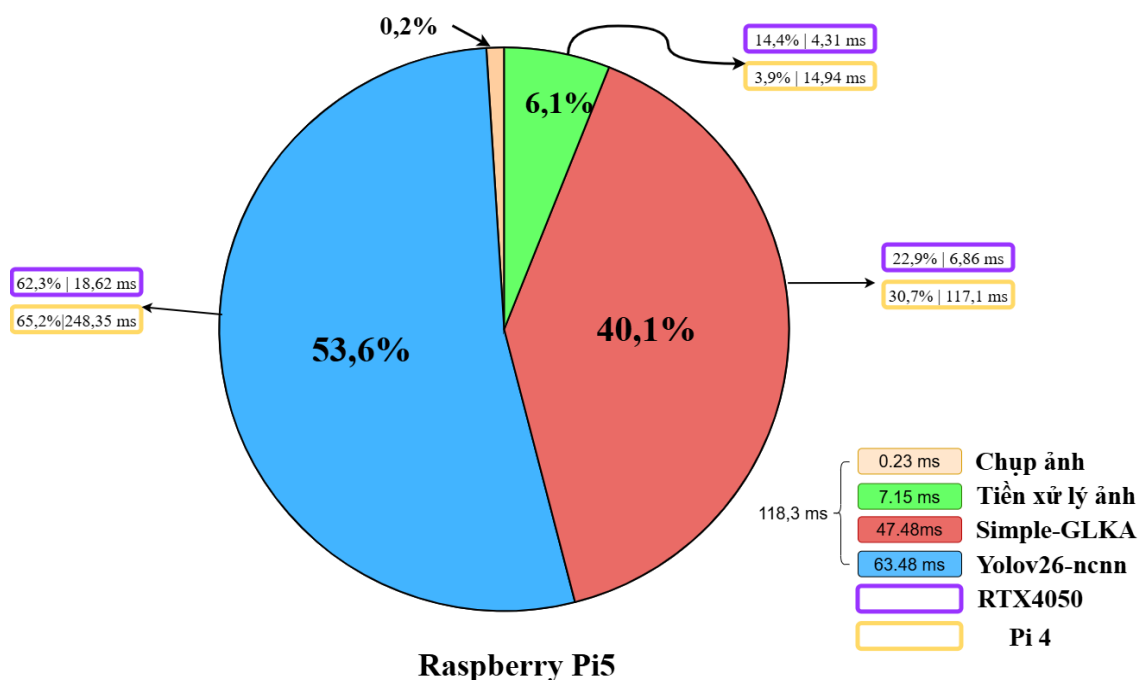


Hình 4.8: Thời gian xử lý một chu kỳ của Raspberry Pi 5

Ở nhóm thiết bị nhúng, Raspberry Pi 4 chỉ đạt 2,63 FPS trong điều kiện sáng và 2,48 FPS trong điều kiện tối, tức chưa đến 3 khung hình mỗi giây, gần như không đáp ứng được yêu cầu nhận diện liên tục trong thực tế. Raspberry Pi 5 cải thiện đáng kể khi đạt 8,45 FPS (sáng) và 8,15 FPS (tối), nhanh hơn Pi 4 khoảng 3 lần, tuy nhiên vẫn còn khá giới hạn cho các tình huống giao thông có tốc độ cao.

Điểm đáng chú ý là cả Pi 4 và Pi 5 đều sử dụng CPU để xử lý toàn bộ chu kỳ, khiến giai đoạn tiền xử lý ảnh cũng trở thành một nút thắt cộng hưởng cùng với hai mô hình học sâu. Khi điều kiện ánh sáng thay đổi, chất lượng ảnh đầu vào thay đổi theo và CPU phải gánh thêm khối lượng tính toán ở bước chuẩn hóa, dẫn đến FPS giảm nhẹ trong điều kiện tối.

Ngược lại, RTX 4050 đạt 33,43 FPS (sáng) và 36,92 FPS (tối) nhờ khả năng xử lý song song của GPU. Quan trọng hơn, RTX 4050 gần như không bị ảnh hưởng bởi điều kiện ánh sáng vì giai đoạn tiền xử lý chỉ chiếm 4,31 ms, tức không phải nút thắt của hệ thống. Toàn bộ tốc độ xử lý của GPU phụ thuộc chủ yếu vào giai đoạn YOLO, và khi ảnh tối có ít đặc trưng cần phát hiện hơn thì YOLO hoàn thành nhanh hơn, giải thích tại sao FPS ở điều kiện tối lại cao hơn sáng trên nền tảng này.



Hình 4.9: Thời gian xử lý một chu kỳ

Giai đoạn chụp ảnh gần như không tạo ra gánh nặng xử lý, chỉ chiếm 0,2% tương đương 0,23 ms. Tiền xử lý ảnh tiêu tốn thêm 6,1% với 7,15 ms cho các bước chuẩn hóa đầu vào. Phần tính toán tập trung hoàn toàn ở hai mô hình học sâu: Simple-GLKA chiếm 40,1% với 47,48 ms và YOLOv26-ncnn chiếm 53,6% với 63,48 ms, cho thấy tác vụ phát hiện vật thể là nút thắt chính của toàn bộ chu kỳ trên nền tảng CPU nhúng.

So sánh với hai nền tảng còn lại được thể hiện trực tiếp trên biểu đồ, ở giai đoạn Simple-GLKA, RTX 4050 chỉ mất 6,86 ms (22,9%) trong khi Pi 4 tốn tới 117,1 ms (30,7%), tức Pi 4 chậm hơn Pi 5 146,5% và RTX 4050 nhanh hơn Pi 5 tới 85,5%. Chênh lệch còn rõ hơn ở giai đoạn YOLOv26-ncnn: RTX 4050 hoàn thành trong 18,62 ms (62,3%) so với 248,35 ms (65,2%) của Pi 4, tức Pi 4 chậm hơn Pi 5 tới 291,2% trong khi RTX 4050 nhanh hơn 70,7%. Tương tự, giai đoạn tiền xử lý của RTX 4050 là 4,31 ms (14,4%) so với 14,94 ms (3,9%) của Pi 4, phản ánh lợi thế rõ ràng của GPU trong các phép tính song song.

Nhìn tổng thể, Raspberry Pi 5 đạt mức hiệu năng trung gian, vượt trội so với Pi 4 nhưng còn khoảng cách đáng kể so với RTX 4050. Nút thắt cổ chai nhất quán trên cả ba nền tảng đều nằm ở YOLOv26-ncnn, đây là giai đoạn cần ưu tiên tối ưu nếu muốn cải thiện tốc độ xử lý thời gian thực của hệ thống.

Bảng 4.3: Thông kê độ ổn định của các thiết bị.

Chỉ số	Trung bình (ms)	Độ lệch chuẩn (ms)	Nhanh nhất (ms)	Chậm nhất (ms)	FPS
Pi 4	380,88	18,31	351,71	467,07	2,63
Pi 5	118,33	7,22	106,31	157,02	8,45
RTX4050	29,91	4,44	23,42	66,73	33,43

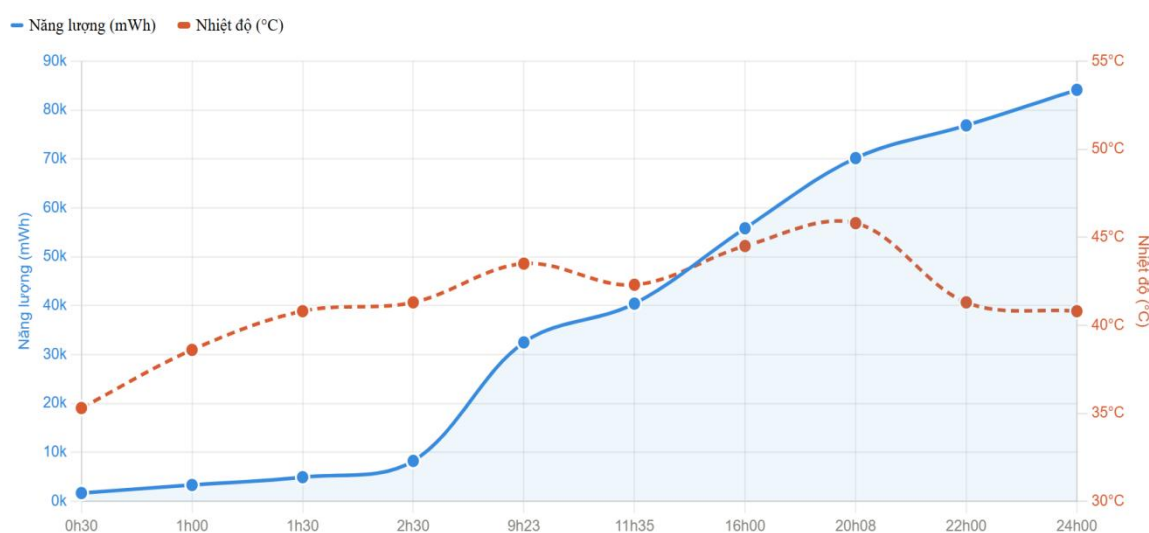
RTX 4050 dẫn đầu toàn diện với thời gian xử lý trung bình chỉ 29,91 ms và độ lệch chuẩn thấp nhất ở mức 4,44 ms, cho thấy hệ thống hoạt động rất nhất quán giữa các lần xử lý. Khoảng dao động từ 23,42 ms đến 66,73 ms là chấp nhận được, trong đó giá trị chậm nhất tuy có cao hơn trung bình nhưng chỉ xảy ra ở một số ít lần gặp khung hình phức tạp và không ảnh hưởng đáng kể đến hiệu năng tổng thể.

Raspberry Pi 5 đạt mức trung bình 118,33 ms với độ lệch chuẩn 7,22 ms, phản ánh hệ thống hoạt động khá ổn định trên nền CPU. Khoảng cách giữa lần nhanh nhất 106,31 ms và chậm nhất 157,02 ms là khoảng 50 ms, mức dao động này nằm trong tầm kiểm soát và không có hiện tượng spike đột ngột đáng lo ngại.

Raspberry Pi 4 là nền tảng kém ổn định nhất với độ lệch chuẩn lên tới 18,31 ms và khoảng dao động trải rộng từ 351,71 ms đến 467,07 ms, tức chênh lệch giữa lần nhanh nhất và chậm nhất lên tới 115 ms. Mức dao động lớn này cho thấy hiệu năng của Pi 4 không ổn định, phụ thuộc nhiều vào trạng thái tức thời của hệ thống và khó đảm bảo tính nhất quán trong môi trường vận hành thực tế.

4.3 Năng lượng tiêu thụ của hệ thống

Pi5 hoạt động ổn định trong suốt 24 giờ với mức tiêu thụ điện thấp và nhiệt độ nằm trong ngưỡng an toàn. Được minh chứng qua hình 4.10 dưới đây, số liệu được đo đạc qua thiết bị chuyên dụng dành cho đo công suất thấp.



Hình 4.10: Điện năng tiêu thụ của Raspberry Pi5 trong 24h

Về tiêu thụ điện, ở chế độ chờ dòng chỉ 0.57A (~2.98W), khi chạy suy luận AI tăng lên 0.70–0.73A (~3.65–3.81W), chênh lệch chưa đến 1W so với chế độ chờ, cho thấy chuỗi xử lý ảnh và nhận dạng được tối ưu tốt. Tải cao nhất xuất hiện khi có truy cập web và luồng camera với 0.90–0.95A (~4.7–4.96W), tức nút thắt cổ chai đến từ vào/ra dữ liệu chứ không phải tính toán.

Về năng lượng tích lũy, thiết bị duy trì tốc độ tăng gần như hằng số ~3.5k mWh/giờ xuyên suốt 24 giờ, kết thúc ở ~84k mWh. Đường tích lũy hoàn toàn tuyến tính, phản ánh mức tải hệ thống ổn định và không có sự cố bất thường trong toàn bộ quá trình vận hành.

Về điện năng tiêu thụ, Pi5 chỉ ngốn trung bình khoảng 3.7W khi chạy suy luận AI liên tục, một con số cực kỳ nhỏ. Để dễ hình dung, một chiếc laptop thông thường

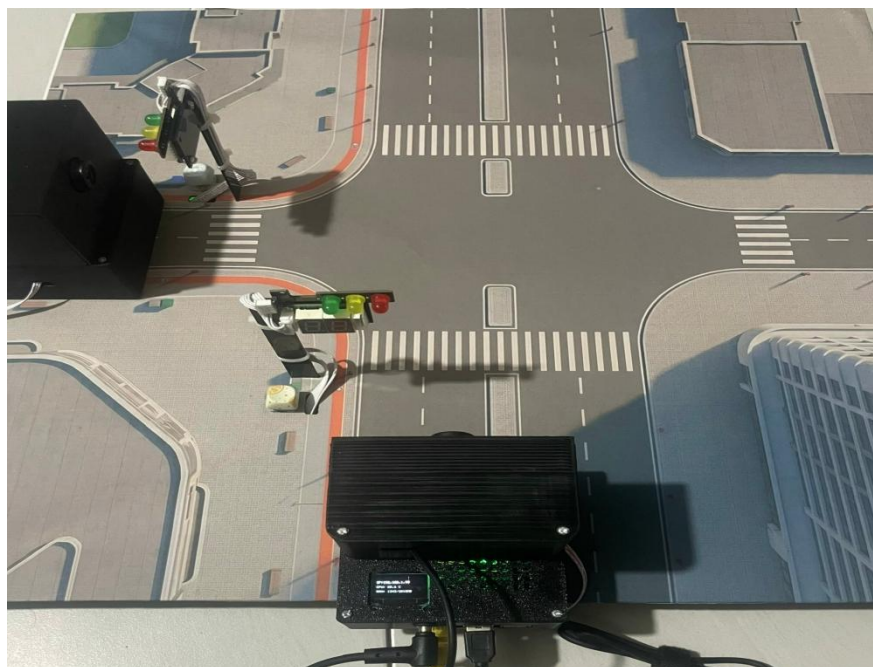
tiêu thụ khoảng 45 đến 65W, còn một chiếc TV LED 43 inch vào khoảng 80 đến 100W, tức Pi5 vận hành toàn bộ hệ thống nhận dạng phương tiện và điều khiển đèn tín hiệu chỉ bằng chưa đến một phần mười lượng điện so với TV đang bật trong phòng khách.

Về nhiệt độ, nhờ được trang bị quạt tản nhiệt chủ động, nhiệt độ được kiểm soát tốt với mức cao nhất chỉ đạt $\sim 46^{\circ}\text{C}$ lúc 20h08 và duy trì ổn định trong khoảng 40 đến 46°C suốt một ngày. Pi5 bắt đầu giảm xung ở 85°C nên còn dư gần 40°C so với ngưỡng giới hạn, đảm bảo thiết bị có thể vận hành liên tục cả ngày mà không lo quá nhiệt.

4.4 Kết quả thi công phần cứng

4.4.1 Kết quả mô hình sau khi thi công hoàn tất.

Hình 4.11 minh họa toàn cảnh mô hình đèn giao thông ở ngã tư sau khi hoàn tất thi công và lắp ráp phần cứng. Hệ thống xử lý trung tâm (Raspberry Pi 5 và camera) được bảo vệ gọn gàng trong hộp kỹ thuật, có tích hợp màn hình OLED theo dõi thông số. Hệ thống trụ đèn, bao gồm cụm LED giao thông và màn hình đếm ngược, được bố trí rõ ràng tại từng ngã rẽ để thuận tiện cho việc quan sát.

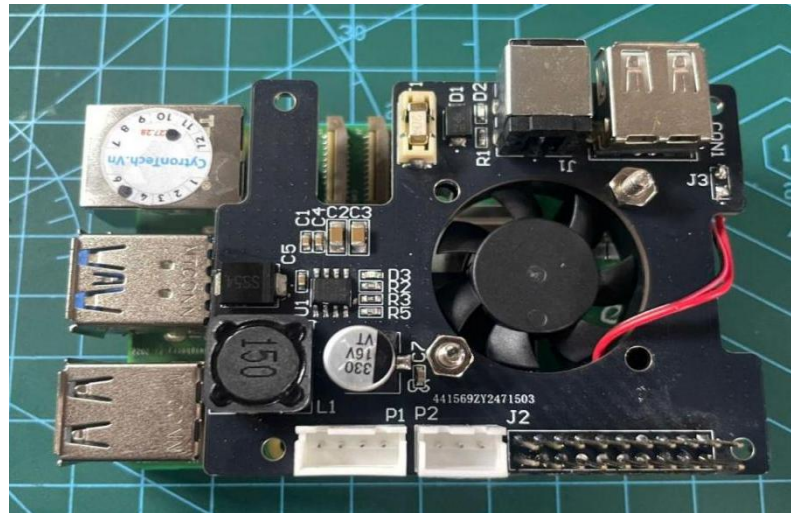


Hình 4.11: Mô hình đèn giao thông ngã tư

4.4.1 Kết quả thi công mạch nguồn Raspberry Pi 5

Để đảm bảo toàn bộ hệ thống vận hành ổn định, khối bo mạch nguồn hạ áp dành riêng cho Raspberry Pi 5 đã được thi công hoàn chỉnh và đưa vào thực nghiệm

để đánh giá hiệu năng toàn diện. Hình 4.12 là tổng thể board nguồn cấp cho Raspberry Pi 5 sau khi được thi công.



Hình 4.12: Board mạch nguồn cấp riêng cho Raspberry

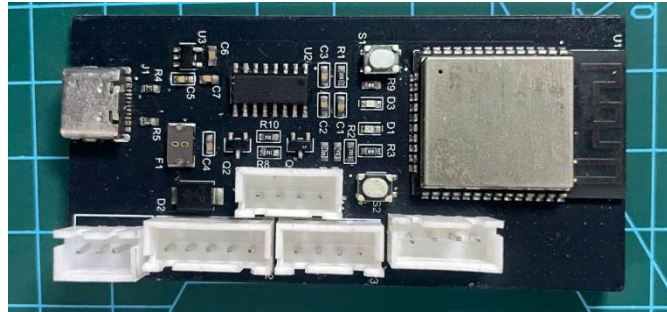
Sau khi cấp nguồn cho board, tiến hành kiểm tra chức năng theo từng bước đo: chạy không tải, tải nhẹ, tải nặng và vận hành trong thời gian dài. Bảng 4.4 tổng hợp kết quả kiểm thử của mạch nguồn.

Bảng 4.4: Kết quả kiểm tra và đánh giá mạch nguồn.

STT	Nội dung kiểm tra	Phương pháp kiểm tra	Điện áp ngõ ra thực tế	Kết quả
1	Chế độ không tải (Chỉ cấp nguồn cho board, chưa cắm Pi 5)	Dùng đồng hồ vạn năng đo tại nguồn ra.	5.12 V	Đạt
2	Chế độ tải nhẹ (Pi 5 ở trạng thái chờ)	Dùng đồng hồ vạn năng đo tại nguồn ra.	5.11 V	Đạt
3	Chế độ tải đỉnh (Pi 5 xử lý YOLOv26n + Simple_CNN)	Dùng đồng hồ vạn năng đo tại nguồn ra.	5.09 V	Đạt
4	Chế độ vận hành dài hạn (Hệ thống chạy liên tục trong 8 giờ)	Dùng đồng hồ vạn năng đo tại nguồn ra.	5.06 V	Đạt

4.4.2 Kết quả thi công mạch điều khiển ESP32

Song song với khối nguồn, khối mạch điều khiển sử dụng vi điều khiển ESP32 cũng đã được thi công hoàn chỉnh và đưa vào vận hành thực nghiệm để đánh giá hiệu năng toàn diện. Hình 4.13 là tổng thể board điều khiển sử dụng ESP32 sau khi được thi công.



Hình 4.13: Board mạch điều khiển giao thông

Sau khi cấp nguồn cho board, tiến hành kiểm tra từng khối chức năng theo phương pháp từng bước: đo điện áp nguồn trước, sau đó nạp firmware kiểm tra từng ngoại vi độc lập, cuối cùng chạy firmware tổng hợp toàn bộ luồng hoạt động của board điều khiển giao thông. Bảng 4.5 tổng hợp kết quả kiểm thử của mạch điều khiển giao thông.

Bảng 4.5: Kết quả kiểm tra và đánh giá của mạch điều khiển giao thông

Khối chức năng	Nội dung kiểm tra	Phương pháp kiểm tra	Kết quả
Khối nguồn	Điện áp ngõ ra đạt 3.3V	Đo bằng đồng hồ tại chân Vout của IC nguồn	Đạt
USB-to-UART	Máy tính nhận diện cổng COM	Kết nối USB, quan sát Device Manager	Đạt
Khối xử lý trung tâm	ESP32 khởi động và nạp firmware thành công	Nạp chương trình led sáng và tắt trong 1 giây	Đạt
Khối hiển thị	Các LED 7 đoạn hiển thị đúng các con số, không bị chớp nháy, đứt nét.	Nạp chương trình đếm ngược luân phiên từ 99 đến 00.	Đạt
Khối thực thi	Các module LED giao thông sáng/tắt đúng.	Nạp chương trình mô phỏng chu kỳ đèn.	Đạt

4.5 Kết quả hoạt động của hệ thống giao thông.

Để kiểm chứng tính chính xác của thuật toán điều khiển thích ứng thời gian thực và khả năng phối hợp đồng bộ giữa Raspberry Pi 5 và khối điều khiển ESP32. Các kịch bản thử nghiệm được chia làm hai phân hệ chính bao gồm: Chế độ tự động thích ứng dựa trên mật độ phương tiện và Chế độ ưu tiên, xử lý kẹt xe khẩn cấp.

Trước khi tiến hành các bước thử nghiệm, các thông số cấu hình chu kỳ thời gian phân pha cho từng chế độ vận hành (từ m0 đến m4) đã được thiết lập sẵn trong giải thuật và lưu trữ trên bộ nhớ của khối điều khiển. Các giá trị định thời tiêu chuẩn này đóng vai trò làm cơ sở dữ liệu đối chứng cho toàn bộ quá trình đánh giá thực nghiệm, được thể hiện chi tiết trong Bảng 4.6.

Bảng 4.6: Các chế độ vận hành của hệ thống giao thông.

Chế độ vận hành (mode)	Ngưỡng mật độ xe (xe)	Thời gian đèn xanh (giây)	Thời gian đèn đỏ (giây)	Trạng thái giao thông
m0	Lúc mới khởi động	27	30	Khởi tạo hệ thống
m1	≤ 2	17	20	Lưu lượng xe ít
m2	≤ 5	42	45	Lưu lượng xe ổn định
m3	≤ 8	57	60	Lưu lượng tăng cao
m4	> 8	87	90	Lưu lượng xe giờ cao điểm.

4.5.1 Kết quả vận hành ở chế độ tự động thích ứng (MODE m0 - m4).

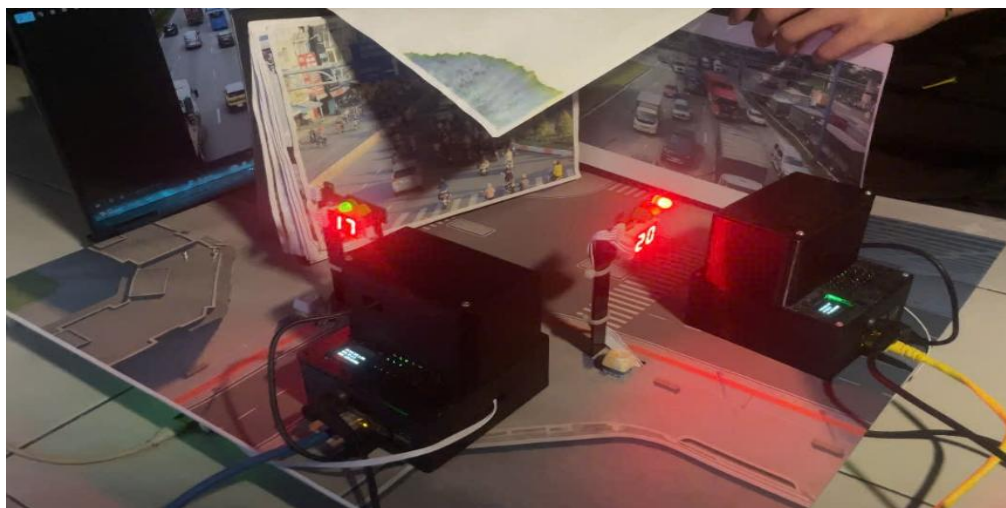
Trong kịch bản này, hệ thống vận hành hoàn toàn tự động. Camera sẽ chụp và gửi luồng dữ liệu hình ảnh về Raspberry Pi 5 để đếm số lượng xe thực tế. Dựa trên số lượng xe đếm được, Raspberry Pi 5 sẽ tính toán đưa ra chế độ thời gian đèn tối ưu từ m1 đến m4 và truyền lệnh xuống vi điều khiển ESP32 qua giao tiếp UART để thay đổi trạng thái các cụm LED giao thông và LED 7 đoạn.

Chế độ khởi động an toàn (Mode m0): Khi hệ thống vừa cấp nguồn, ESP32 tự động chạy chu kỳ đèn mặc định (m0) nhằm duy trì tín hiệu điều tiết giao thông trong lúc chờ Raspberry Pi 5 khởi động và nạp mô hình AI. Khi thời gian đếm ngược của pha Xanh còn 3 giây hoặc pha Đỏ còn 5 giây, ESP32 chủ động gửi tín hiệu "run" qua UART. Nhận được lệnh, Raspberry Pi 5 lập tức kích hoạt camera chụp ảnh, phân tích mật độ phương tiện và truyền lệnh xuống mới xuống ESP32. Hình 4.14 thể hiện hệ thống đang chạy ở chế độ m0.



Hình 4.14: Hệ thống ở trạng thái m0

Chế độ tự động (Mode m1): Kịch bản kiểm thử với mật độ phương tiện thấp (ngưỡng dưới 2 xe). Khi đưa ảnh giả lập vào camera, Raspberry Pi 5 nhận diện chính xác 1 phương tiện và kích hoạt trạng thái m1. Lệnh điều khiển lập tức được truyền qua UART xuống ESP32 để thiết lập chu kỳ đèn rút ngắn (Xanh 17 giây, Đỏ 20 giây). Hình 4.15 thể hiện hệ thống đang chạy ở chế độ m1.

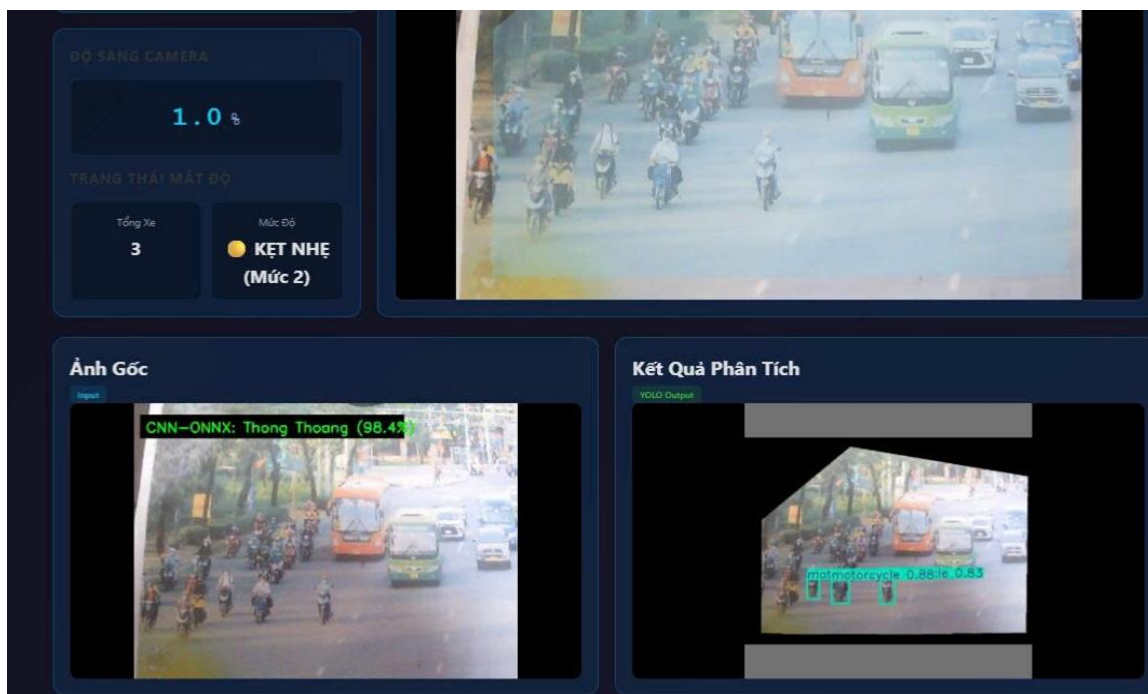


Hình 4.15: Hệ thống ở trạng thái m1

Chế độ tự động (Mode m2): Kích bản kiểm thử với mật độ phương tiện mức trung bình (ngưỡng 2 - 5 xe). Khi đưa ảnh vào camera, Raspberry Pi 5 nhận diện chính xác 3 phương tiện và thiết lập trạng thái m2. Lệnh điều khiển lập tức truyền qua UART xuống ESP32 để tăng chu kỳ đèn (Xanh 42s, Đỏ 45s) nhằm giải tỏa giao lộ nhanh hơn so với m1. Hình 4.16 và 4.17 mô tả hệ thống quét được 3 xe và chạy chế độ m2.



Hình 4.16: Hệ thống ở trạng thái m2

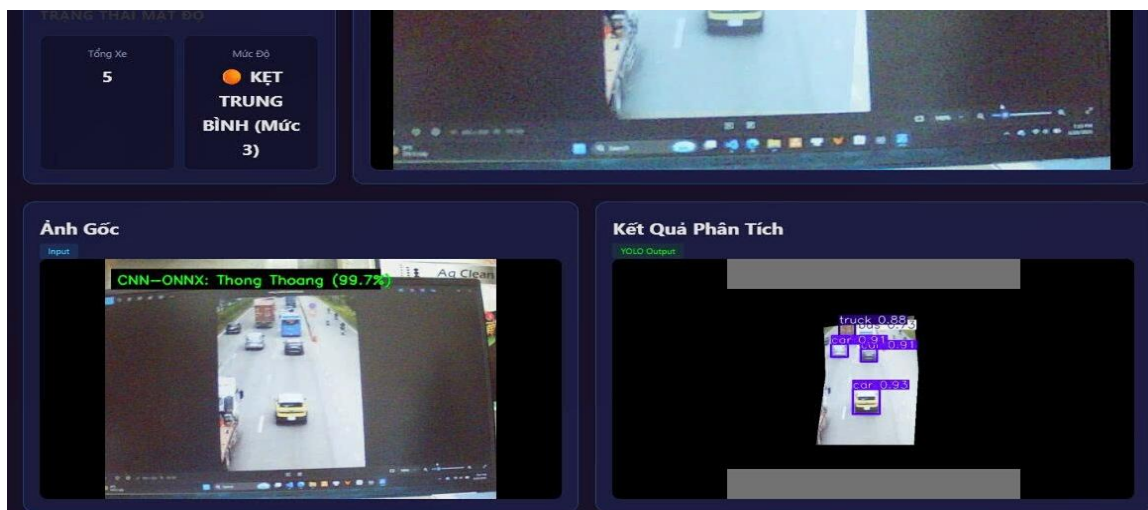


Hình 4.17: Kết quả nhận diện phương tiện ở mode m2

Chế độ tự động (Mode m3): Kích bản kiểm thử với mật độ phương tiện cao (ngưỡng 5 - 8 xe). Khi đưa ảnh vào camera, Raspberry Pi 5 nhận diện chính xác 5 phương tiện và kích hoạt trạng thái m3. Lệnh điều khiển lập tức được truyền qua UART xuống ESP32 để mở rộng chu kỳ đèn (Xanh 57 giây, Đỏ 60 giây) nhằm giải phóng nhanh dòng xe tắc nghẽn. Hình 4.18 và 4.19 mô tả hệ thống quét được 5 xe và đang chạy ở trạng thái m3.



Hình 4.18: Hệ thống ở trạng thái m3



Hình 4.19: Kết quả nhận diện phương tiện ở mode m3

Chế độ tự động (mode m4): Kịch bản kiểm thử tình trạng ùn tắc cực đoạn với ngưỡng giới hạn từ 8 xe trở lên. Khi đưa ảnh giả lập giao thông kẹt cứng vào camera, hệ thống AI nhận diện chính xác 11 xe, lập tức kích hoạt chu kỳ đèn dài nhất (m4) và truyền lệnh xuống vi điều khiển ESP32.



Hình 4.20: Hệ thống ở trạng thái m4

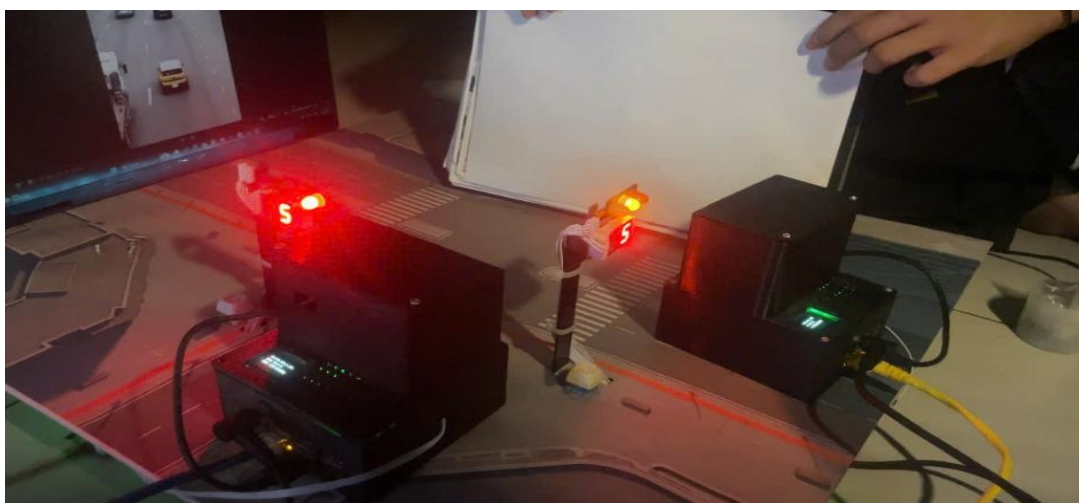
4.5.2 Kết quả vận hành ở chế độ ưu tiên

Bên cạnh chu trình tự động điều tiết theo bốn cấp độ mật độ xe, tiến hành kiểm thử với các tình huống nhận lệnh khẩn cấp, nhằm đánh giá khả năng hoạt động của hệ thống.

Hình 4.21 và 4.22 mô tả hệ thống khi kích hoạt trạng thái ưu tiên cần xả kệt cho làn đường bị ùn tắc và thoát chế độ xả kệt ưu tiên.



Hình 4.21: Hệ thống ở chế độ ưu tiên



Hình 4.22: Hệ thống thoát khỏi chế độ ưu tiên

Để minh chứng cho độ ổn định của hệ thống ở chế độ ưu tiên, nhóm đã thực hiện kích hoạt Chế độ ưu tiên lặp lại 10 lần liên tục.

Bảng 4.7: Bảng kiểm tra hoạt động của chế độ Ưu tiên.

Số lần test	Lệnh kích hoạt	Phản hồi tại Trạm A (tín hiệu đèn)	Phản hồi tại Trạm B (Tín hiệu đèn)	Thoát khỏi chế độ Ưu Tiên (tín hiệu đèn)	Kết quả
1	Ưu tiên Trạm A	Xanh	Đỏ	Đồng bộ	Đạt
2	Ưu tiên Trạm B	Đỏ	Xanh	Đồng bộ	Đạt
3	Ưu tiên Trạm A	Xanh	Đỏ	Đồng bộ	Đạt
4	Ưu tiên Trạm A	Xanh	Đỏ	Đồng bộ	Đạt
5	Ưu tiên Trạm B	Đỏ	Xanh	Không đồng bộ	Chưa đạt
6	Ưu tiên Trạm B	Đỏ	Xanh	Đồng bộ	Đạt
7	Ưu tiên Trạm A	Xanh	Đỏ	Đồng bộ	Đạt
8	Ưu tiên Trạm B	Đỏ	Xanh	Đồng bộ	Đạt
9	Ưu tiên Trạm A	Xanh	Đỏ	Đồng bộ	Đạt
10	Ưu tiên Trạm B	Đỏ	Xanh	Đồng bộ	Đạt

Qua 10 lần kiểm thử liên tục, hệ thống đạt tỷ lệ phản hồi và đồng bộ thành công ở mức 90% (9/10 lần). Hệ thống hoạt động tuyệt đối chính xác, luôn đảm bảo trạm yêu cầu ưu tiên sáng đèn xanh và trạm còn lại bị ép chuyển đỏ ngay lập tức để tránh xung đột tai nạn. Tuy nhiên, hiện tượng mất đồng bộ tín hiệu thỉnh thoảng vẫn xảy ra ngẫu nhiên ở khâu thoát chế độ ưu tiên (ghi nhận tại lần 5 ở Trạm B. Sự cố này có thể xuất phát từ phần cứng ESP32 do kết nối vật lý của đường truyền nối tiếp UART giữa Raspberry Pi 5 và ESP32 chưa thực sự ổn định dẫn đến hiện tượng lỏng dây và gây sai lệch gói tin.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đề án đã nghiên cứu, thiết kế và triển khai thành công hệ thống điều khiển giao thông thông minh ứng dụng Edge AI trên nền tảng Raspberry Pi 5, tích hợp mô hình phân loại mật độ Simple_GLKA, mô hình phát hiện và đếm phương tiện giao thông YOLOv26n và mô hình tiền xử lý ánh sáng SCI, hoạt động ổn định theo thời gian thực tại nút giao thông. Các mục tiêu nghiên cứu cụ thể đã được hoàn thành bao gồm:

Thứ nhất, về thiết kế khối chú ý thị giác chuyên biệt cho bài toán phân loại mật độ giao thông: Mô-đun GLKA (Global Large Kernel Attention) được xây dựng theo hướng mở rộng vùng cảm nhận hiệu dụng (ERF) đa quy mô từ 3×3 lên 13×13 thông qua bốn luồng tích chập sâu giãn song song với tỉ lệ $r = \{1, 3, 2, 3\}$, kết hợp cơ chế chú ý kênh SE song song nhằm tối ưu hóa đặc trưng theo cả hai chiều không gian và kênh. Thiết kế này cho phép mô hình nắm bắt trọn vẹn sự phân bố phương tiện trên toàn khung hình, đặc tính then chốt để phân biệt chính xác các mức độ mật độ giao thông.

Thứ hai, về cải tiến kiến trúc mạng nơ-ron tích chập siêu nhẹ Simple_GLKA: Trên nền SimpleEfficientCNN, khối GLKA cải tiến được nhúng vào Block 2 đến Block 4, đồng thời áp dụng kỹ thuật tái tham số hóa cấu trúc để hợp nhất toàn bộ bốn nhánh tích chập sâu giãn và các lớp Batch Normalization thành một lớp tích chập đơn kích thước 13×13 khi triển khai. Quá trình hợp nhất đạt sai số thực nghiệm dưới 10^{-5} , loại bỏ hoàn toàn chi phí tính toán của các nhánh dư thừa trong khi vẫn bảo toàn toàn bộ năng lực biểu diễn đặc trưng đã học.

Thứ ba, về triển khai hệ thống Edge AI theo thời gian thực: Hệ thống liên kết hoàn chỉnh Simple_GLKA, YOLOv26n và mô-đun SCI thành một chu kì điều khiển giao thông thống nhất, vận hành ổn định trong các điều kiện ánh sáng thay đổi và đảm bảo hiệu năng mượt mà trực tiếp trên phần cứng giới hạn năng lượng Raspberry Pi 5 mà không phát sinh quá tải bộ nhớ.

5.1.1 Ưu điểm

Về độ chính xác phân loại, kiến trúc Simple_GLKA với mô-đun GLKA tái tham số hóa cấu trúc đạt 98,99%, tăng 0,90 điểm phần trăm so với mô hình cơ sở SimpleEfficientCNN (98,09%), đồng thời giảm 60% giá trị mất mát trên tập kiểm thử (từ 0,093 xuống 0,037).

Về hiệu năng triển khai, toàn bộ chuỗi xử lý đạt 8,45 FPS trong điều kiện sáng và 8,15 FPS trong điều kiện tối trên Raspberry Pi 5, nhanh hơn Pi 4 khoảng 3 lần và đáp ứng yêu cầu xử lý thời gian thực. Trong cấu trúc chu kỳ xử lý, mô-đun Simple_GLKA chiếm 40,1% thời gian và tiền xử lý SCI chiếm 6,1%, cho thấy chu kỳ xử lý được phân bổ hợp lý trên phần cứng nhúng hạn chế tài nguyên.

Về khả năng thích ứng ánh sáng, mô-đun SCI chỉ sử dụng Enhance Network khi suy luận đã khắc phục một phần đáng kể vấn đề thiếu sáng và ngược sáng, duy trì hiệu năng ổn định giữa ban ngày và ban đêm, thể hiện qua mức FPS gần như không thay đổi giữa hai điều kiện chiếu sáng.

5.1.2 Nhược điểm

Thứ nhất, lớp xe van có số lượng mẫu huấn luyện thấp nhất trong tập dữ liệu với 1.338 mẫu, phân bố chưa đủ đa dạng về góc độ, điều kiện ánh sáng và bối cảnh xuất hiện, dẫn đến tỷ lệ nhầm lẫn cao hơn so với các lớp còn lại và làm tăng nguy cơ quá khớp; do đó cần bổ sung và mở rộng thêm dữ liệu cho lớp này trong các nghiên cứu tiếp theo.

Thứ hai, chất lượng dữ liệu kiểm định còn hạn chế do ảnh đầu vào được thu thập từ camera chất lượng thấp, sau đó in ra giấy và chụp lại để tạo tập kiểm định, khiến chất lượng ảnh suy giảm đáng kể so với điều kiện thực tế triển khai. Điều này dẫn đến kết quả đo kiểm chưa phản ánh đầy đủ hiệu suất thực của mô hình trong môi trường vận hành thực tế.

Thứ ba, mô-đun tiền xử lý SCI chưa được kiểm định trong các điều kiện thực địa khắc nghiệt như mưa lớn, rung lắc camera mạnh hoặc môi trường có nhiều nguồn nhiễu phức tạp, do đó khả năng tổng quát hóa của mô-đun này trong thực tế vẫn chưa được xác nhận đầy đủ.

Thứ tư, tầng phân loại mật độ hiện chỉ xử lý ảnh tĩnh mà chưa khai thác thông tin thời gian, khiến hệ thống chưa phân biệt được trường hợp mật độ phương tiện cao nhưng vẫn lưu thông bình thường với tình trạng ùn tắc thực sự, dễ dẫn đến phân loại sai trong các tình huống giao thông phức tạp.

5.2 Hướng phát triển

Xuất phát từ hạn chế cuối nêu trên, hướng phát triển trọng tâm là nâng cấp tầng phân loại mật độ sang mô hình có khả năng hiểu ngữ cảnh thời gian. Thay vì phân tích từng khung hình đơn lẻ, hệ thống cần học đồng thời từ đặc trưng ngoại hình của khung hình hiện tại và thông tin chuyển động giữa các khung hình liên tiếp, từ đó phân biệt được ùn tắc thực sự (mật độ cao, tốc độ di chuyển thấp hoặc dừng hẳn) với mật độ cao tạm thời (xe dày nhưng vẫn lưu thông bình thường).

Để giải quyết bài toán này, đề án đề xuất kiến trúc GLKAFlow, một mô hình hai nhánh xử lý song song hai luồng đặc trưng bổ trợ nhau. Nhánh thứ nhất là nhánh ngoại hình, sử dụng AppearanceBackbone với các khối EfficientBlock tích hợp mô-đun GLKA để trích xuất đặc trưng không gian từ khung hình hiện tại. Nhánh thứ hai là nhánh chuyển động, sử dụng VelocityBranch với các khối tích chập tách chiều (depthwise separable convolution) nhẹ để mã hóa thông tin động học từ ảnh hiệu giữa hai khung hình liên tiếp. Hai luồng đặc trưng được hợp nhất qua mô-đun MergeFusion trước khi đưa vào đầu phân loại chung.

Kiến trúc này kế thừa trực tiếp mô-đun GLKA với cơ chế tái tham số hóa cấu trúc từ Simple_GLKA, giữ nguyên ưu điểm về tốc độ suy luận khi triển khai, đồng thời bổ sung khả năng phân tích chuyển động mà mô hình một nhánh không có. Đầu vào của GLKAFlow là cặp khung hình liên tiếp trích xuất từ video, phù hợp với luồng dữ liệu thực tế của camera giám sát giao thông.

Tài Liệu Tham Khảo

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” May 09, 2016, *arXiv*: arXiv:1506.02640. doi: 10.48550/arXiv.1506.02640.
- [2] P. Hidayatullah, N. Syakrani, M. R. Sholahuddin, T. Gelar, and R. Tubagus, “YOLOv8 to YOLO11: A Comprehensive Architecture In-depth Comparative Review,” *J. RESTI Rekayasa Sist. Dan Teknol. Inf.*, vol. 10, no. 2, pp. 341–354, Apr. 2026, doi: 10.29207/resti.v10i2.6598.
- [3] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: 10.1109/5.726791.
- [4] M. D. Zeiler and R. Fergus, “Visualizing and Understanding Convolutional Networks,” Nov. 28, 2013, *arXiv*: arXiv:1311.2901. doi: 10.48550/arXiv.1311.2901.
- [5] “Ultralytics YOLO26,” Ultralytics Docs. Accessed: Jun. 15, 2026. [Online]. Available: <https://docs.ultralytics.com/models/yolo26>
- [6] S. Chakrabarty, “YOLO26: An Analysis of NMS-Free End to End Framework for Real-Time Object Detection,” Mar. 18, 2026, *arXiv*: arXiv:2601.12882. doi: 10.48550/arXiv.2601.12882.
- [7] R. Sapkota, R. H. Cheppally, A. Sharda, and M. Karkee, “YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection,” Mar. 16, 2026, *arXiv*: arXiv:2509.25164. doi: 10.48550/arXiv.2509.25164.
- [8] “Quick Start Guide: Raspberry Pi with Ultralytics YOLO26,” Ultralytics Docs. Accessed: Jun. 15, 2026. [Online]. Available: <https://docs.ultralytics.com/guides/raspberry-pi>
- [9] J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, “Squeeze-and-Excitation Networks,” May 16, 2019, *arXiv*: arXiv:1709.01507. doi: 10.48550/arXiv.1709.01507.

- [10] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “CBAM: Convolutional Block Attention Module,” Jul. 18, 2018, *arXiv*: arXiv:1807.06521. doi: 10.48550/arXiv.1807.06521.
- [11] Z. Liu and B. Jiang, “Revisiting the Ordering of Channel and Spatial Attention: A Comprehensive Study on Sequential and Parallel Designs,” Jan. 23, 2026, *arXiv*: arXiv:2601.07310. doi: 10.48550/arXiv.2601.07310.
- [12] X. Ding *et al.*, “UniRepLKNet: A Universal Perception Large-Kernel ConvNet for Audio, Video, Point Cloud, Time-Series and Image Recognition,” Mar. 18, 2024, *arXiv*: arXiv:2311.15599. doi: 10.48550/arXiv.2311.15599.
- [13] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, “RepVGG: Making VGG-style ConvNets Great Again,” Mar. 29, 2021, *arXiv*: arXiv:2101.03697. doi: 10.48550/arXiv.2101.03697.
- [14] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” Mar. 02, 2015, *arXiv*: arXiv:1502.03167. doi: 10.48550/arXiv.1502.03167.
- [15] X. Ding, X. Zhang, J. Han, and G. Ding, “Diverse Branch Block: Building a Convolution as an Inception-like Unit,” Mar. 29, 2021, *arXiv*: arXiv:2103.13425. doi: 10.48550/arXiv.2103.13425.
- [16] P. Wang *et al.*, “Understanding Convolution for Semantic Segmentation,” Jun. 01, 2018, *arXiv*: arXiv:1702.08502. doi: 10.48550/arXiv.1702.08502.
- [17] X. Ding, X. Zhang, Y. Zhou, J. Han, G. Ding, and J. Sun, “Scaling Up Your Kernels to 31x31: Revisiting Large Kernel Design in CNNs,” Apr. 02, 2022, *arXiv*: arXiv:2203.06717. doi: 10.48550/arXiv.2203.06717.
- [18] M.-H. Guo, C.-Z. Lu, Z.-N. Liu, M.-M. Cheng, and S.-M. Hu, “Visual Attention Network,” Jul. 11, 2022, *arXiv*: arXiv:2202.09741. doi: 10.48550/arXiv.2202.09741.
- [19] X. Wang *et al.*, “SimpleEfficientCNN: A Lightweight and Efficient Deep Learning Framework for High-Precision Rice Seed Classification,” *Agriculture*, vol. 16, no. 3, p. 357, Feb. 2026, doi: 10.3390/agriculture16030357.
- [20] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen,

“MobileNetV2: Inverted Residuals and Linear Bottlenecks,” Mar. 21, 2019, *arXiv*: arXiv:1801.04381. doi: 10.48550/arXiv.1801.04381.

[21] L. Ma, T. Ma, R. Liu, X. Fan, and Z. Luo, “Toward Fast, Flexible, and Robust Low-Light Image Enhancement,” Apr. 21, 2022, *arXiv*: arXiv:2204.10137. doi: 10.48550/arXiv.2204.10137.

[23] Espressif Systems, ESP32 Series Datasheet v5.2, [Online]. Available: https://www.espressif.com/documentation/esp32_datasheet_en.pdf

[24] Raspberry Pi, "Raspberry Pi computer hardware," Raspberry Pi Documentation. [Online]. Available: <https://www.raspberrypi.com/documentation/computers/getting-started.html>

[25] Microchip Technology Inc., "MIC5219 - 500mA-Peak Output LDO Regulator Datasheet," 2018, [Online]. Available: <https://ww1.microchip.com/downloads/en/DeviceDoc/MIC5219-500mA-Peak-Output-LDO-Regulator-DS20006021A.pdf>

[26] Texas Instruments, "TPS5450 5-A, Wide Input Range, Step-Down Converter Datasheet," Doc. No. SLVS757E, Mar. 2007, Rev. Jul. 2022. [Online]. Available: <https://www.ti.com/lit/ds/symlink/tps5450.pdf>

[27] Espressif Systems, ESP32-WROOM-32 Series Datasheet v3.4, [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf